

STEP-ICE-UP5K 实验指导书

小脚丫 STEP FPGA

STEP

--/--/--

STEP-ICE-UP5K 实验指导书

目录

第一章 实验平台	4
1.1 概述	5
1.2 产品布局与元件	6
1.2.1 开发板布局	6
1.2.2 FPGA 器件	6
1.2.3 编程配置	7
1.2.4 引脚	7
1.2.5 显示设备	7
1.2.6 电源	7
1.3 FPGA 引脚分配	7
第二章 开发工具的安装与注册	9
2.1. 概述	10
2.2. 软件安装	10
2.2.1 Radiant 软件下载	10
2.2.2 Radiant 安装步骤	10
2.2.3 Radiant 软件注册	15
2.3. 创建第一个工程	21
2.3.1 新建工程	21
2.3.2 添加设计文件	26
2.3.3 管脚约束	30
2.3.4 FPGA 加载	35
2.4 仿真工具 Modelsim	36
2.4.1 仿真流程	37
第三章 iCE40UP 入门实验	46
3.1 实验一 使用内部时钟和外部时钟点亮 RGB LED	48
3.1.1 实验任务	48
3.1.2 实验原理	48
3.1.3 Verilog HDL 建模描述	48
3.1.4 实验步骤	50
3.1.5 实验现象	50
3.2 实验二 UART 串口通讯实验	52
3.2.1 实验任务	52
3.2.2 实验原理	52

3.2.3 Verilog HDL 建模描述	54
3.2.4 实验步骤	67
3.2.5 实验现象	67
3.3 实验三 DS18B20 与 UART 通讯实验	69
3.3.1 实验任务	69
3.3.2 实验原理	69
3.3.3 Verilog HDL 建模描述	69
3.3.4 实验步骤	92
3.3.5 实验现象	92
第四章 DS18B20 温度传感器实验	94
4.1 实验四 DS18B20 温度传感器实验	99
4.1.1 实验任务	99
4.1.2 实验原理	99
4.1.3 Verilog HDL 建模描述	99
4.1.4 实验步骤	123
4.1.5 实验现象	123
第五章 基于比较器的数字电压表实验	125
5.1 实验五 基于比较器的数字电压表实验	128
5.1.1 实验任务	128
5.1.2 实验原理	128
5.1.3 Verilog HDL 建模描述	128
5.1.4 实验步骤	156
5.1.5 实验现象	156
第六章 姿态传感器实验	157
6.1 实验六 姿态传感器实验	161
6.1.1 实验任务	161
6.1.2 实验原理	161
5.1.3 Verilog HDL 建模描述	161
5.1.4 实验步骤	183
5.1.5 实验现象	184
版本信息	185

第一章 实验平台

1.1 概述

iCE40UP5K 由于其独特的性能得到海外创客们的热捧，尤其是其支持开源的开发工具链。不少 RISC-V 软核都是运行在 iCE40UP5K 上，基于这个大背景我们开发了一款 FPGA 核心模块，主要特点：

1、兼容树莓派基金会刚推出的 Pico 模块的管脚定义，Pico 的 GPIO 性能比较强大，因此玩家可以对比 Pico 和 FPGA IO 管脚的使用，另外也可以借助位 PICO 推出的各种外设模块（本模块没有串行 ADC 的功能，只能支持数字端口的模块）；

2、板上集成了基于 DAPLink 下载功能的 STEPLINK 功能，可以直接通过 USB 端口对 FPGA 的串行 Flash 进行配置，并可以通过同一个 USB 端口支持 UART 通信；

3、有开源的 RISC-V 可以移植使用，进而学习在 FPGA 上完成软核+FPGA 的异构设计
本设计基于 Lattice 的 ICE40UP5K FPGA，板载 LPC11U35 下载器，可以通过 USB-C 接口进行 FPGA 的配置，支持在 ICE40UP5K 上对 RISC-V 软核的移植以及开源的 FPGA 开发工具链，板上 RGB 三色 LED 灯用于简单的调试，总计 28 个 IO 用于扩展使用。

板载资源：

- 1 路 USB C 接口；
- 1 个 RGB 三色 LED；
- 1 个 12MHz 晶振时钟；
- 28 个用户可扩展 I/O；
- 集成 USB-Blaster 编程器。

1.2 产品布局与元件

1.2.1 开发板布局

本节将为您介绍 STEP-ICE-UP5K 的产品硬件信息及其特点。

图 1 介绍了硬件布局信息：

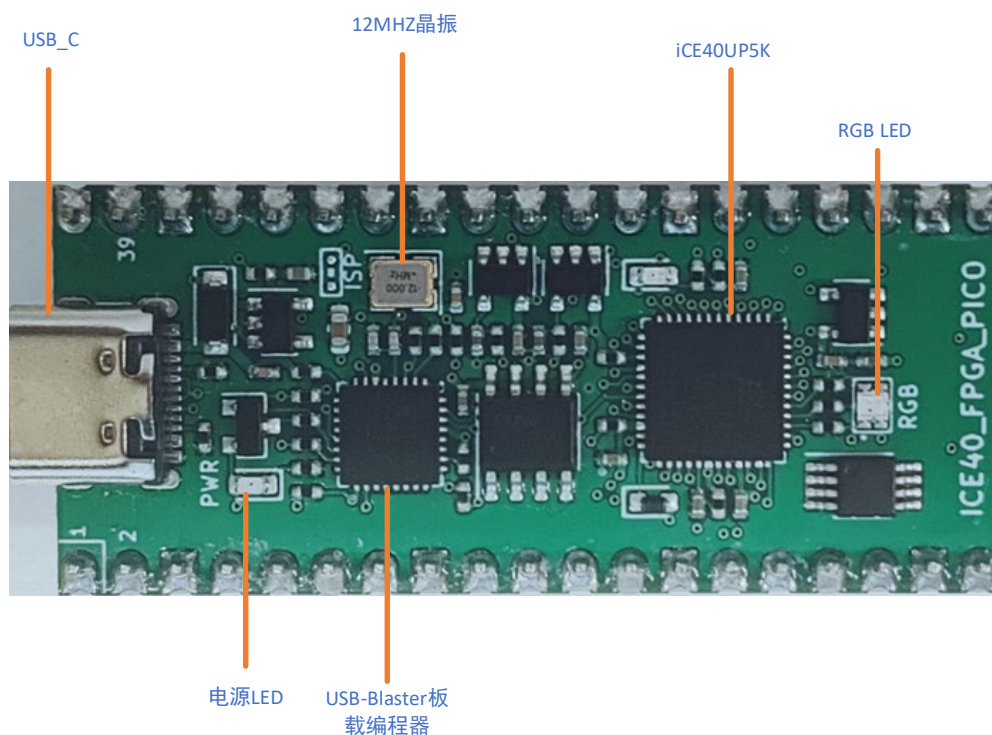


图 1 STEP-ICE-UP5K 布局图

1.2.2 FPGA 器件

目前采用的是 iCE40UP5K 芯片具有以下特性：

- 1、灵活的逻辑架构，拥有 2800 或 5280 个 4 输入 LUT、自定义 I/O、多达 80 Kb 和 1Mb 的嵌入式存储器
- 2、超低功耗的先进工艺，睡眠电流低至 75 uA，工作电流仅为 1-10mA
- 3、使用 DSP 模块实现高性能信号处理，支持乘法和累加功能
- 4、神经网络软 IP 和编译器实现灵活的机器学习/人工智能应用

1.2.3 编程配置

STEP-ICE-UP5K上采用成熟稳定的USB-Blaster编程器解决方案，完美支持Radiant系列开发设计工具。用户只需要一根USB C连接线就能够实现板卡的供电和FPGA下载编程工作。

1.2.4 引脚

28 个 GPIO 引脚。

1.2.5 显示设备

1 路的全彩色 RGB LED。

1.2.6 电源

STEP-ICE-UP5K 可以通过板上的 USB C 口 5V 供电，同时预留了 VBUS 口，可以外接 5V 电源实现供电。

1.3 FPGA 引脚分配

STEP PINs	FPGA PINs	STEP PINs	FPGA PINs	UART	FPGA PINs
GPI00	20	GPI013	42	U_TX	18
GPI01	21	GPI014	43	U_RX	19
GPI02	23	GPI015	44	比较器 1	FPGA PINs
GPI03	25	GPI016	45		
GPI04	26	GPI017	46	PWM_V1	11
GPI05	27	GPI018	47	C_OUT1	10
GPI06	28	GPI019	48	比较器 2	FPGA PINs
GPI07	31	GPI020	2		
GPI08	32	GPI021	3	PWM_V2	13
GPI09	34	GPI022	4	C_OUT2	12
GPI010	36	GPI023	6	12M	FPGA

GPI011	37	GPI024	9	时钟	PINs
GPI012	38			CLK	35
RGB LED1	R	G	B	温度传感器	FPGA PINs
FPGA PINs	39	40	41	DS18B20	35

第二章 开发工具的安装与注册

2.1. 概述

Lattice Radiant 设计软件适用于网络边缘应用的全功能工具套件，支持 iCE40 UltraPlus 产品系列，提供业界领先的工具和功能，可帮助用户高效地开发网络边缘应用。

- 业界标准 IEEE 1735 IP 加密和 Synopsys 设计约束（SDC），最大化互操作性
- 集成的工具环境，实现易于使用的设计导航和调试功能
- 完整的物理到逻辑设计流程，实现交叉探索

在 Lattice 官网可以免费下载最新的 Radiant 工具，支持 Windows 系统和 Linux 系统。

2.2. 软件安装

2.2.1 Radiant 软件下载

1、打开 Lattice 官网，找到 Radiant 系列软件安装包下载页面，下载软件安装包。

- Lattice 官网：
- <https://www.latticesemi.com/>
- 软件下载：
- <https://www.latticesemi.com/Products/DesignSoftwareAndIP/FPGAandLDS/Radiant>
- 选择软件版本和操作系统。

2、根据安装指南进行安装。

2.2.2 Radiant 安装步骤

1、将 Radiant 压缩文件解压缩，然后双击 Radiant.exe 文件

文件夹	diamond	2023/2/15 10:56	文件夹
文件夹	R2019a(64bit)	2023/2/28 14:38	文件夹
文件夹	达芬奇PPT	2023/2/17 15:40	文件夹
应用程序	2022.1.0.52.3_Radiant.exe	2022/12/2 10:26	应用程序
WinRAR ZIP 压缩文件	2022.1.0.52.3_Radiant.zip	2023/3/3 10:25	WinRAR ZIP 压缩文件

2、Radiant 启动界面之后出现安装提示窗口，点击下一步。



3、安装路径，默认是安装在 C 盘，可以根据实际情况修改，点击下一步。

Radiant Software 2022.1 设置

✕

选择安装文件夹

LATTICE
SEMICONDUCTOR

Specify the folder where Lattice Radiant Software 2022.1 is to be installed.

F:\software\Radiant\

浏览(R)...

< 上一步(B)

下一步(N) >

取消

4、选择安装组件，如下图，勾选对应的组件，点击下一步。

Radiant Software 2022.1 设置

✕

Select Component(s)

Select the component(s) you want to install.

LATTICE
SEMICONDUCTOR

Base

▼ ☒ FPGA Devices☒ Lattice MachXO5-NX☒ Lattice Avant☒ Lattice CertusPro-NX☒ Lattice Certus-NX☐ Lattice CrossLink-NX☒ Lattice iCE40 UltraPlus☒ Programmer 2022.1☒ Mentor ModelSim Lattice Edit...

This option installs Mentor ModelSim Simulation tool.

This component will occupy approximately 6.58 GB of 14.45 GB on your hard disk drive.

默认(A)

全选(S)

取消全选(D)

< 上一步(B)

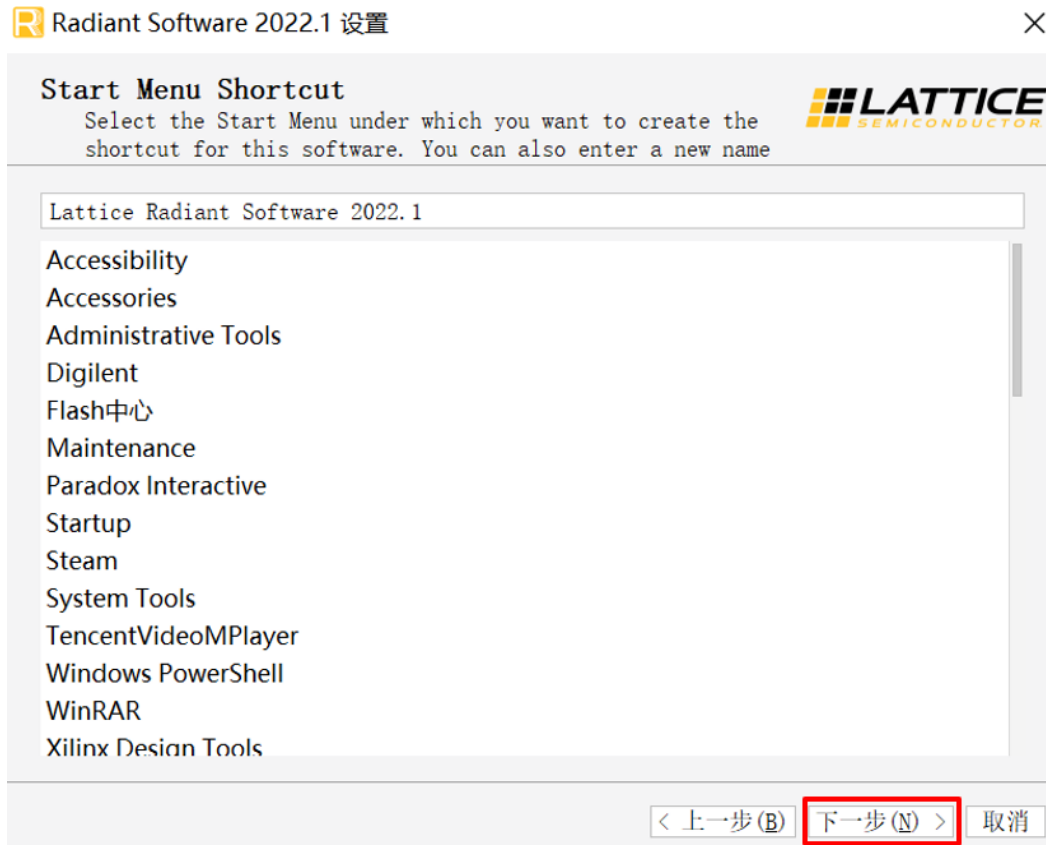
下一步(N) >

取消

5、选择我接受此许可，点击下一步。



6、点击下一步。



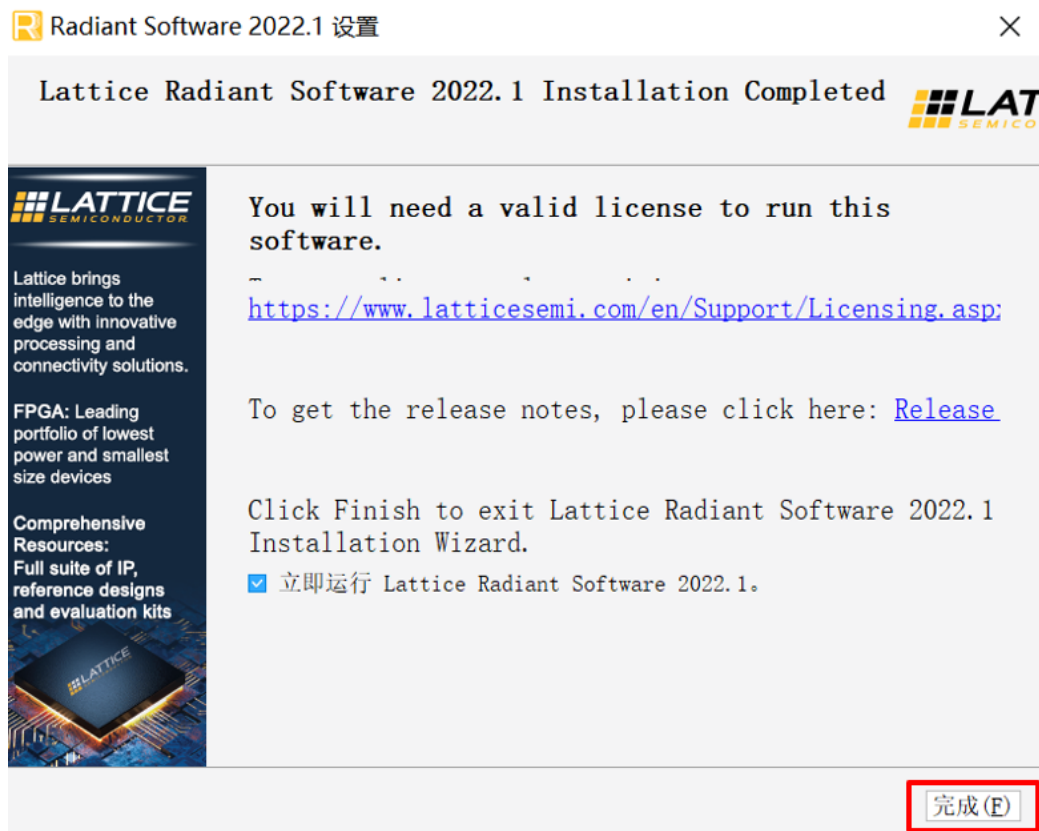
7、点击安装，进入安装过程



8、大约需要半个小时的等待，软件会将 Radiant 软件和 Modelsim 软件等全部安装。



9、随后进入安装完成界面，点击完成，完成软件安装



2.2.3 Radiant 软件注册

1、Radiant 软件不仅提供了简单易用的操作环境，而且提供免费许，免费许可证获取十分简单，首先在 Lattice 官网注册一个账号（需要邮箱）。

2、访问官网页面 <https://www.latticesemi.com/Support/Licensing>，点击 Request a Free License。

Additional Software & IP Features

This section is for customers who wish to upgrade their existing Diamond license files f

Lattice Propel License

The Lattice Propel design environment for Lattice FPGA-based processor systems includes an IP catalog linked to Lattice’s on-demand IP server, IP system integration tools, and a software development kit (SDK). The Propel License enables users to access this easy to use system integration environment.

To request a license you will need the following:

- Physical MAC address (12-digit hexadecimal value)

[Request a Free License](#)

3、填写机器物理地址，可以选择所需要的 IP 核，然后点击 yes，如下图所示。最后点击 Generate License，生成一个 license.dat 文件，将会发送到注册所用的邮箱。

Host NIC (physical address) *

Please enter your 12-digit NIC without any separator or spaces.

☒ I verify that I am not an employee of Cadence Design Systems, Mentor Graphics Corporation, or Synopsys, Inc.

Radiant Free IP (optional). Please select all that apply.

CertusPro-NX		CrossLink-NX/Certus-NX	
☒	Select All	☐	Select All
☒	I2C Master	☐	I2C Master
☒	I2C Slave	☐	I2C Slave
☒	Multi-Port Memory Controller	☐	Multi-Port Memory Controller
☒	SPI Flash Controller	☐	SPI Flash Controller
☒	SPI Master	☐	SPI Master
☒	SPI Slave	☐	SPI Slave
☒	UART 16550	☐	UART 16550
☒	Watchdog Timer	☐	Watchdog Timer
☒	Timer-Counter	☐	Timer-Counter

IP Core License Agreement

IP Core License Agreement

Lattice SG Pte. Ltd.
IP Core License Agreement

THIS IS A LEGAL AGREEMENT BETWEEN YOU, THE END USER, AND LATTICE SG PTE. LTD. ("LATTICE"). BY PROCEEDING WITH THE USE OF THE IP CORE, YOU: (1) ACKNOWLEDGE YOU HAVE READ THIS AGREEMENT, YOU UNDERSTAND IT, AND YOU AGREE TO BE BOUND BY THE TERMS AND CONDITIONS OF THIS AGREEMENT; AND (2) IF YOU ARE ENTERING INTO THIS AGREEMENT ON BEHALF OF YOUR EMPLOYER, YOU REPRESENT THAT YOU HAVE THE AUTHORITY TO ENTER INTO THIS AGREEMENT ON BEHALF OF YOUR EMPLOYER. IF YOU DO NOT AGREE TO THE TERMS AND CONDITIONS OF THIS AGREEMENT, DO NOT USE THE IP CORE, AND IF YOU HAVE ALREADY OBTAINED THE IP CORE FROM AN AUTHORIZED SOURCE, PROMPTLY RETURN IT FOR A REFUND.

1. Definitions.

1.1 "Configured Object Code" means the form of the IP Core in bitstream, encrypted net list, or other non-human readable device programming file format.

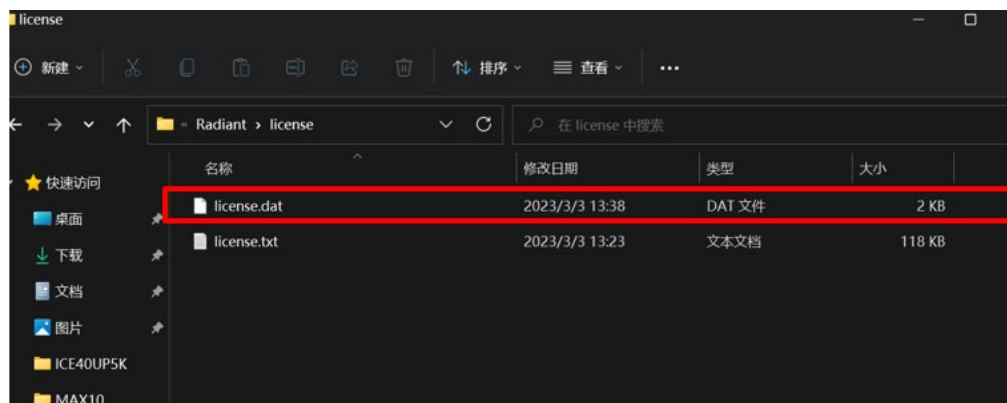
I have read and agree to the above License Agreement. *

☒ Yes

☐ No

Generate License

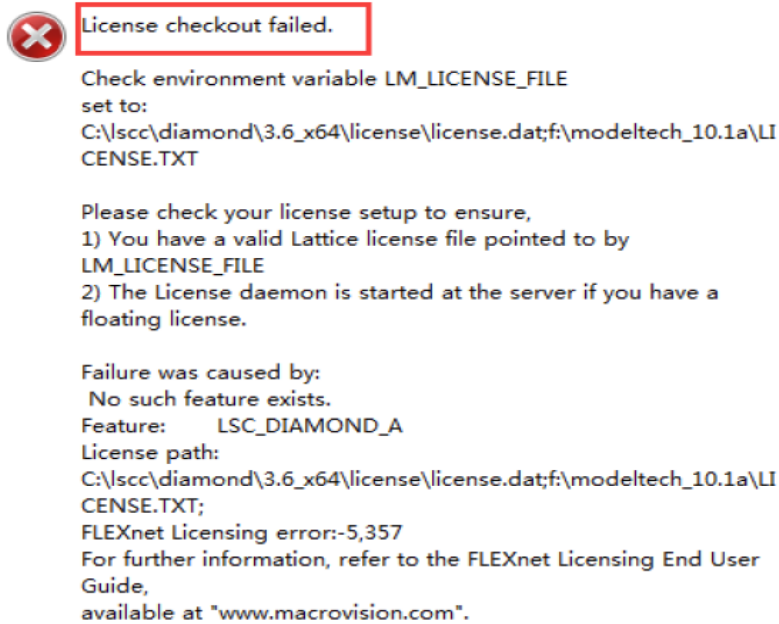
4、在软件安装过程中，已自动添加系统环境变量。只需将申请的 license.dat 文件拷贝到安装目录下的 license 文件夹中，本文例为：F:\software\Radiant\license



至此完成 License 文件设置，可以开始使用 Radiant 进行设计了。

注意！！

如双击打开 Diamond 软件是出现如下错误，则表示环境变量设置有问题。



解决方法是：右击“计算机”选择属性，进入控制面板中的系统属性界面。



点击高级系统设置。选择高级，点击环境变量！



进入环境变量设置界面，查看用户变量和系统变量 `LM_LICENSE_FILE`，将 License 位置（本文案例为：`F:\software\Radiant\license`）添加进用户变量和系统变量的 `LM_LICENSE_FILE` 中。

环境变量



XX 的用户变量(U)

变量	值
LM_LICENSE_FILE	F:\software\Radiant\license\license.dat
OneDrive	C:\Users\XX\OneDrive
Path	C:\Users\XX\AppData\Local\Microsoft\WindowsApps;D:\ruanjian\...
PyCharm Community Edition	D:\ruanjian\PyCharm Community Edition 2021.3\bin;
QSYS_ROOTDIR	F:\software\intelFPGA_lite\18.1\quartus\sopc_builder\bin
TEMP	C:\Users\XX\AppData\Local\Temp
TMP	C:\Users\XX\AppData\Local\Temp

新建(N)...

编辑(E)...

删除(D)

系统变量(S)

变量	值
ComSpec	C:\WINDOWS\system32\cmd.exe
DriverData	C:\Windows\System32\Drivers\DriverData
LATTICE_INI_PATH	C:\lsc_env
LM_LICENSE_FILE	F:\software\diamond\diamond\3.12\license\license.dat;F:\software...
MGLS_LICENSE_FILE	D:\ruanjian\modelsim\win64\LICENSE.TXT
NUMBER_OF_PROCESSORS	16
OS	Windows_NT
Path	C:\Program Files\Microsoft\jdk-11.0.12.7-hotspot\bin;D:\ruanjian\...

新建(W)...

编辑(I)...

删除(L)

确定

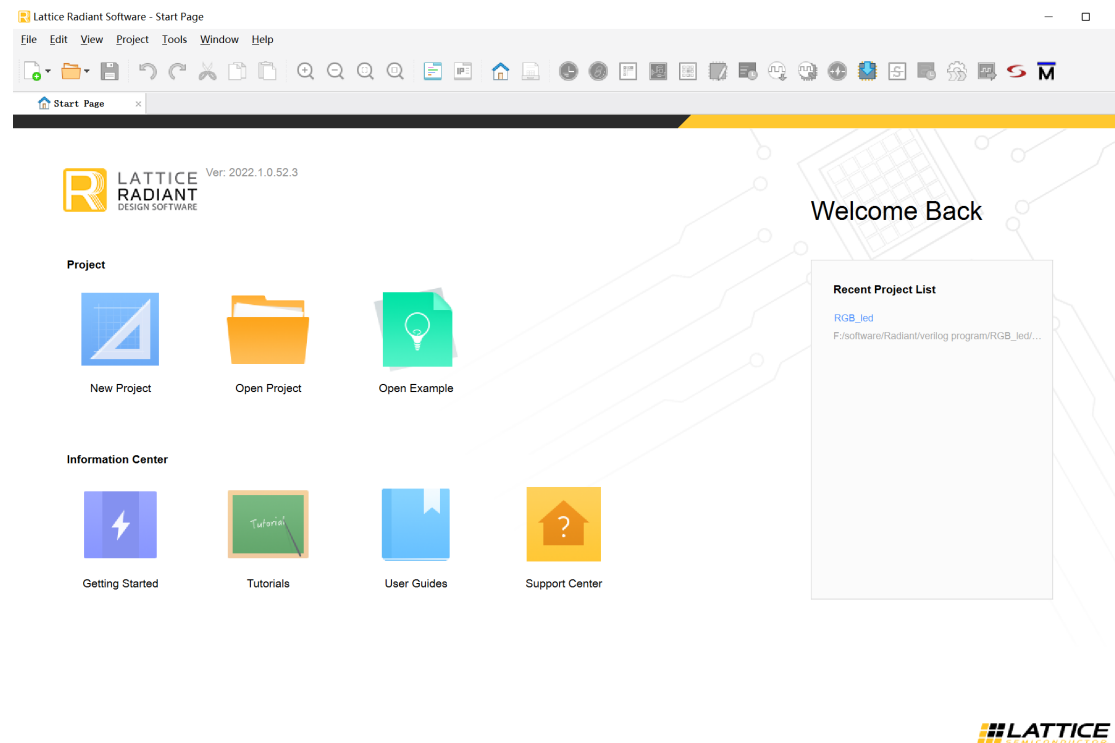
取消

2.3. 创建第一个工程

2.3.1 新建工程

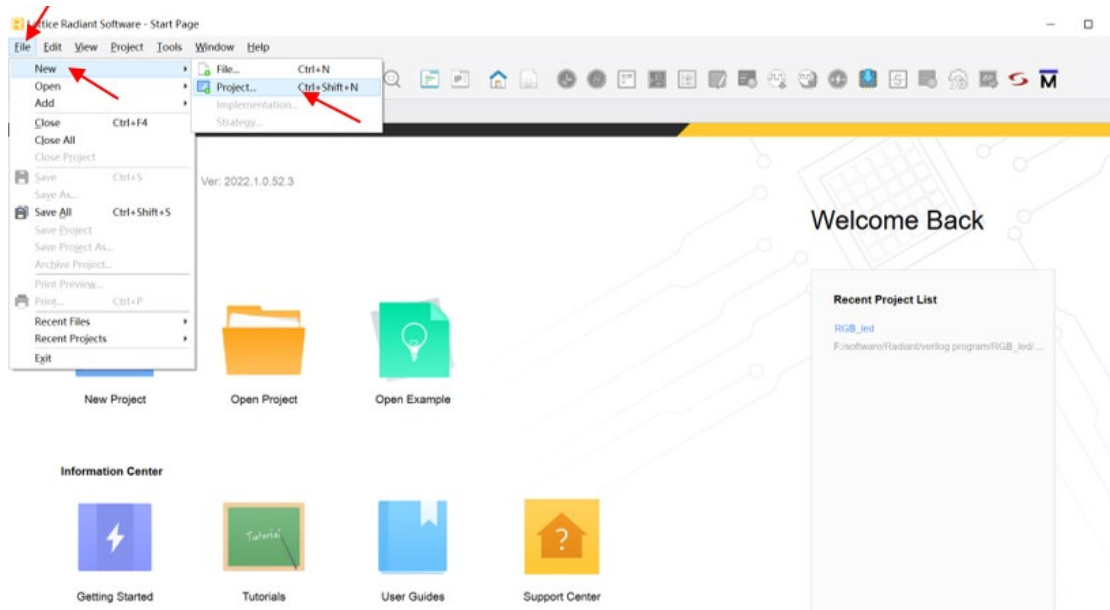
1、启动 Radiant 软件：

双击 Radiant 系列软件图标，界面如下。

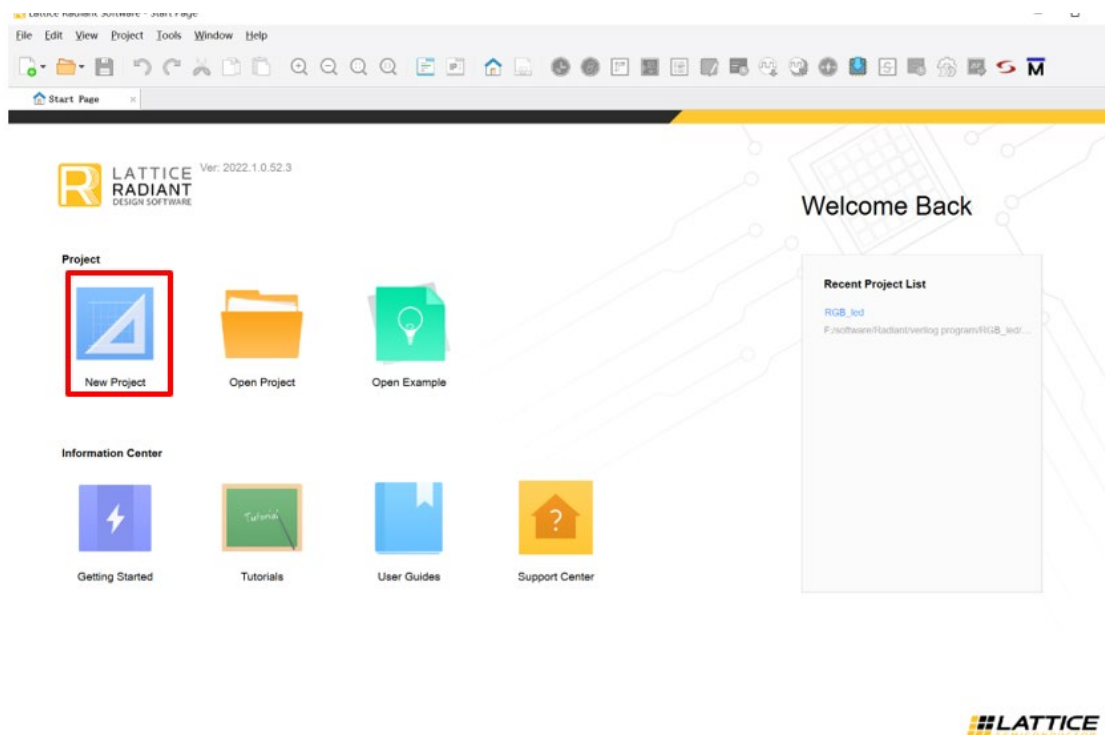


2、创建工程：

点击 File → New → Project 选项 或 点击 Home 页面中的 New Project 图标。



LATTICE



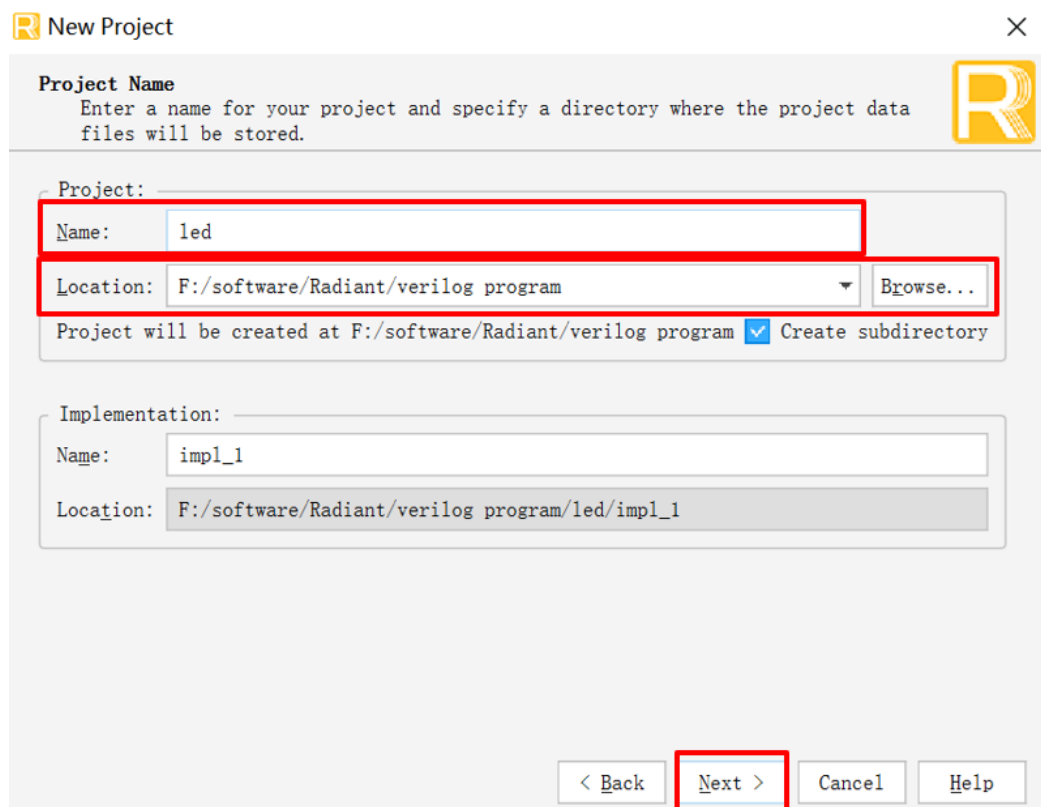
LATTICE

3、点击 Next 进入新建工程目录。



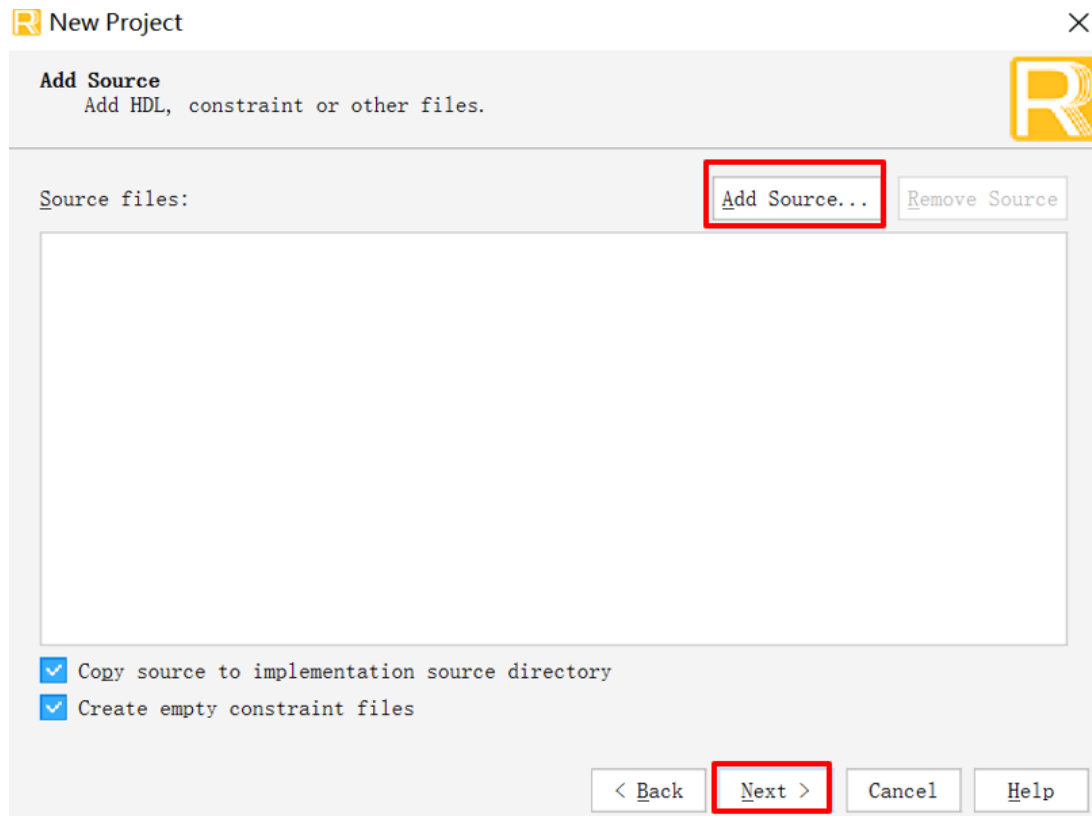
4、工程目录、名称填写：（注意：工程目录中不能有汉语、空格等字符）

- 工程目录：选择新建工程的目录
- 工程名称：填写工程名称



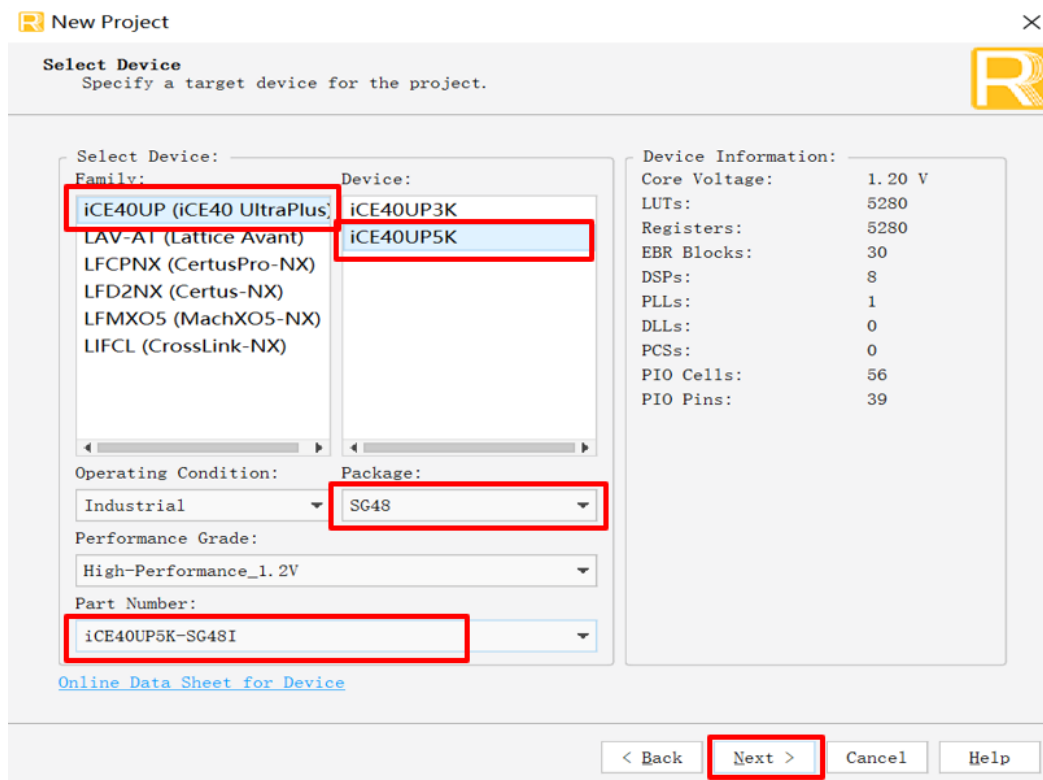
5、添加源文件、约束文件。

如已有源文件，点击 Add Source 添加文件；如没有，直接点击 Next。



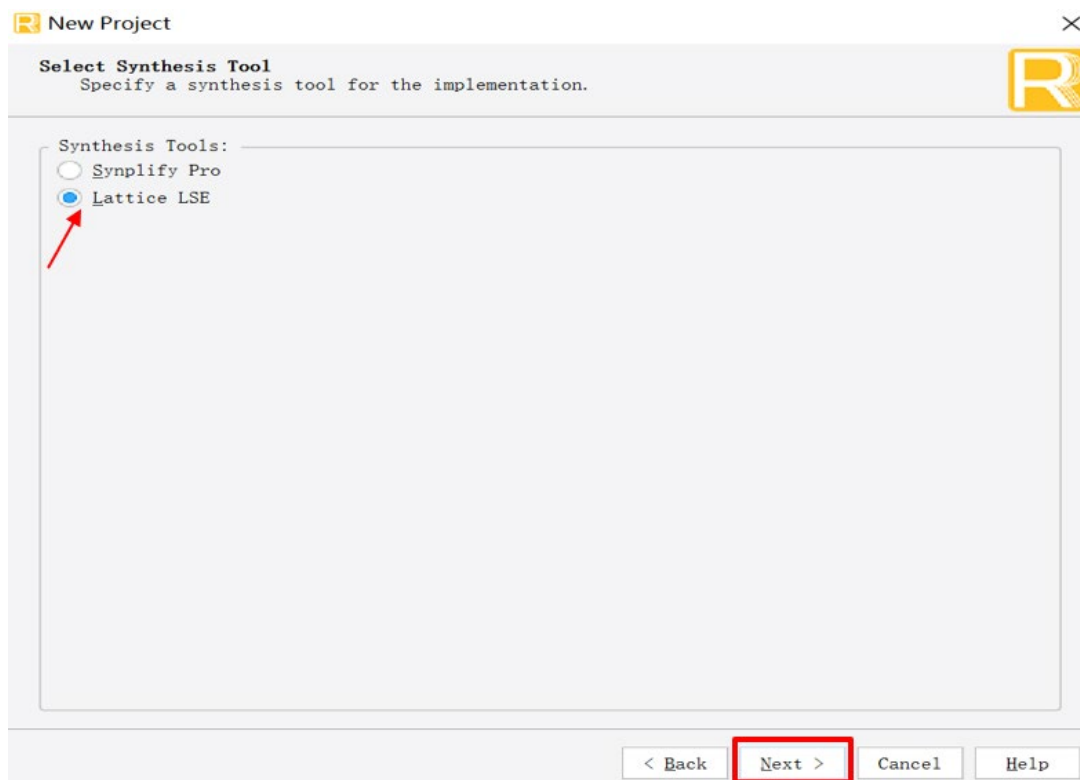
6、器件选择：

根据开发平台使用的 FPGA 选择对应器件例如 iCE40UP5KSG48I（注意：这里需要根据实物板卡上面的芯片标识选择对应的芯片型号），点击 Next。



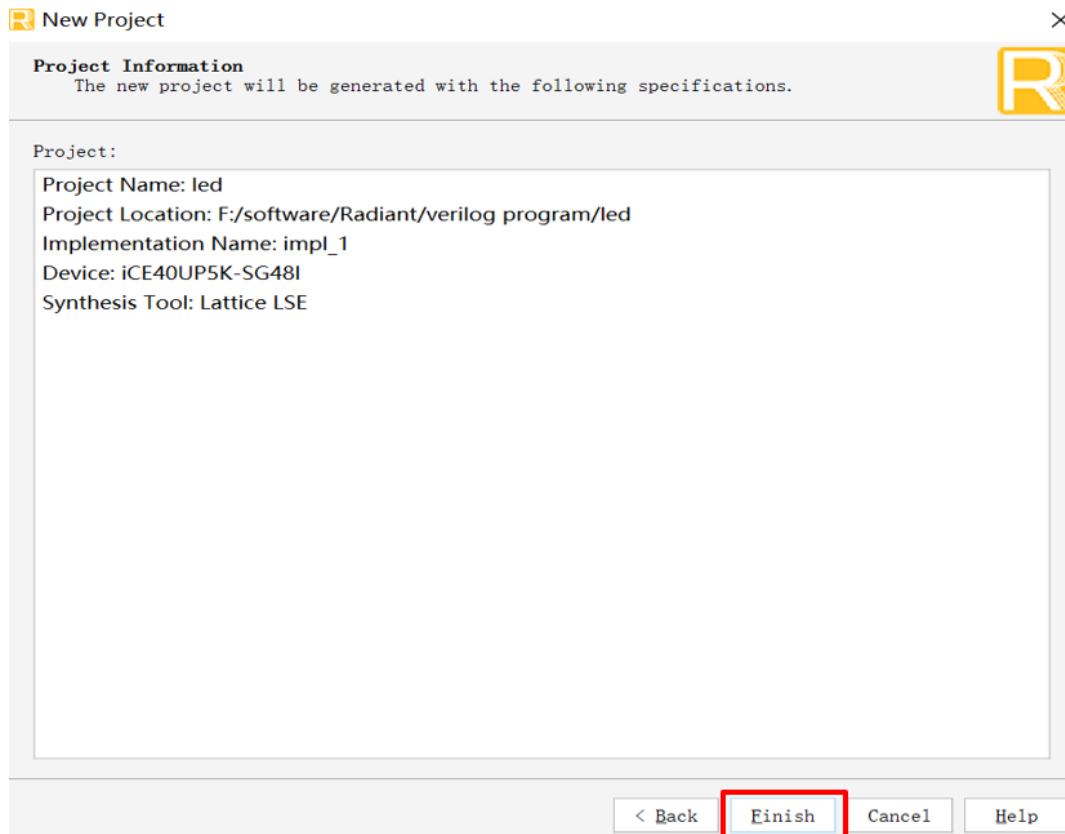
7、综合分析工具选择:

选择 Lattice LSE，如电脑上装有 Synplify Pro，也可以用它综合分析，随后点击 Next。

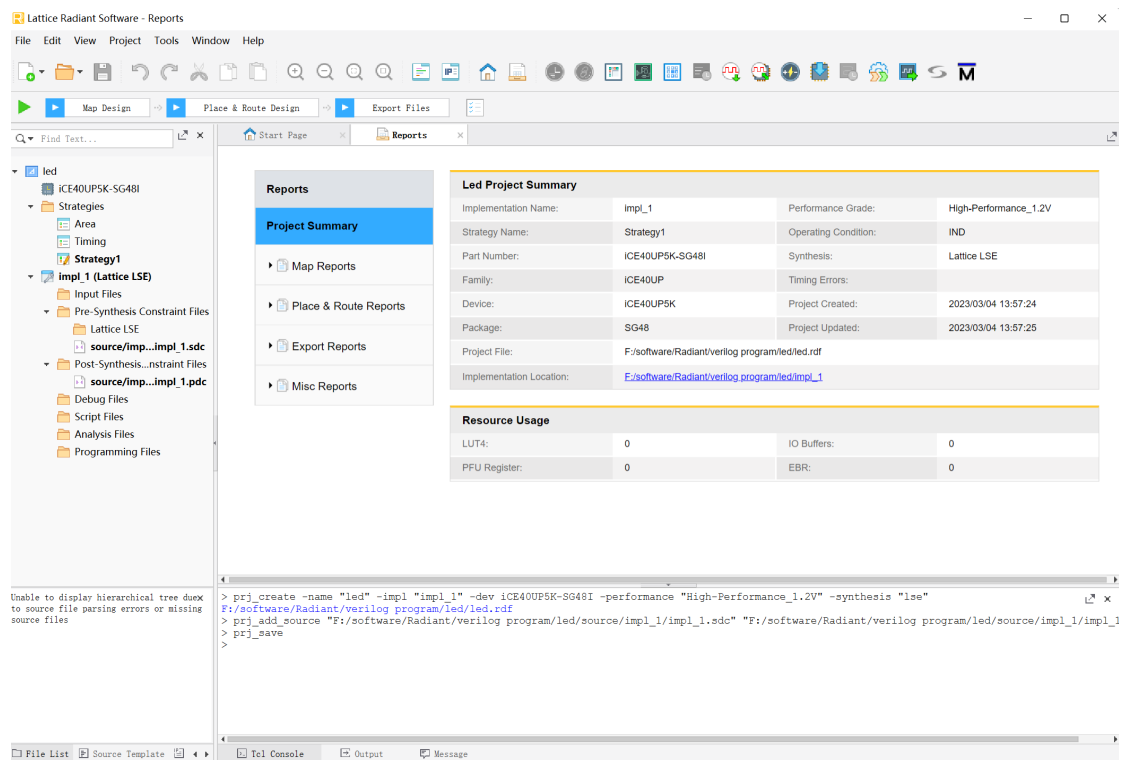


8、工程设置确认:

确认工程相应的设置，如需调整点击 Back 返回修改，若确认设置，点击 Finish。

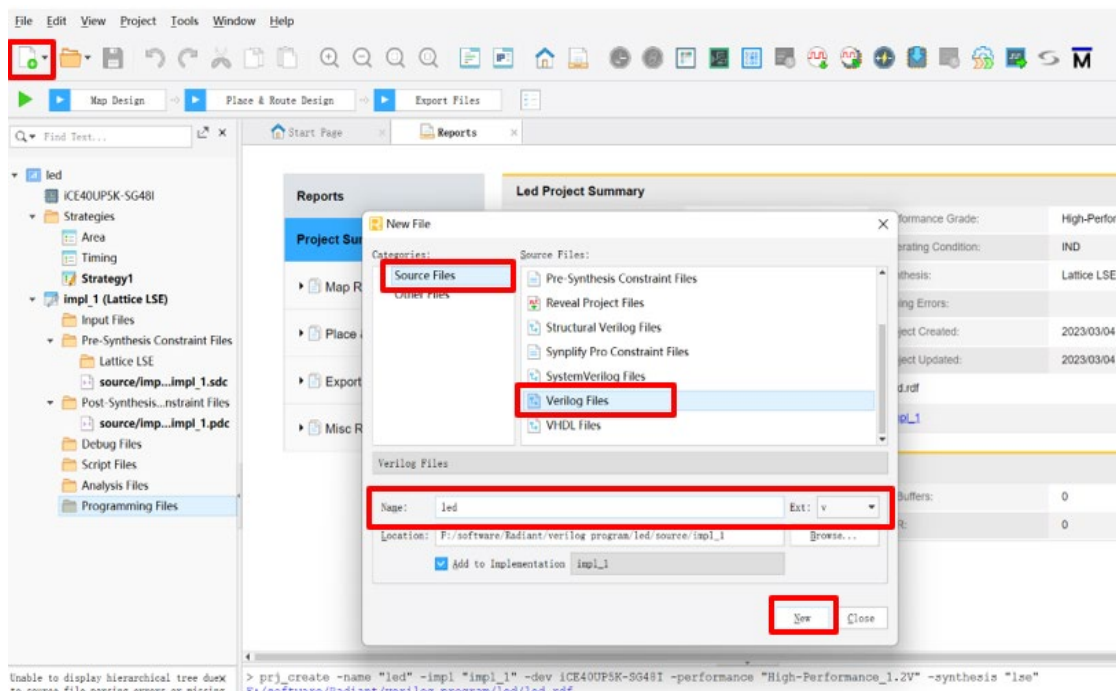


9、工程创建完毕，Radiant 软件自动进入开发界面，如下：

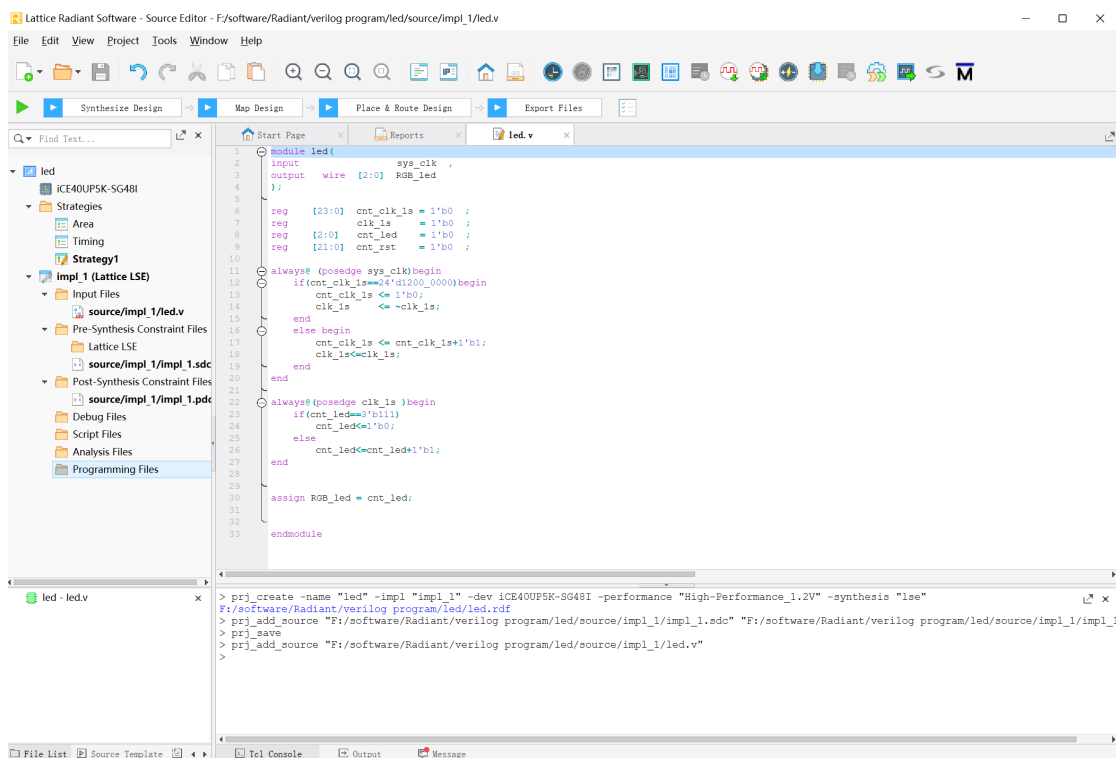


2.3.2 添加设计文件

1、选择 File→New→File 或 点击工具栏中的 New 按钮，选择 Verilog File 文件类型，将文件命名为 led，点击 New，Radiant 软件会新建并打开 Verilog 文件。



2、在新建的 Verilog 文件中进行 Verilog 代码编写、保存，如下图所示，程序源码如下。



程序源码：

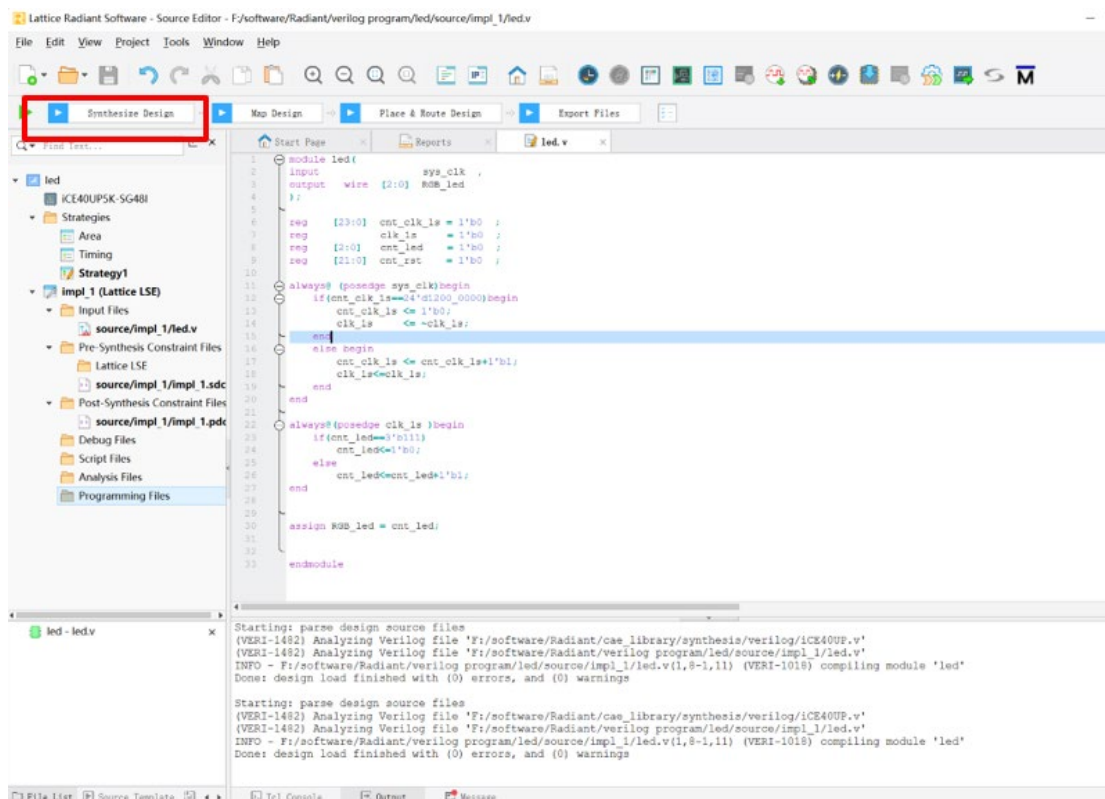
```
reg    [23:0] cnt_clk_1s = 1'b0 ;
reg          clk_1s      = 1'b0  ;
reg    [2:0]  cnt_led    = 1'b0  ;
reg    [21:0] cnt_rst     = 1'b0  ;

always@ (posedge sys_clk)begin
    if(cnt_clk_1s==24'd1200_0000)begin
        cnt_clk_1s <= 1'b0;
        clk_1s     <= ~clk_1s;
    end
    else begin
        cnt_clk_1s <= cnt_clk_1s+1'b1;
        clk_1s<=clk_1s;
    end
end

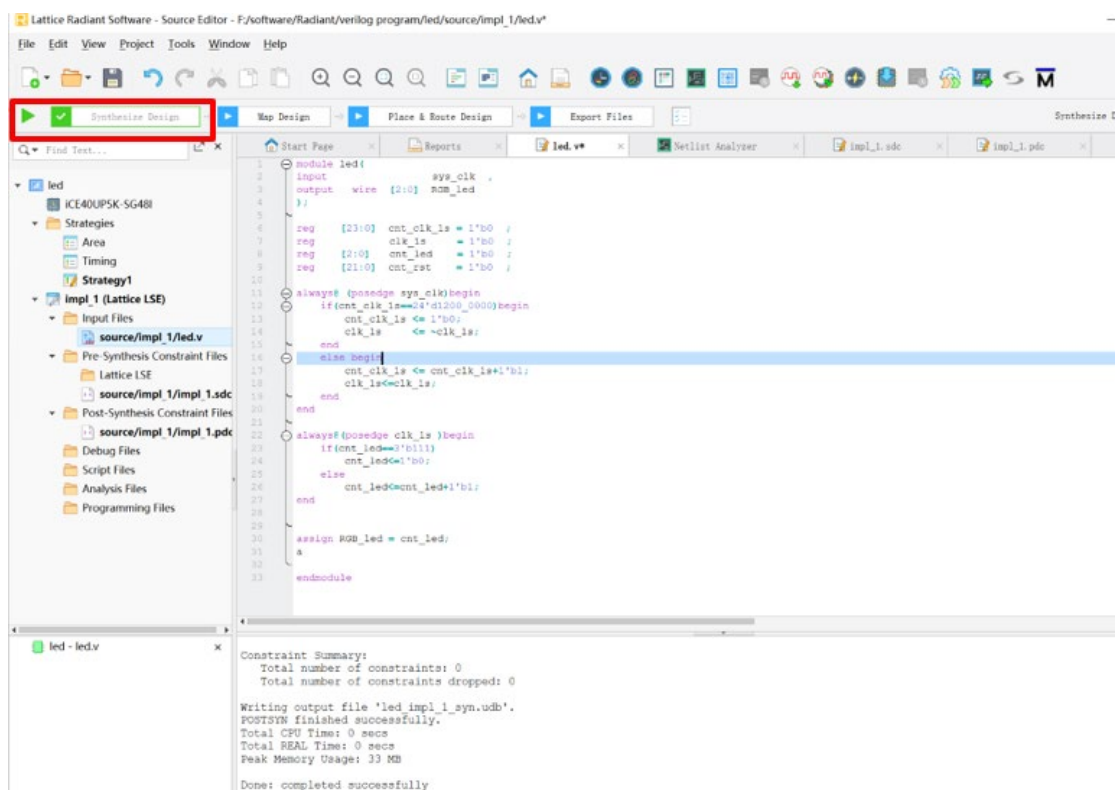
always@(posedge clk_1s )begin
    if(cnt_led==3'b111)
        cnt_led<=1'b0;
    else
        cnt_led<=cnt_led+1'b1;
end

assign RGB_led = cnt_led;
endmodule
```

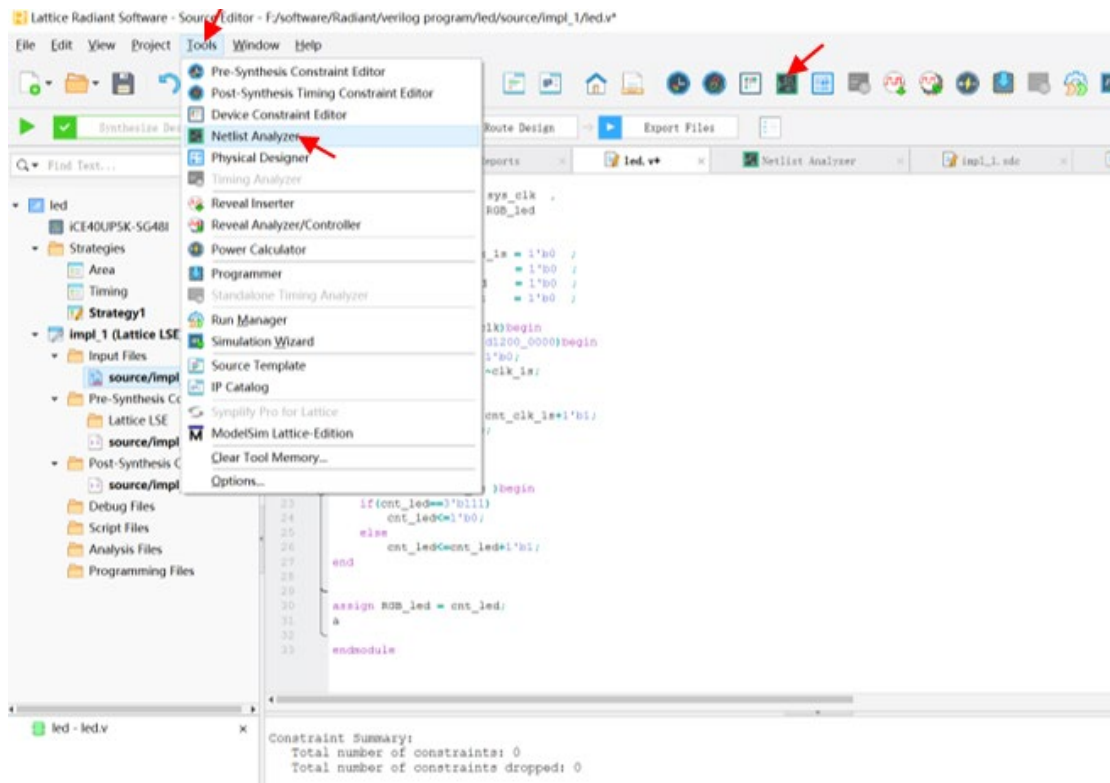
3、选择工具栏中 Synthesis Design 按钮，如下图所示。



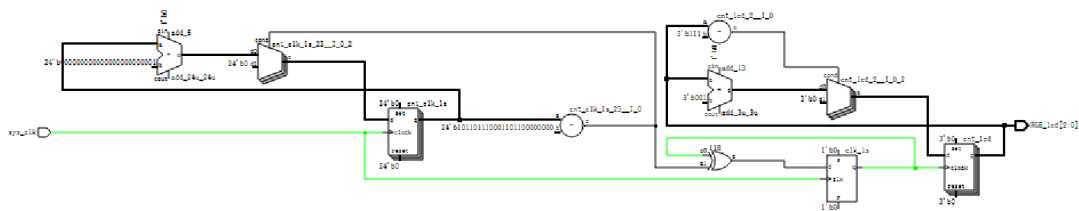
4、Radiant 软件会完成分析综合，若设计没有问题，综合 Synthesis Design 会变成绿色，同时左侧出现绿色对勾。



5、分析完成后可以选择工具栏中 Netlist Analyzer 或者 Tools→Netlist Analyzer 查看电路。

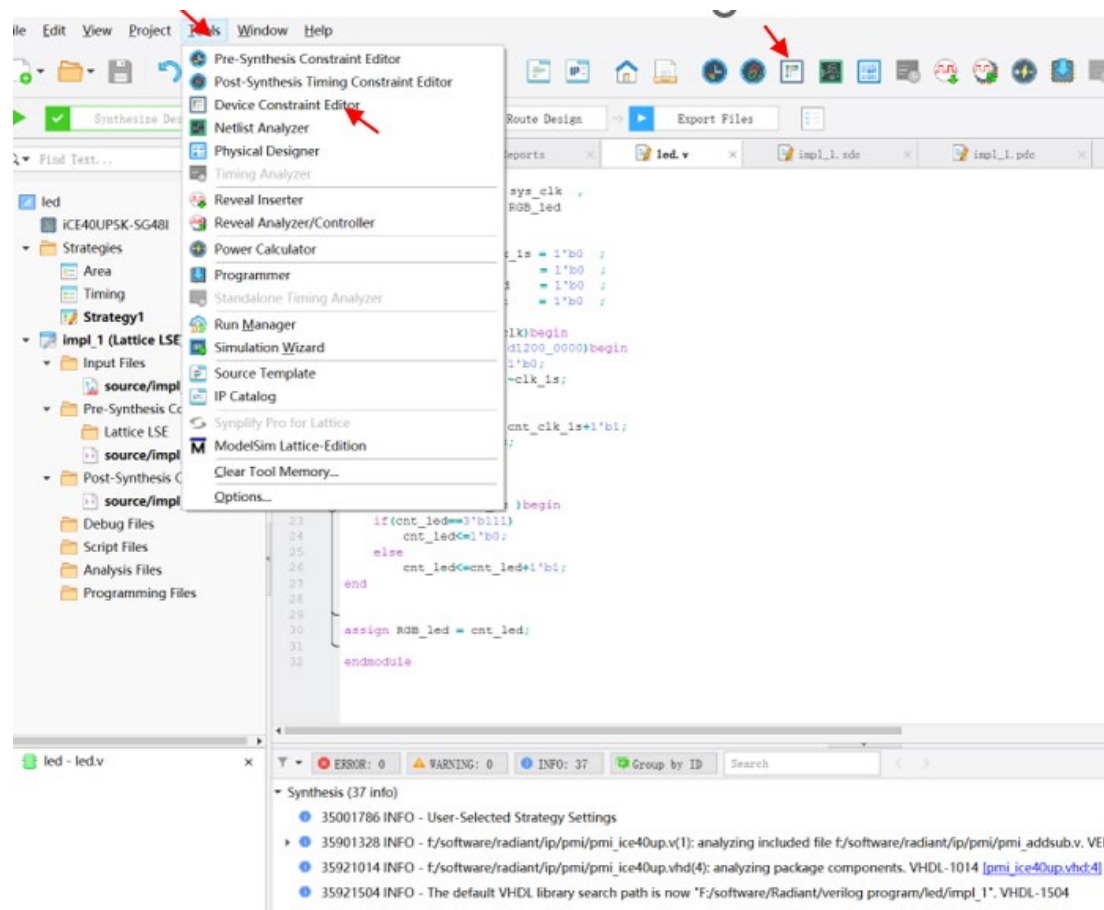


RTL 电路图如下



2.3.3 管脚约束

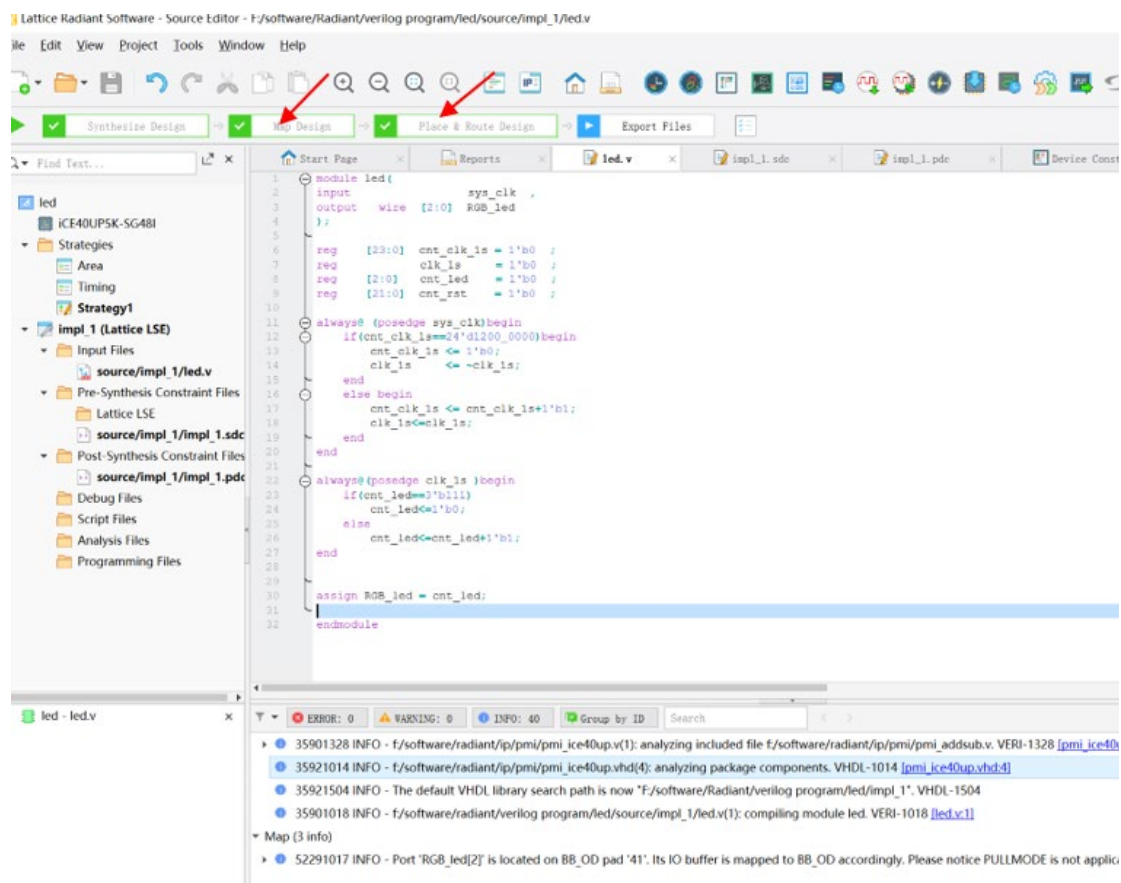
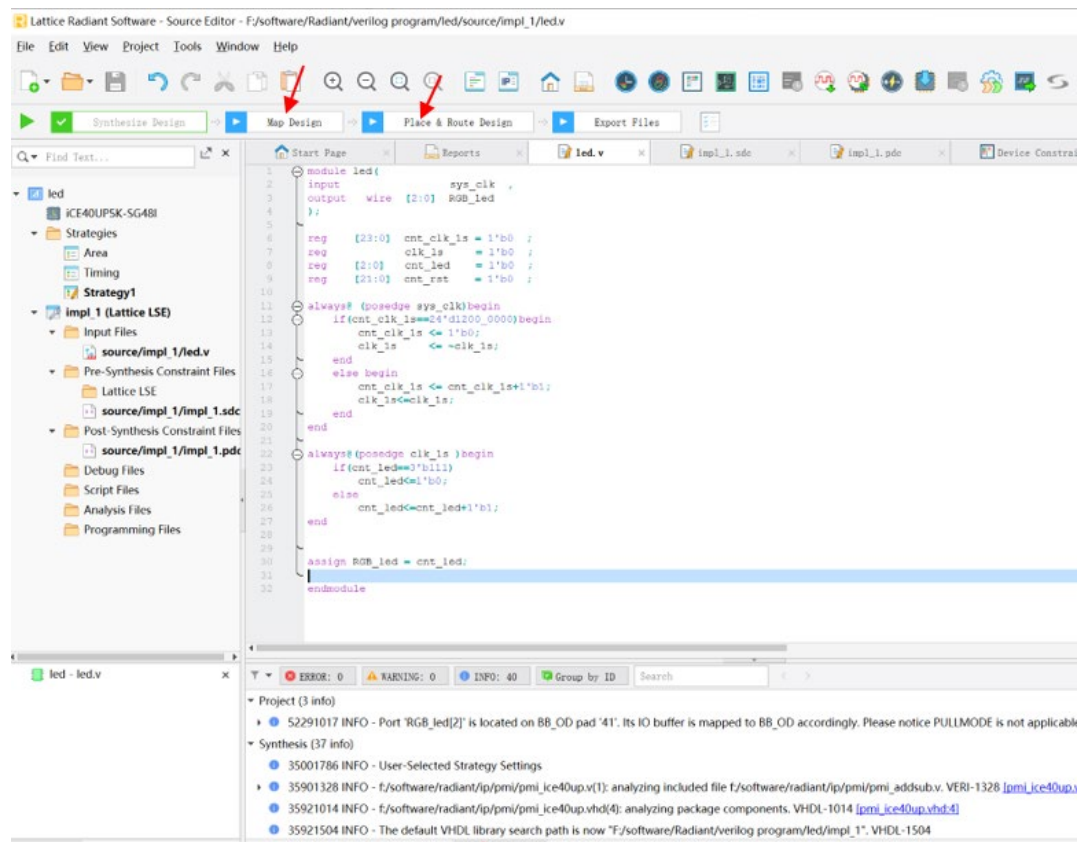
1、选择工具栏中的 Device Constraint Editor 或者 Tools→Device Constraint Editor 进入管脚分配页面。



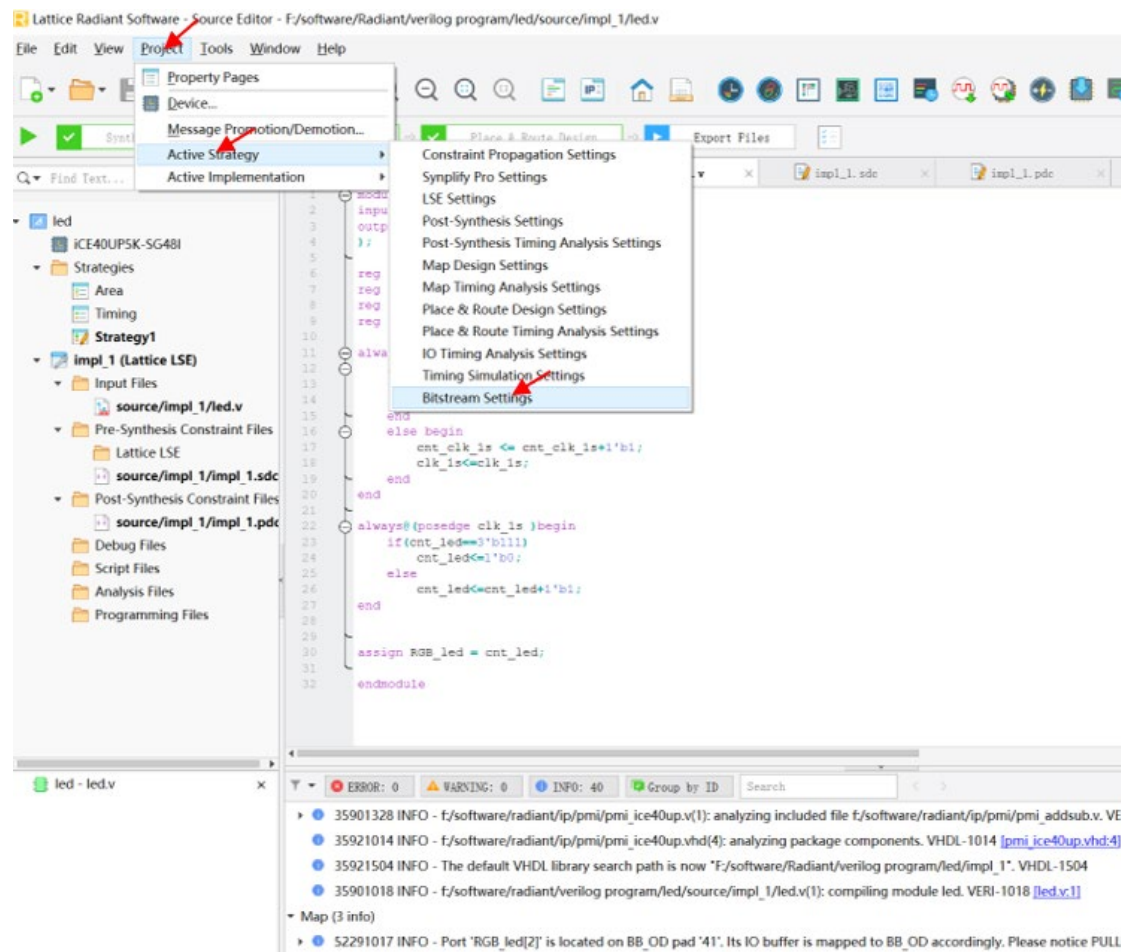
2、在管脚分配页面中将所有端口分配对应的 FPGA 管脚，如下图，然后保存关闭。

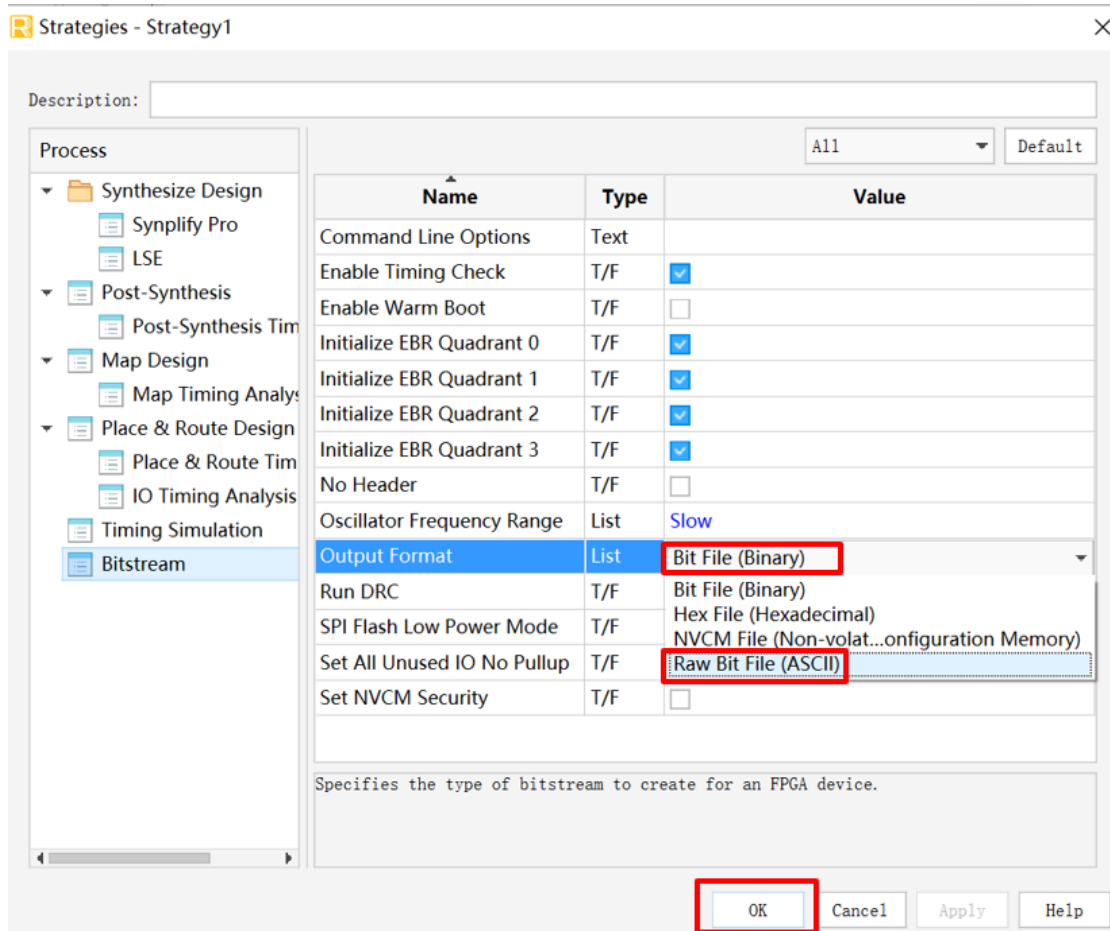
Name	Group By	Pin	BANK	IO_TYPE	DRIVE	PULLMOD
sys_clk	N/A	35	0	LVC MOS33	NA	100K
Output	N/A	N/A	N/A	N/A	N/A	N/A
RGB_led[0]	N/A	39	0	LVC MOS33	8	NA
RGB_led[1]	N/A	40	0	LVC MOS33	8	NA
RGB_led[2]	N/A	41	0	LVC MOS33	8	NA

3、点击 Map Design 和 Place & Route Design，线路分析完后，Map Design 和 Place & Route Design 会变成绿色，同时左侧出现绿色对勾。

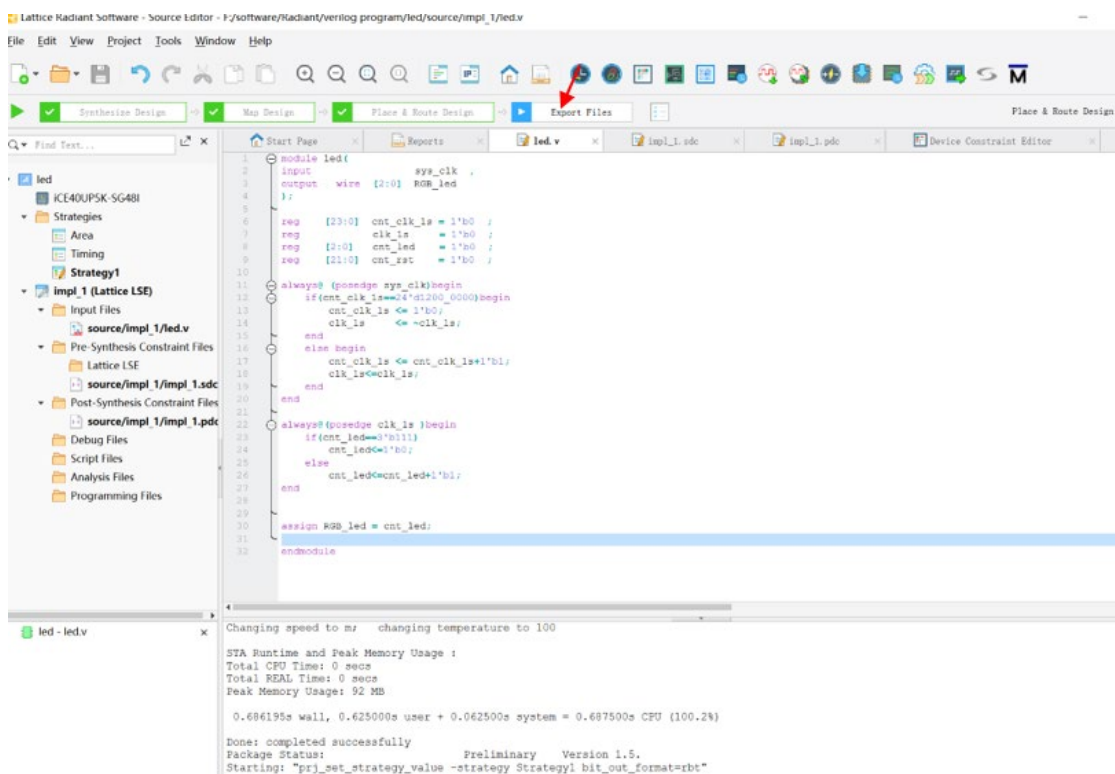


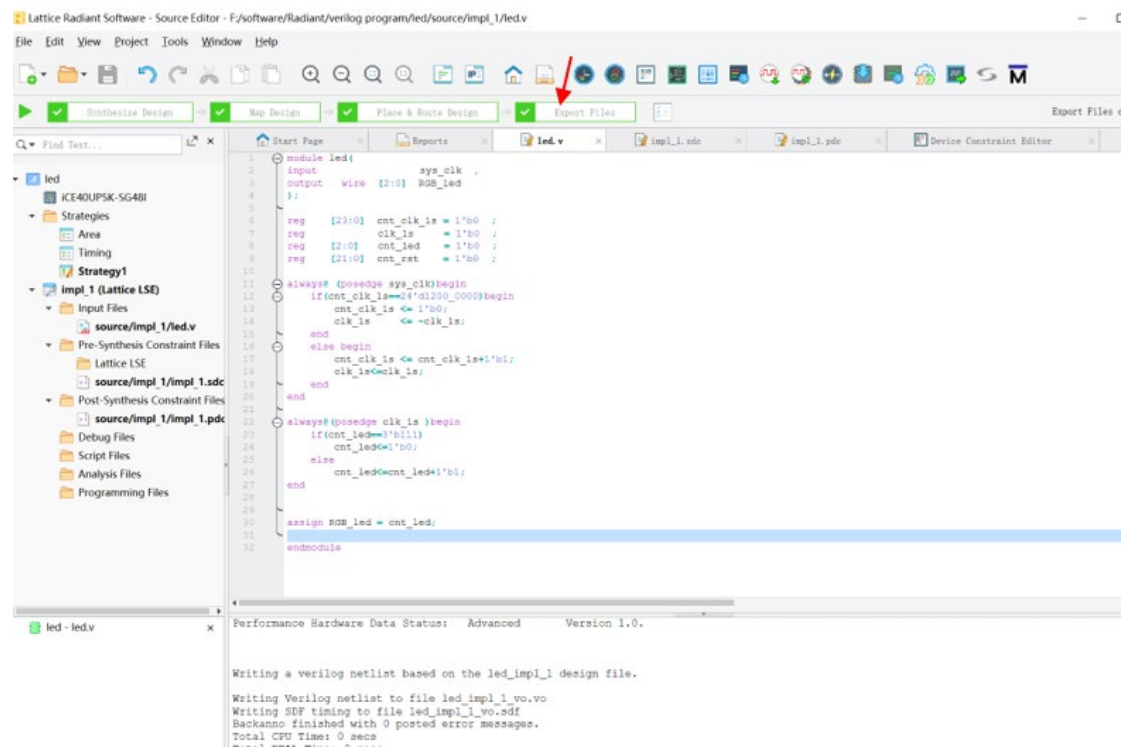
4、点击 Project → Active Strategy → Bitstream Settings。双击 Bit File，选择 Raw Bit File (ASCII)，随后点击 OK。





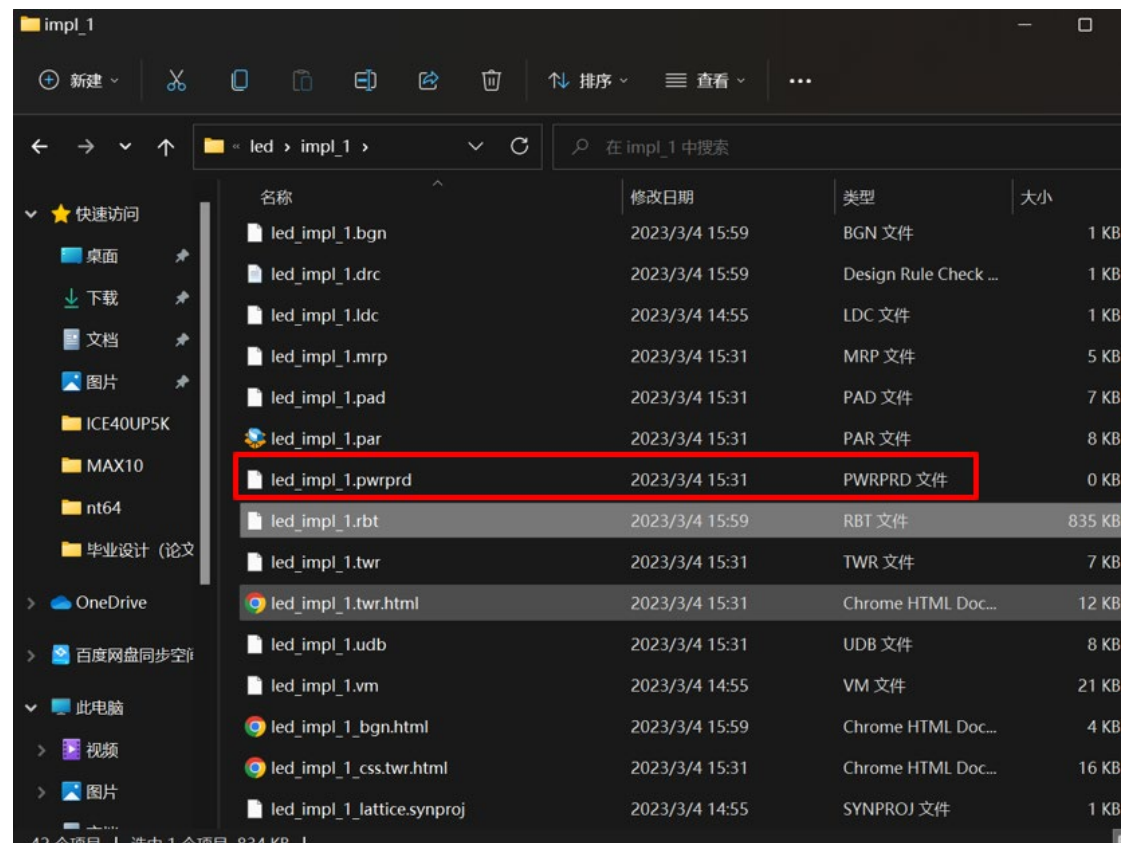
5、点击 Export Files，完成后 Export Files 会变成绿色，同时左侧出现绿色对勾。

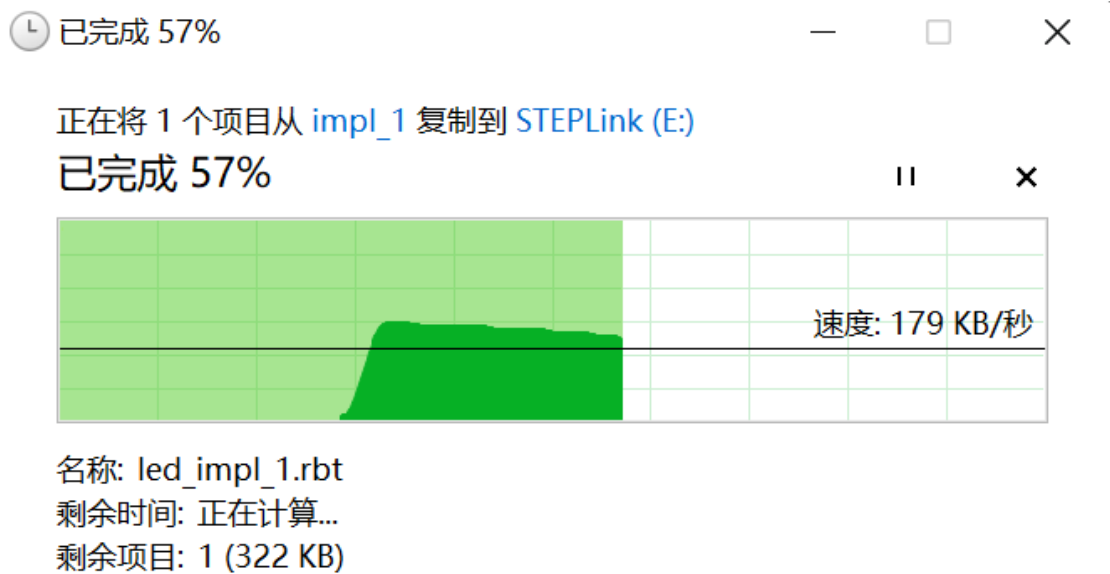
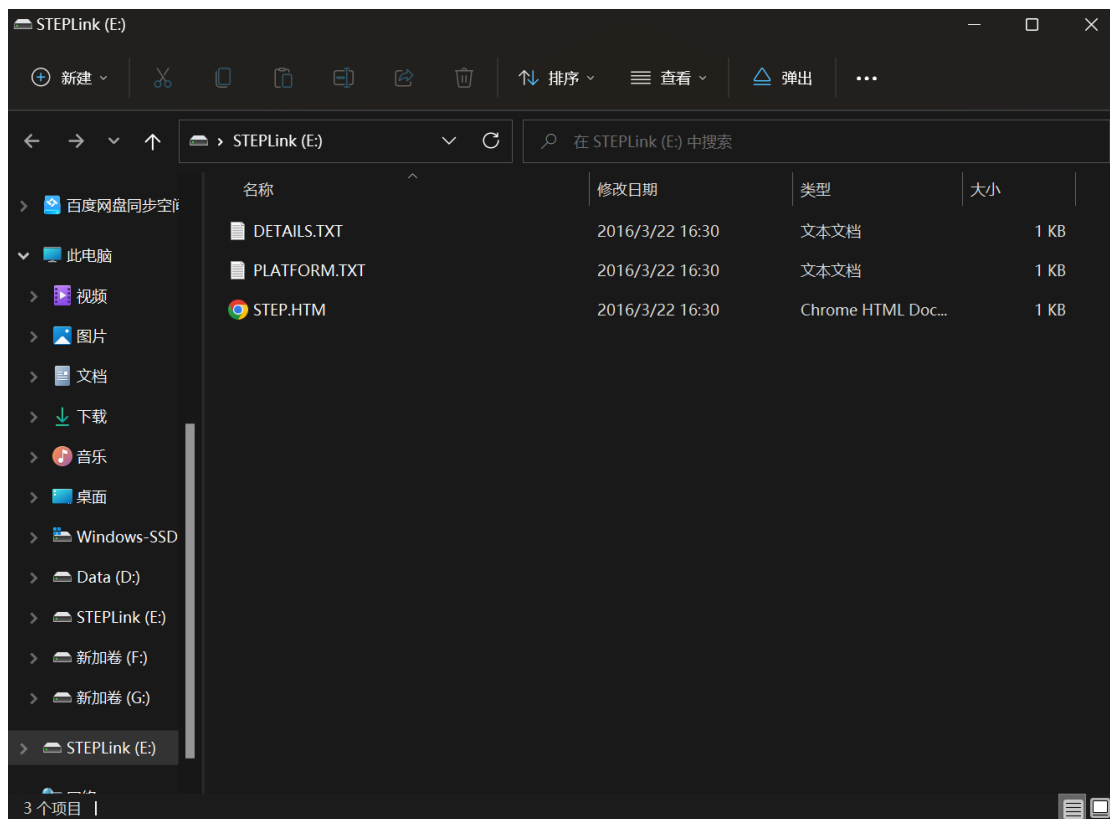




2.3.4 FPGA 加载

在工程的文件夹中找到.rbt 文件，将此文件复制到 FPGA 的文件夹中完成下载。



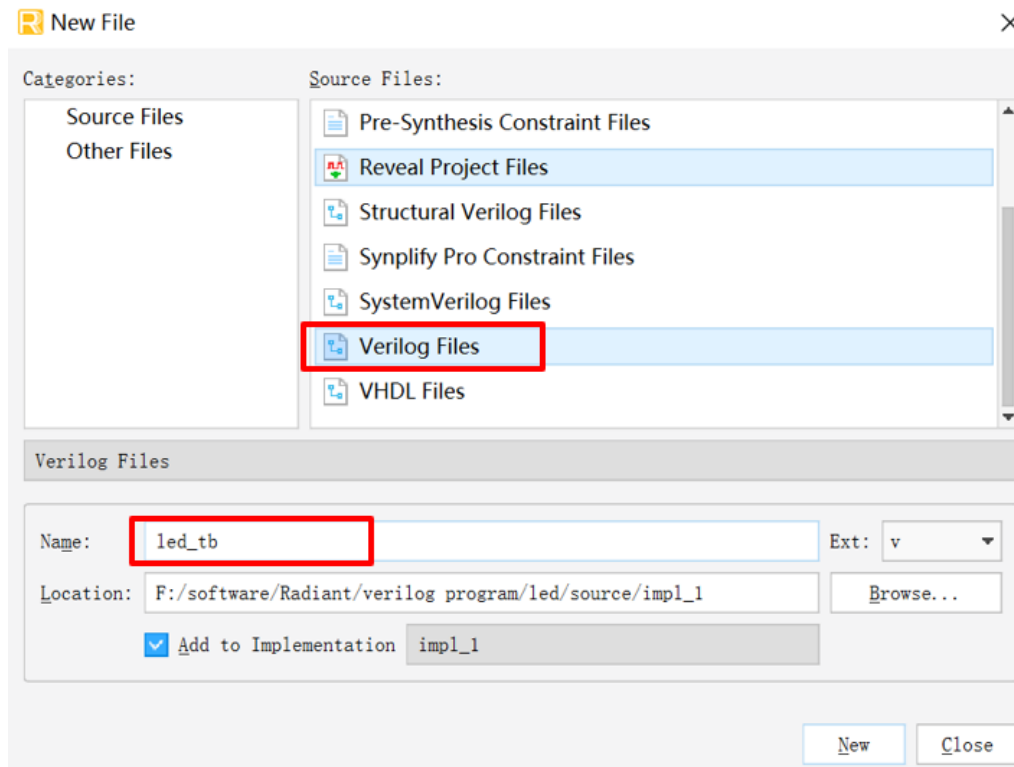


^ 简略信息

2.4 仿真工具 Modelsim

2.4.1 仿真流程

1、创建.v 文件，命名为 led_tb。



2、将测试代码输入测试文件中，代码如下：

```
`timescale 1ns/100ps

module led_tb();
    reg          sys_clk;
    wire  [2:0]  RGB_led;

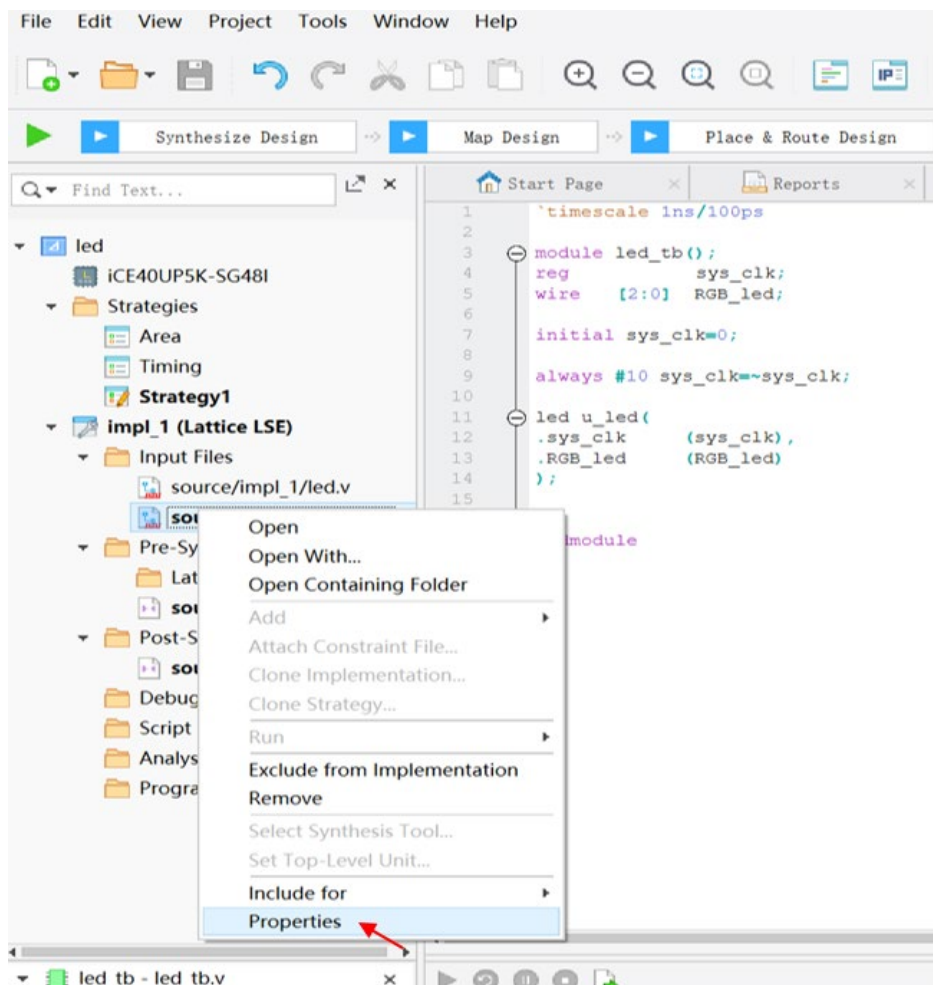
    initial sys_clk=0;

    always #10 sys_clk=~sys_clk;

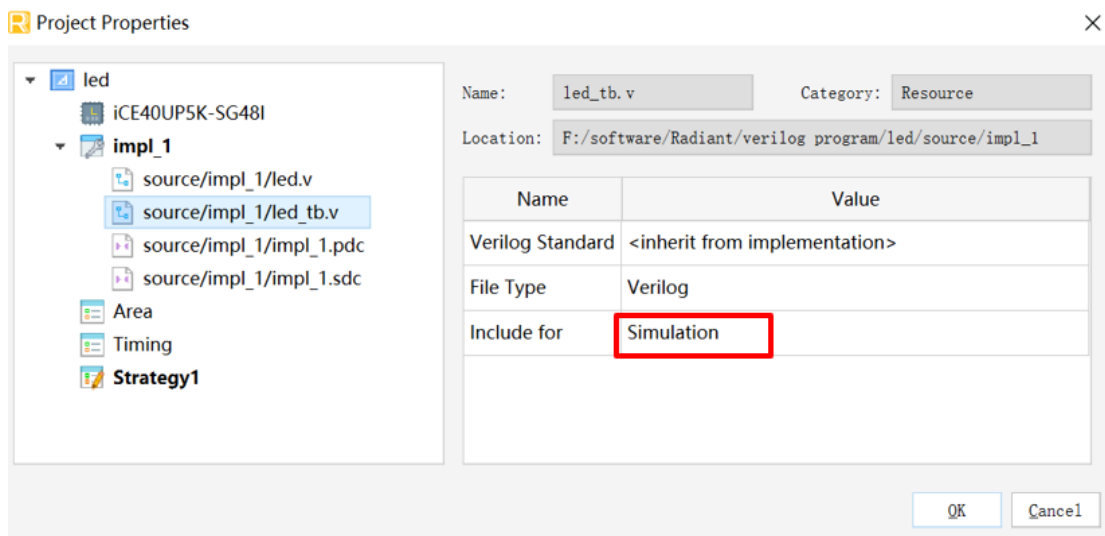
    led u_led(
        .sys_clk      (sys_clk),
        .RGB_led      (RGB_led)
    );

endmodule
```

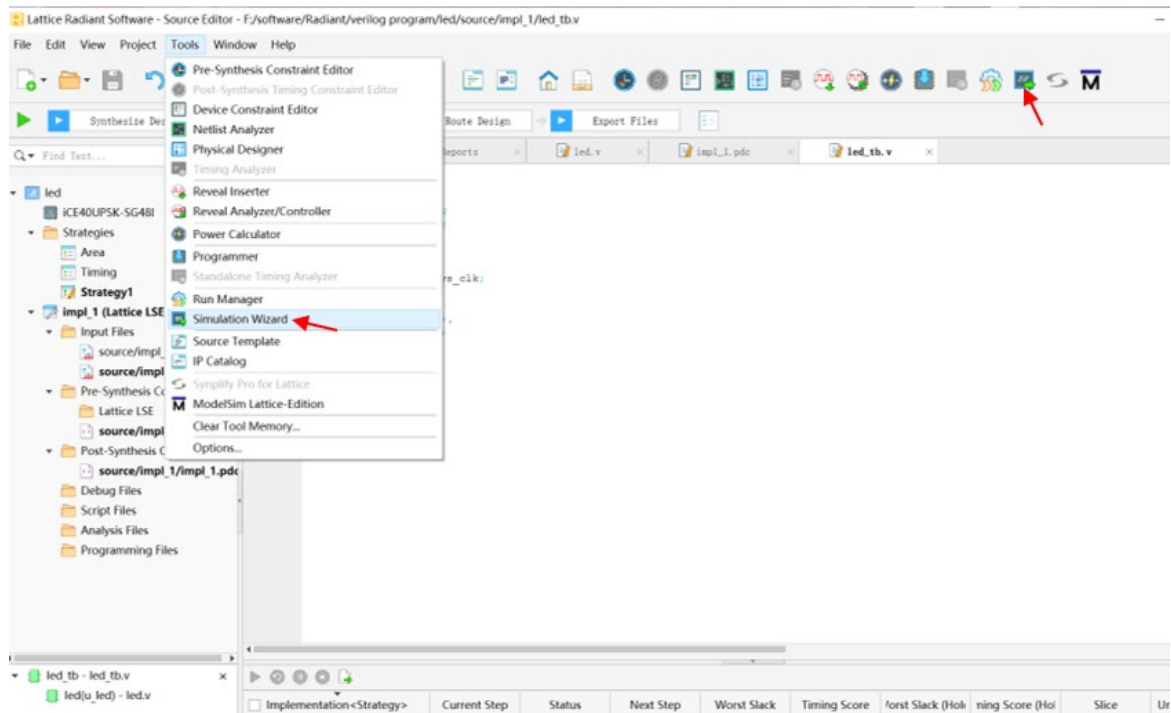
3、这里需要注意测试文件只是用来仿真的，很多语句是不可综合的，所以要设置文件的属性。右击工程目录里的 led_tb 文件，选择 Properties，



会弹出一个属性窗口，弹出一个属性窗口，把测试文件属性值改为 simulation 表示只做仿真。



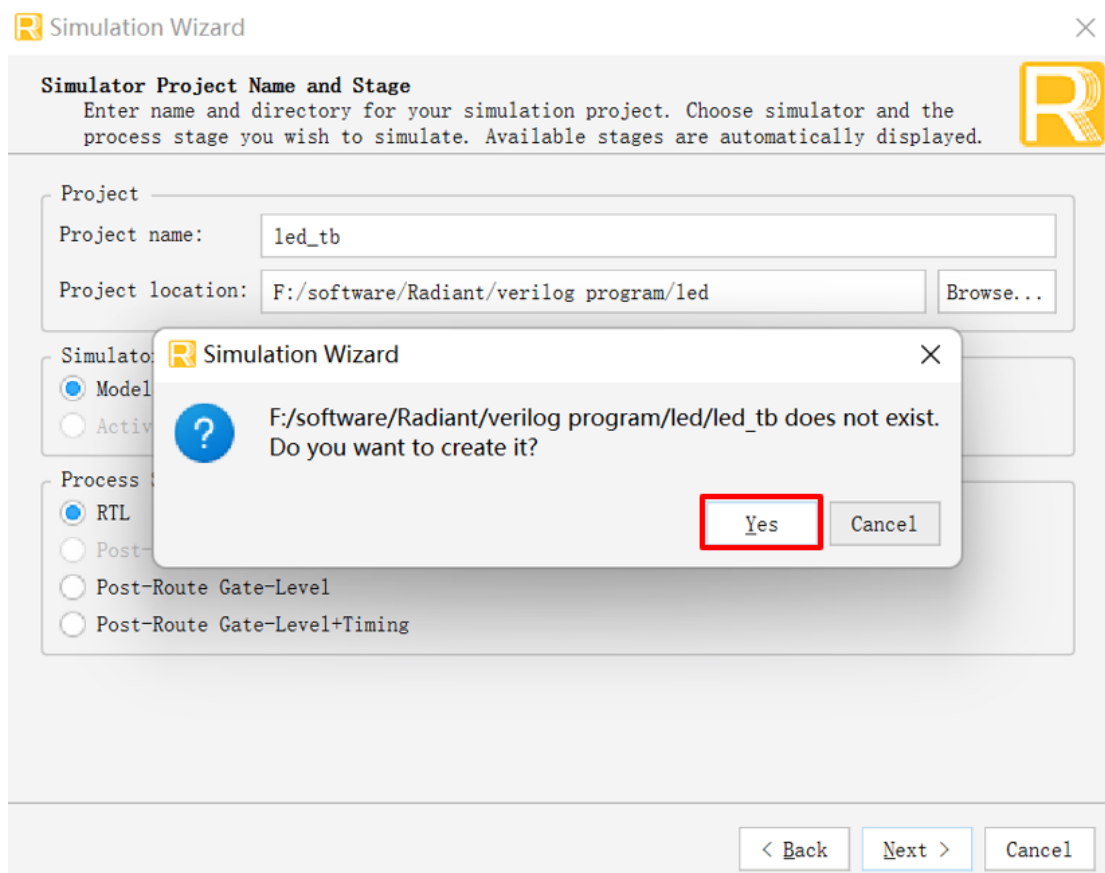
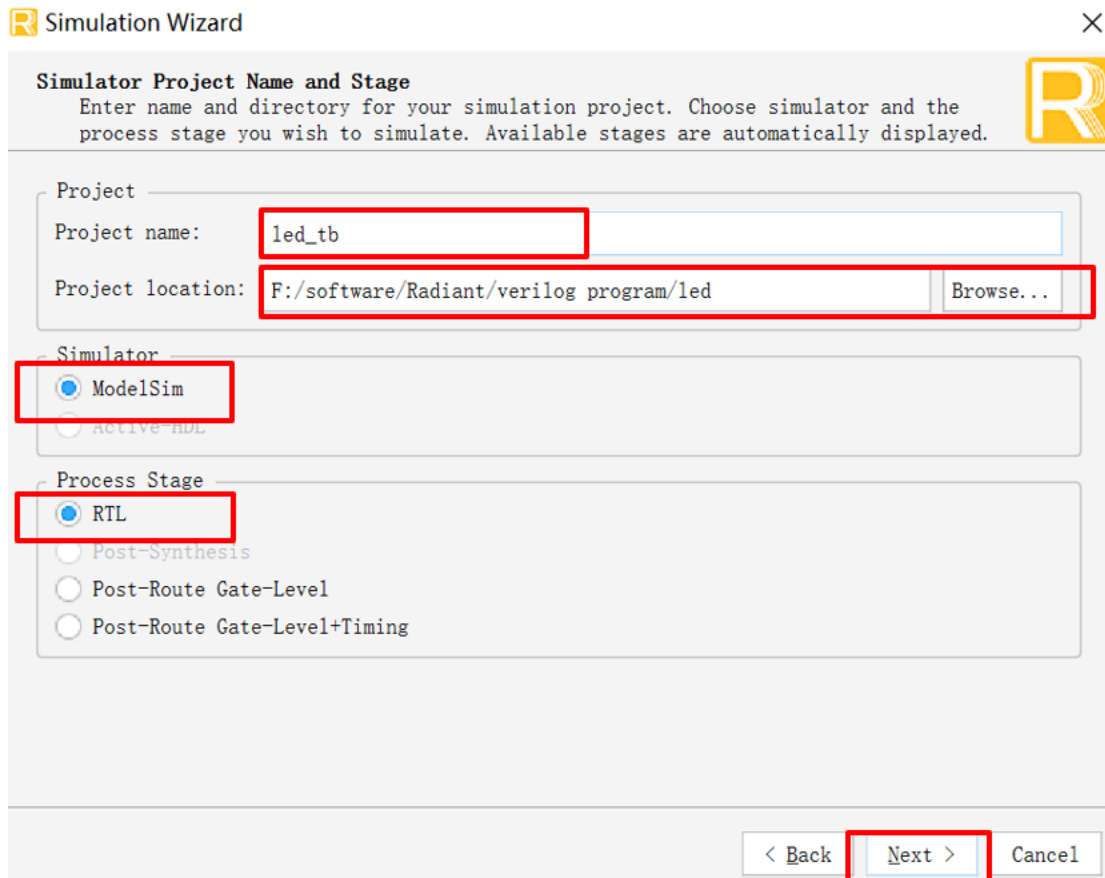
4、选择菜单栏 Tools→Simulation Wizard 或 工具栏中 Simulation Wizard 按钮，进入仿真设置界面。



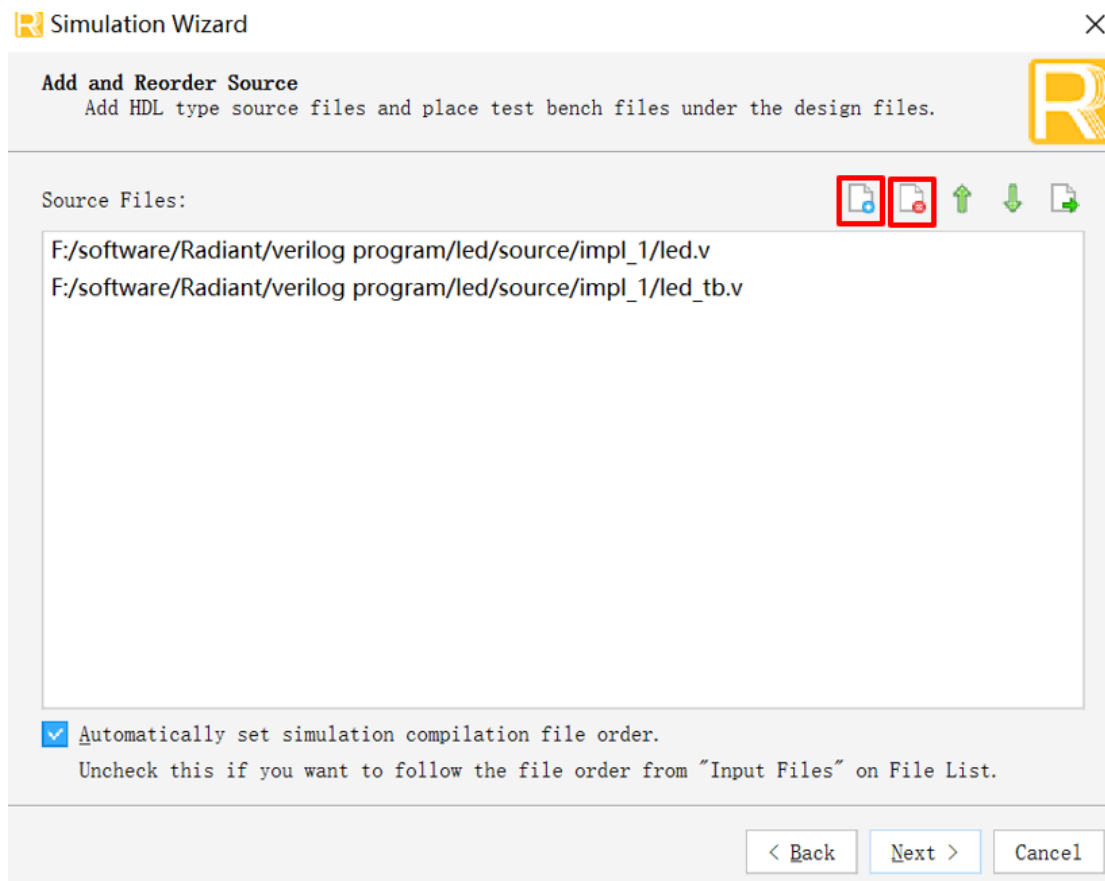
5、直接点击 Next。



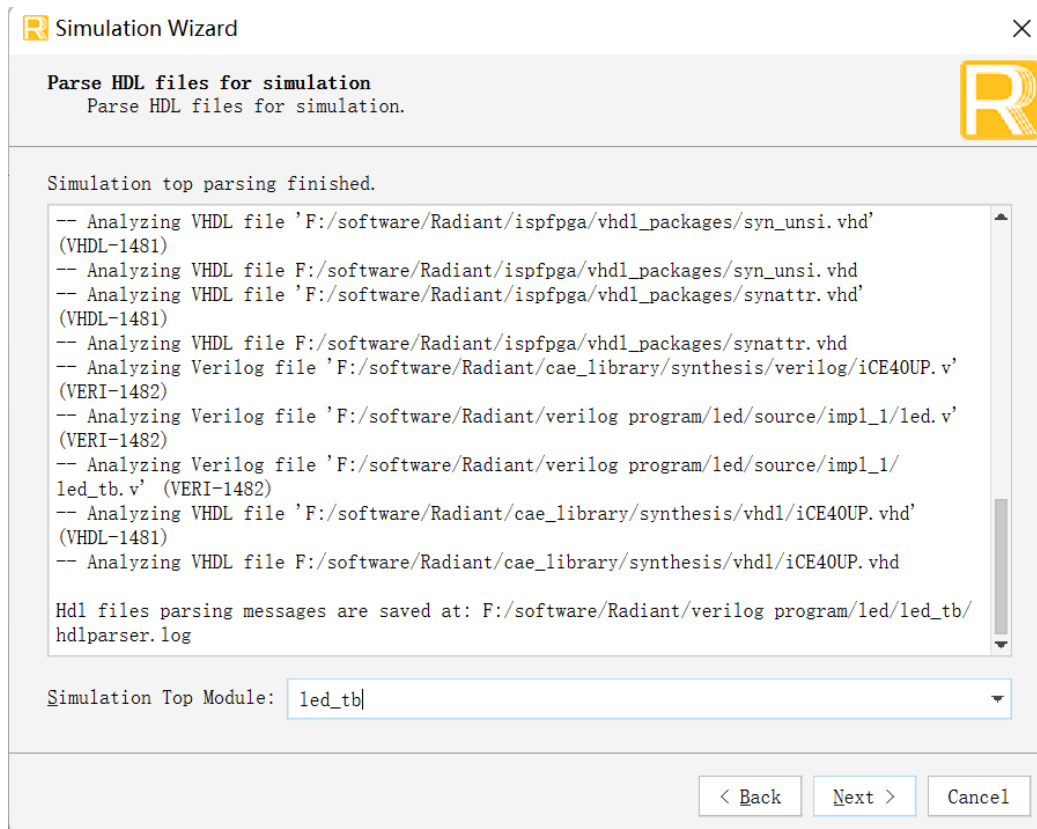
6、创建仿真文件的工程及其路径，选择仿真的阶段，这里是功能仿真，所以只能选 RTL 级。如果没有 led_tb 这个文件夹，点击 Next，然后点击 Yes 会自动创建。



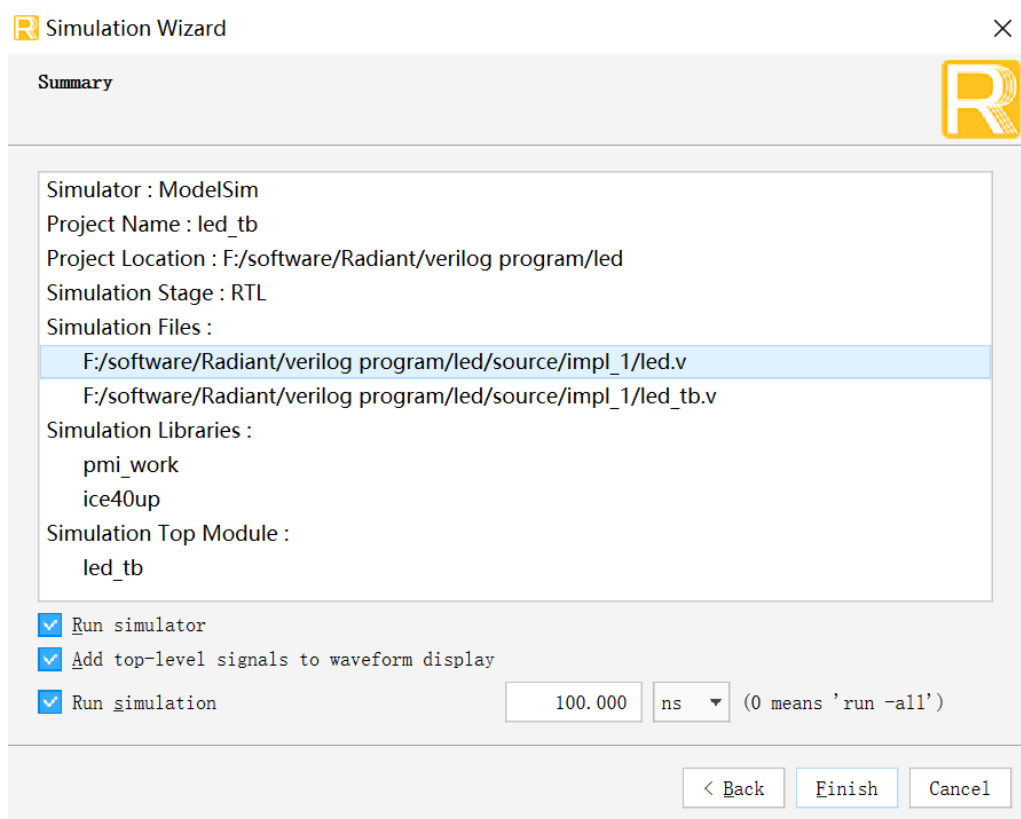
7、选择添加或者删除所要仿真的源文件和测试文件，点击 Next。



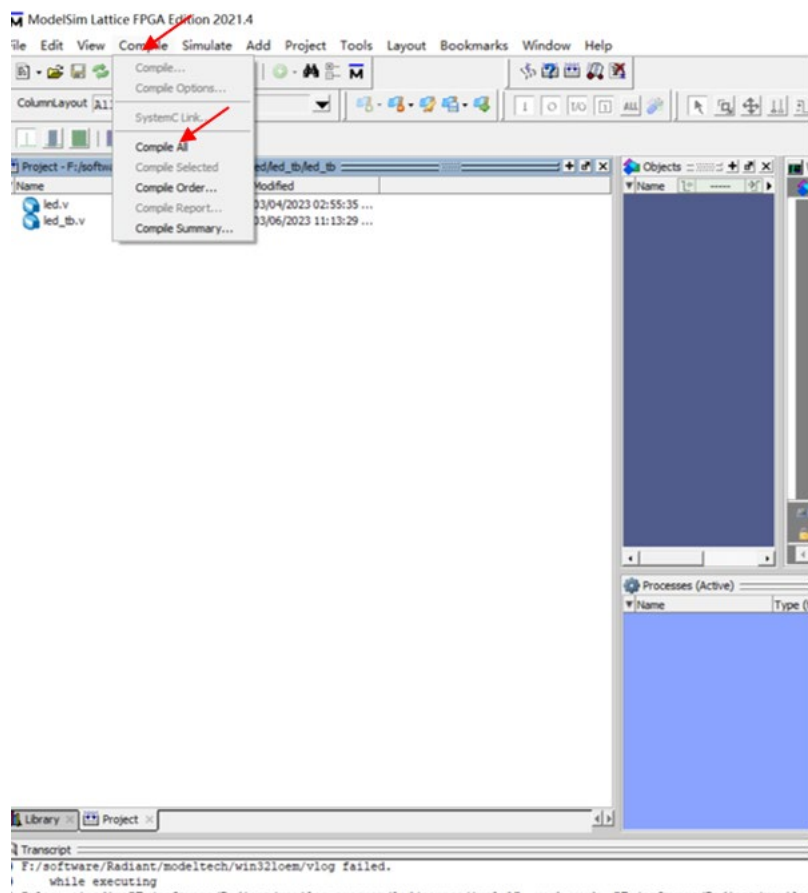
8、仿真工程列表，如果有多个测试文件可以选择顶层测试文件。

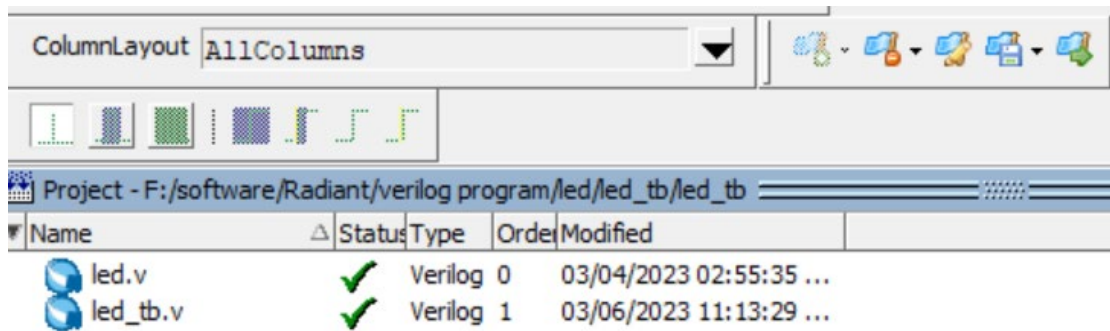


9、仿真工程项目创建完成，如果下方选项确定，点击 Finish，将会打开 Modelsim。

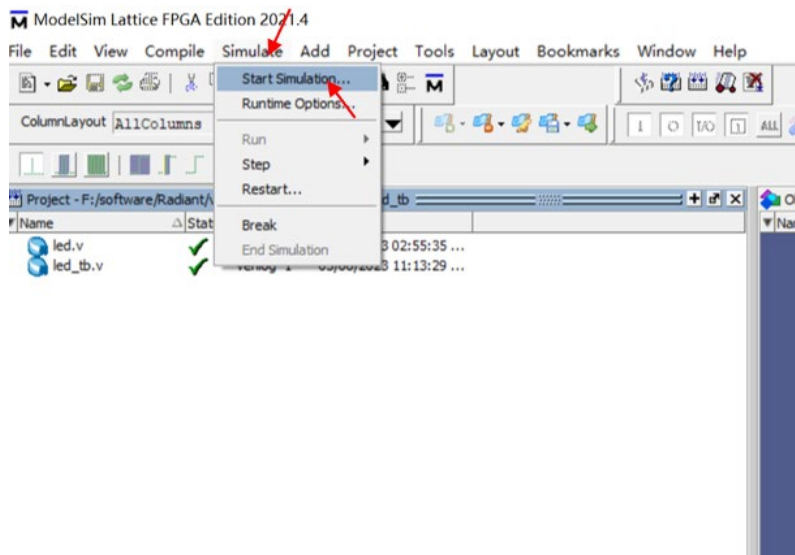


10、选择 Compile→Compile All，等待完成后工程目录的 Status 状态会变成绿色的 ✓。

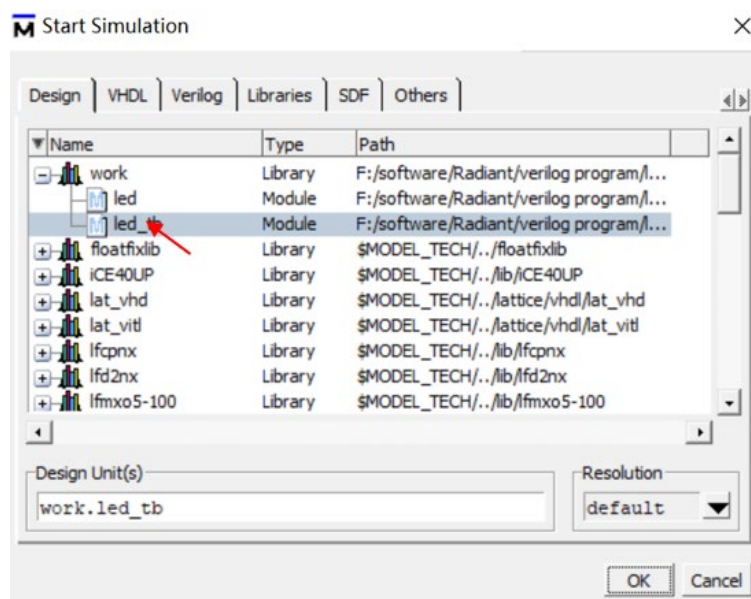




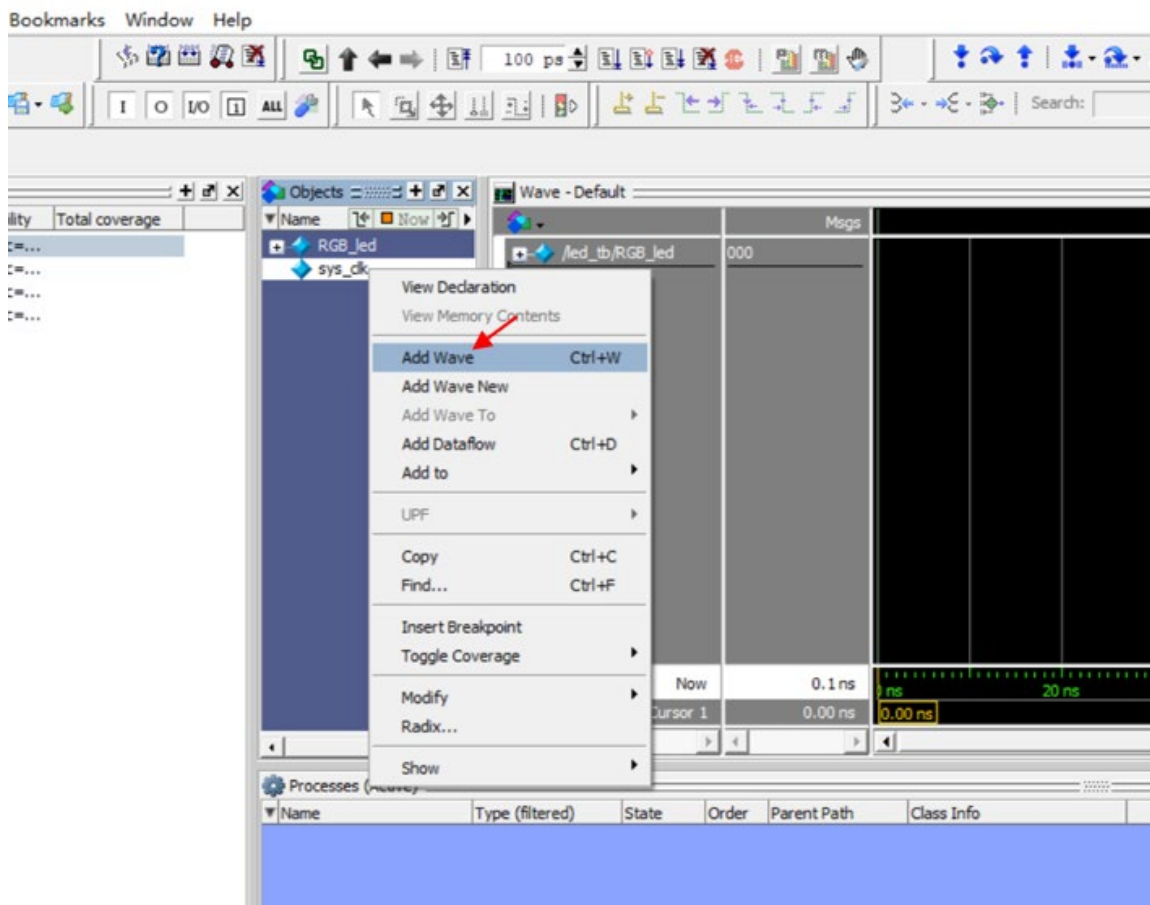
11、选择菜单栏的 Simulate→Start Simulation,



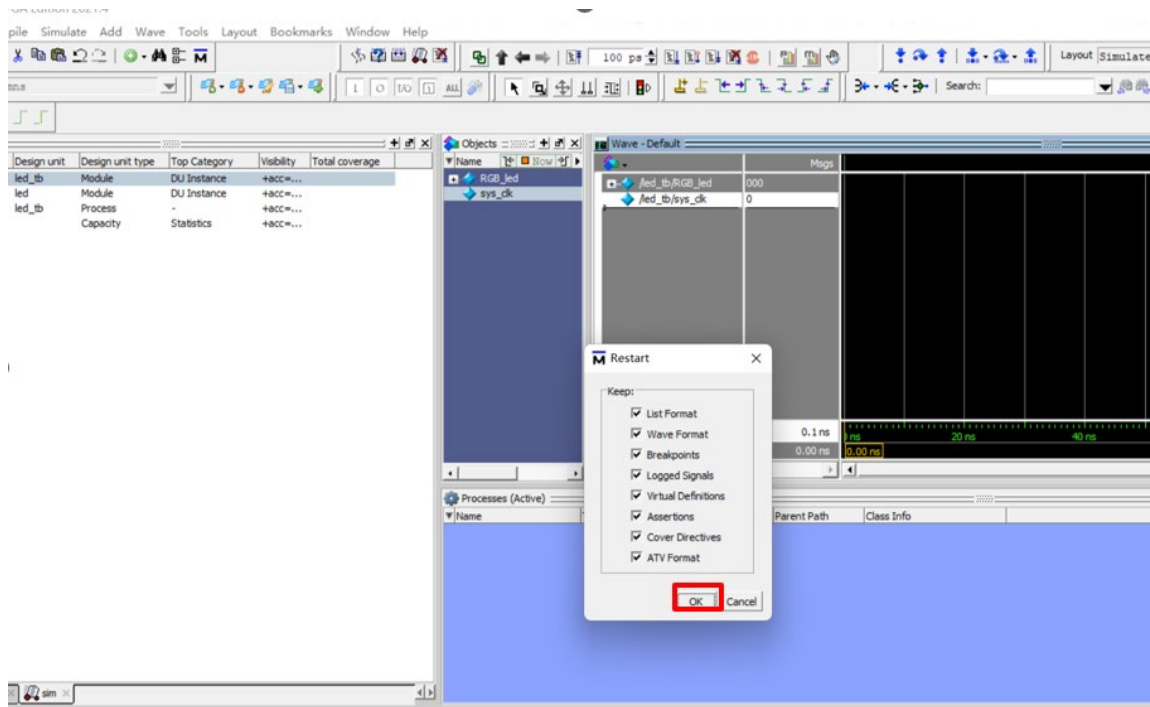
选择 led_tb 文件



12、右击需要观察的信号，点击 Add Wave，这样就将对应信号添加至 Wave 窗口。

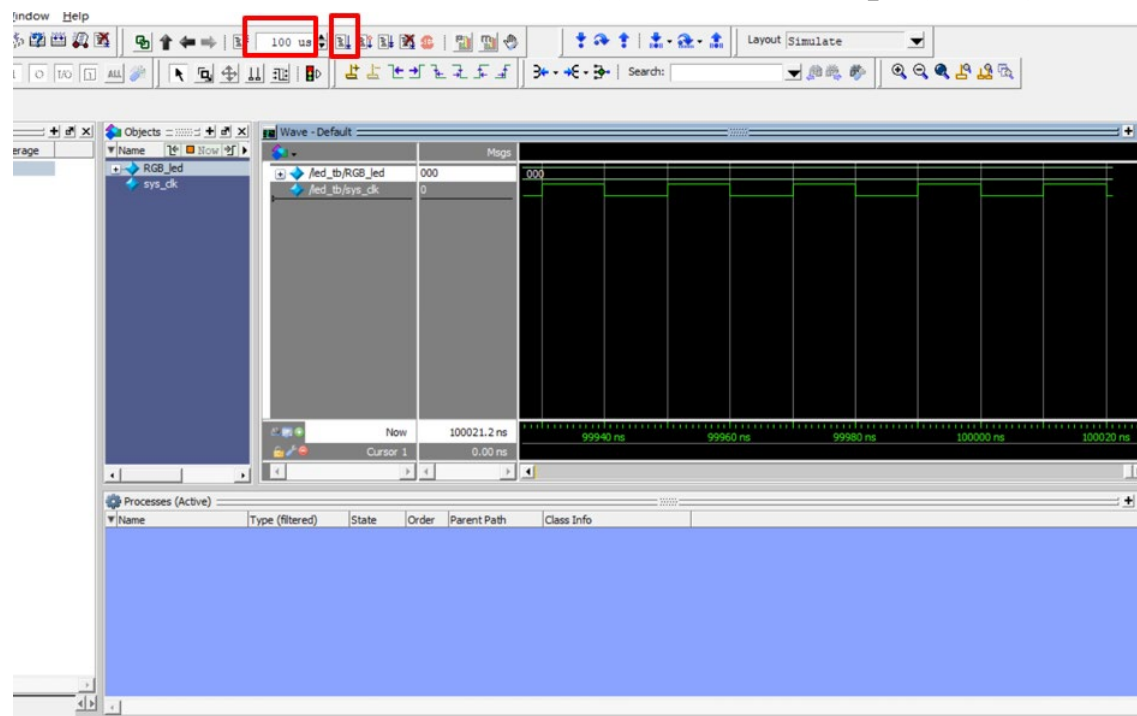


13、弹出的 Wave 窗口如下，选择工具栏中 Restart 按钮，在弹出的 Restart 窗口点击 OK，复位仿真 Wave 窗口。



14、修改工具栏中仿真时间，点击工具栏中 Run 按钮，进行仿真，仿真波形如下，仿真完

成。(由于例程是三色灯一秒切换一个颜色, 而仿真时间太短, 所以 RGB_led 信号不会切换。)



第三章 iCE40UP 入门实验

实验简介

本教程以实验为主，结合相应的实验案例，以详细的讲解和实验步骤让大家尽快熟悉 FPGA 开发的基本流程、Ridiant 软件平台的使用方法以及本节实验相关模块的工作原理。

- 熟悉和掌握 FPGA 开发的基本流程
- 掌握 Ridiant 软件平台的基本使用方法
- 掌握使用板卡的内部时钟和外部时钟点亮 RGB LED 的方法
- 掌握使用 UART 串口通讯原理
- 掌握使用 Verilog HDL 语言基于 FPGA 实现 UART 通讯和点亮 RGB LED 的原理及实现方法

实验环境

- 硬件环境：STEP FPGA 实验平台
- 软件环境：Ridiant 开发环境

3.1 实验一 使用内部时钟和外部时钟点亮 RGB LED

3.1.1 实验任务

在这个实验里我们将学习使用 iCE40UP 控制板卡上的 RGB 三色 LED 的显示，每秒 RGB LED 显示一种颜色，直至熄灭，一共显示 8 种颜色。

3.1.2 实验原理

iCE40UP 上有一个三色 LED，这个三色 LED 采用的是共阳极设计，RGB 三种信号分别连接到 FPGA 的引脚，作为 FPGA 输出信号控制。当 FPGA 输出低电平时 LED 变亮，当 FPGA 输出高电平时 LED 熄灭，当两种或者三种颜色变亮时会混合出不同颜色，一共能产生 8 种颜色。



```

        clk_1s      <= ~clk_1s;
    end
    else begin
        cnt_clk_1s <= cnt_clk_1s+1'b1;
        clk_1s<=clk_1s;
    end
end

always@(posedge clk_1s)begin
    if(cnt_led==3'b111)
        cnt_led<=1'b0;
    else
        cnt_led<=cnt_led+1'b1;
end
assign RGB_led = cnt_led;
endmodule

```

(2) 内部时钟驱动 RGB LED。

```

module led(
output [2:0] RGB_led
);
reg    [23:0]  cnt_clk_1s = 1'b0    ;
reg          clk_1s      = 1'b0    ;
reg    [2:0]   cnt_led    = 1'b0    ;
wire  sys_clk;
//内部 12M 时钟
HSOSC #(
.CLKHF_DIV ("0b10")//0b00 = 48 MHz, 0b01 = 24 MHz, 0b10 = 12 MHz, 0b11 = 6 MHz
) OSCInst0 (
.CLKHFEN (1'b1    ),
.CLKHFPU (1'b1    ),
.CLKHF   (sys_clk)
);

always@ (posedge sys_clk)begin
    if(cnt_clk_1s==24'd599_0000)begin
        cnt_clk_1s <= 1'b0;
        clk_1s      <= ~clk_1s;
    end
end

```

```

        end
    else begin
        cnt_clk_1s <= cnt_clk_1s+1'b1;
        clk_1s<=clk_1s;
    end
end

always@(posedge clk_1s )begin
    if(cnt_led==3'b111)
        cnt_led<=1'b0;
    else
        cnt_led<=cnt_led+1'b1;
end
assign RGB_led = cnt_led;
endmodule

    else
        cnt_led<=cnt_led+1'b1;
end
assign RGB_led = cnt_led;
endmodule

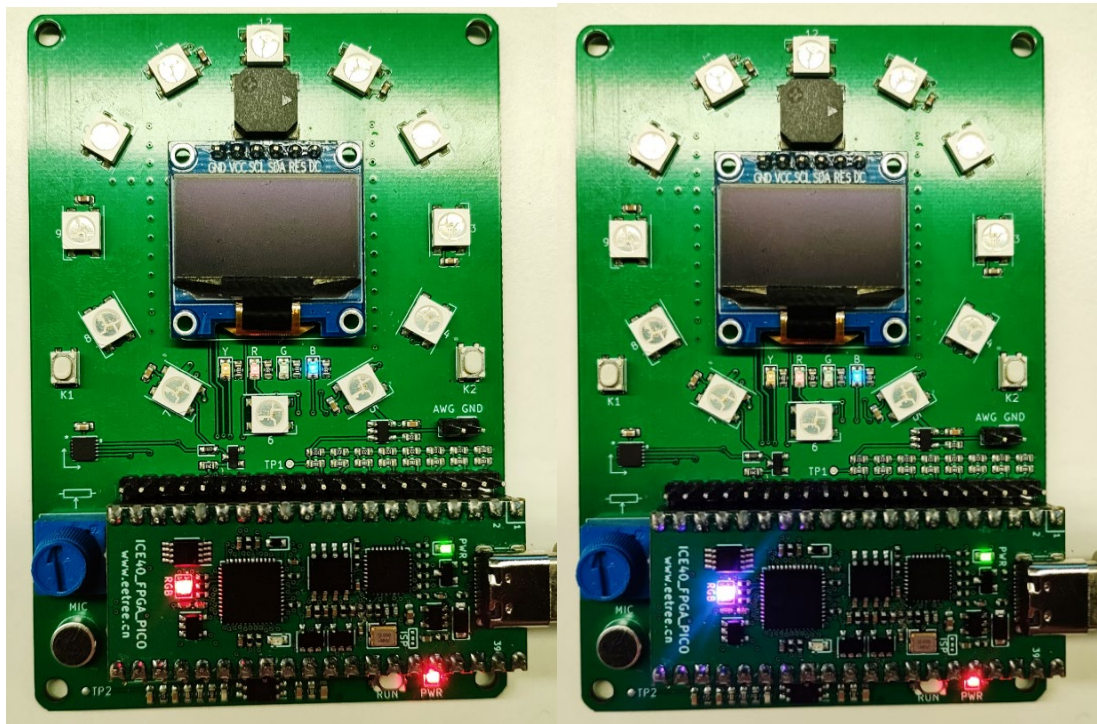
```

3.1.4 实验步骤

- 1.打开 Ridian, 建立工程。
- 2.新建 Verilog HDL 设计文件, 并键入设计代码, 包括添加需要调用的模块文件。
- 3.构建并输出编程文件, 烧写至 FPGA 的 Flash 之中。

3.1.5 实验现象

实验结果如下:



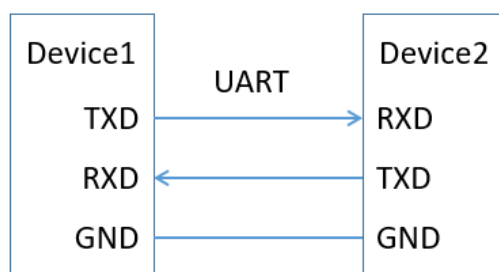
上图为烧入代码之后的结果，可以发现 RGB LED 一秒变换一个颜色。

3.2 实验二 UART 串口通讯实验

3.2.1 实验任务

在这个实验里我们将学习使用 UART 串口与上位机进行通讯，并且在复位时候向上位机发送 step。

3.2.2 实验原理



UART 通信接口

在嵌入式领域里说的串口一般就是说的 UART 接口，通用异步收发传输器（Universal Asynchronous Receiver/Transmitter），通常称作 UART，是一种通用串行数据总线，用于异步通信。该总线双向通信，可以实现全双工传输和接收。

对于 UART 总线，TXD 和 RXD 分别用于发送和接收数据，相当于两根独立工作的单线总线，没有了时钟线的配合，那么接收端应该怎样获取发送端传输的数据呢？其实也是有方法的，那就是通信双方需要约定好 UART 总线数据传输的**通信速率**和**时序格式**。

通信速率：

UART 的数据传输速度用波特率来描述，也就是 UART 每秒接收或发送的数据位。例如 9600 波特率表示每秒钟发送或接收 9600 比特的数据，即发送端需要将发送的每个数据位保持对应的时间，计算如下：

$$\text{➤ } 1\text{s} / 9600 = 1000000\text{us} / 9600 = 104.17\text{us}$$

小脚丫硬件上使用 12MHz 的时钟晶振，如果以 12MHz 时钟信号作为系统时钟，使用计数器延时完成 UART 通信数据采样，那么计数器延时计数终值计算如下：

$$\text{➤ } 12\text{M} / 9600 = 1250$$

因为波特率是协议里约定的，为保证协议的通用性和灵活性，波特率参数有固定的选项，不可以随意设置（如果 UART 通信双方都是自己编程的，可以根据自己的要求定义自己需要的波特率，这种情况除外），波特率参数选项很多，大家可以打开串口调试助手工具找到波特率配置列表查看，我们比较常用的波特率值有以下几种：

UART 常用波特率：

1200	4800	9600	38400	115200
------	------	------	-------	--------

时序格式：

通信过程中时序依次为：起始位、数据位、校验位、停止位、空闲位，其中数据位可以是 5~8 位，本设计我们使用 8 位数据，校验位可以省略，最后确定的时序格式如下：

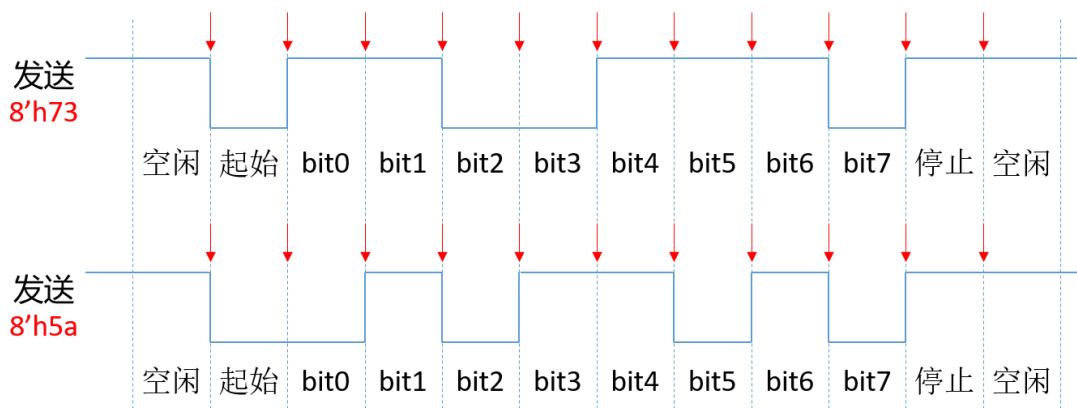


本实验 UART 通信时序

前面所说的通信速率和时序格式其实就是 UART 通信中的两个重要的参数，需要传输的数据根据通信速率的节拍按照 UART 的时序格式输出，就可以实现 UART 通信了，可以按照下面三个步骤实现。

- 1) 将需要发送的数据与起始位和停止位组成 10bit 位宽的数据
- 2) 计数器计数延迟产生相应波特率需要的时序节拍
- 3) 数据按照（起始位—bit0~bit7—停止位）的时序串行输出

例如，将 8'h73 和 8'h5a 通过 UART 发送的时序，红色箭头为波特率对应的节拍点

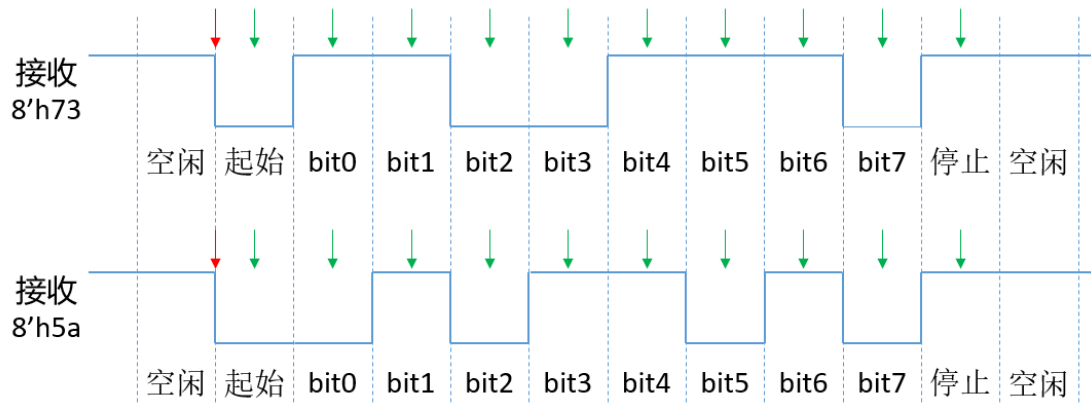


UART 发送数据实例

对于 UART 发送数据来说，波特率节拍是自己产生的，数据是自己主动发出的，逻辑相对简单，而当 UART 接收数据的时候，因为不确定对方什么时候发送数据，所以需要对 RX 信号持续检测，当检测到有数据传送时，根据约定的波特率节拍采样，可以按照下面三个步骤实现。

- 1) 检测 UART 的 RXD 信号的下降沿（自锁，完成接收后再解锁继续检测）
- 2) 接收采样时，采样点应该在计数器的中值点进行
- 3) 将采样后的数据按照 UART 时序的要求重新组成 8bit 的数据

例如，当 UART 的 RX 端接收到数据 8'h73 和 8'h5a 的时候，红色箭头为检测到数据传输的点，绿色箭头为对应的采样节拍点（采样点在数据中间最是稳定）。



UART 接收数据实例

通过以上理论，我们了解了 UART 发送和接收数据的整个流程，两个过程中我们都需要波特率节拍，那么我们就可以设计一个节拍模块 Baud，这样我们的发送和接收都可以实例化节拍模块用于产生对应波特率的节拍信号。

3.2.3 Verilog HDL 建模描述

(1) 顶层文件 step.v。

```
// *****
// File name : step
// Module name : step
//
// Author      : STEP
// Date        : 2023/3/31
// Version     : V 1.0
// Description  : 顶层模块。
//
// Modification history
//-----
// $Log$
//
// *****

module step
(
```

```

    input          rst_n      ,          // system reset, active low
    input          fpga_rx   ,
    output         fpga_tx   //UART 发送输出
);

HSOSC #(
.CLKHF_DIV ("0b10")
) OSCInst0 (
.CLKHFEN (1'b1   ),
.CLKHFPU (1'b1   ),
.CLKHF   (clk)
);

wire rx_data_valid;
wire [7:0]   rx_data;
wire tx_data_valid;
wire [7:0]   tx_data;
//Uart_Bus module
Uart_Bus u1
(
.clk                (clk                ), //系统时钟 12MHz
.rst_n              (rst_n              ), //系统复位，低有效
//负责 FPGA 接收 UART 芯片的数据
.uart_rx            (fpga_rx            ), //UART 接收输入
//负责 FPGA 发送数据给 UART 芯片
.step_data_valid    (tx_data_valid      ),
.step_data          (tx_data            ),
.uart_tx            (fpga_tx            )
);

control control_s
(
.clk                (clk                ), // system clock
.rst_n              (rst_n              ), // system reset, active low
.tx_data_valid      (tx_data_valid),
.tx_data_out        (tx_data            )

```

```
);
endmodule
```

(2) center_control.v。

```
// *****
// File name : center_control
// Module name : control
//
// Author      : STEP
// Date        : 2023/3/31
// Version     : V 1.0
// Description  : 负责将“step”给 uart 发送给上位机。
//
// Modification history
//-----
// $Log$
//
// *****

module control
(
    input          clk,          // system clock
    input          rst_n,        // system reset, active low
    output reg     tx_data_valid,
    output reg [7:0] tx_data_out
);

reg temp_tx_flag;
reg [23:0] cnt_uart;
```

```

always @(posedge clk or negedge rst_n)begin
    if(!rst_n)
        temp_tx_flag <= 1'b1;
    else if(cnt_uart == 24'd8_599_999)
        temp_tx_flag <= 1'b0;
    else
        temp_tx_flag <= temp_tx_flag;
end

always @(posedge clk or negedge rst_n)begin //计数
    if(!rst_n) cnt_uart <= 1'b0;
    else if(cnt_uart >= 24'd11_999_999) cnt_uart <= 1'b0;
    else if (temp_tx_flag) cnt_uart <= cnt_uart + 1'b1;
    else cnt_uart <= 24'd0;
end

always @(posedge clk or negedge rst_n)begin //*****每秒发一个温度
时间数据，字符串显示*****//
    if(!rst_n) begin
        tx_data_valid <= 1'b0;
        tx_data_out <= 1'b0;
    end
    else begin
        if(temp_tx_flag)begin //发送温度数据
            if(cnt_uart == 24'd6_099_999 ) begin //发送 s
                tx_data_valid <= 1'b1;
                tx_data_out <= {1'b0,7'h73};
            end
            else if(cnt_uart == 24'd6_599_999 ) begin //发送 t
                tx_data_valid <= 1'b1;
                tx_data_out <= {1'b0,7'h74};
            end
            else if(cnt_uart == 24'd7_099_999 ) begin //发送 e
                tx_data_valid <= 1'b1;

```

```

        tx_data_out <= {1'b0,7'h65};
    end
    else if(cnt_uart == 24'd7_599_999 ) begin //发送 p
        tx_data_valid <= 1'b1;
        tx_data_out <= {1'b0,7'h70};
    end
    else if(cnt_uart == 24'd8_099_999 ) begin //换行
        tx_data_valid <= 1'b1;
        tx_data_out <= 8'd10;
    end
    else begin
        tx_data_valid <= 1'b0; tx_data_out <= tx_data_out;
    end
end
else begin
    tx_data_valid <= 1'b0; tx_data_out <= 8'd0;
end
end
end
endmodule

```

(3) uart_bus.v。

```

// *****
// File name   :   uart_bus
// Module name  :   Uart_Bus
//
// Author      :   STEP
// Date        :   2023/3/31
// Version     :   V 1.0
// Description  :   uart 串口的顶层模块。
//
// Modification history
//-----
// $Log$
//

```

```
// *****

module Uart_Bus #
(
parameter          BPS_PARA = 1250    //12MHz 时钟时参数 1250 对应 9600 的
波特率
)
(
input              clk,                //系统时钟 12MHz
input              rst_n,              //系统复位，低有效

input              uart_rx,            //UART 接收输入
input              step_data_valid,    //发送数据有效脉冲
input              [7:0] step_data,    //要发送的数据
output             uart_tx             //UART 发送输出
);

wire              rx_data_valid; //接收数据有效脉冲
wire              [7:0] rx_data_out; //接收到的数据

//////////////////////////UART 接收功能模块例化
//////////////////////////

wire              bps_en_rx,bps_clk_rx;

//UART 接收波特率时钟控制模块 例化
Baud #
(
.BPS_PARA          (BPS_PARA          )
)
Baud_rx
(
.clk                (clk                ), //系统时钟 12MHz
.rst_n              (rst_n              ), //系统复位，低有效
.bps_en              (bps_en_rx          ), //接收时钟使能
.bps_clk             (bps_clk_rx         ) //接收时钟输出
);
```

```

//UART 接收数据模块 例化
Uart_Rx Uart_Rx_uut
(
.clk                (clk                ), //系统时钟 12MHz
.rst_n              (rst_n              ), //系统复位，低有效

.bps_en             (bps_en_rx          ), //接收时钟使能
.bps_clk            (bps_clk_rx         ), //接收时钟输入

.uart_rx            (uart_rx            ), //UART 接收输入
.rx_data_valid      (rx_data_valid), //接收数据有效脉冲
.rx_data_out        (rx_data_out        ) //接收到的数据
);

////////////////////////////////////////UART 发送功能模块例化
////////////////////////////////////////
wire                bps_en_tx,bps_clk_tx;

//UART 发送波特率时钟控制模块 例化
Baud #
(
.BPS_PARA           (BPS_PARA           )
)
Baud_tx
(
.clk                (clk                ), //系统时钟 12MHz
.rst_n              (rst_n              ), //系统复位，低有效
.bps_en             (bps_en_tx          ), //发送时钟使能
.bps_clk            (bps_clk_tx         ) //发送时钟输出
);

//UART 发送数据模块 例化
Uart_Tx Uart_Tx_uut
(
.clk                (clk                ), //系统时钟 12MHz
.rst_n              (rst_n              ), //系统复位，低有效

```



```

.bps_en            (bps_en_tx      ), //发送时钟使能
.bps_clk           (bps_clk_tx      ), //发送时钟输入
.step_data_valid   (step_data_valid), //step 的有效脉冲
.step_data         (step_data      ), //step
.tx_data_valid     (bps_en_rx       ), //发送数据有效脉冲
.tx_data_in        (rx_data_out     ), //要发送的数据
.uart_tx           (uart_tx        )  //UART 发送输出
);

endmodule

```

(4) uart_r。

```

// *****
// File name : uart_r
// Module name : Uart_Rx
//
// Author      : STEP
// Date        : 2023/3/31
// Version     : V 1.0
// Description  : uart 串口的接收模块。
//
// Modification history
//-----
// $Log$
//
// *****

module Uart_Rx
(
input          clk,          //系统时钟 12MHz
input          rst_n,        //系统复位，低有效

output reg     bps_en,       //接收时钟使能
input          bps_clk,      //接收时钟输入
input          uart_rx,      //UART 接收输入
output reg     rx_data_valid, //接收数据有效脉冲
output reg     [7:0] rx_data_out //接收到的数据

```

```

);

reg  uart_rx0,uart_rx1,uart_rx2;
//多级延时锁存去除亚稳态
always @ (posedge clk) begin
    uart_rx0 <= uart_rx;
    uart_rx1 <= uart_rx0;
    uart_rx2 <= uart_rx1;
end

//检测 UART 接收输入信号的下降沿
wireneg_uart_rx = uart_rx2 & ~uart_rx1;

reg          [3:0]    num;
//接收时钟使能信号的控制
always @ (posedge clk or negedge rst_n) begin
    if(!rst_n)
        bps_en <= 1'b0;
    else if(neg_uart_rx && (!bps_en))    //当空闲状态（bps_en 为低电平）时检测到 UART
接收信号下降沿，进入工作状态（bps_en 为高电平），控制时钟模块产生接收时钟
        bps_en <= 1'b1;
    else if(num==4'd9)                    //当完成一次 UART 接收操作后，退出工作状
态，恢复空闲状态
        bps_en <= 1'b0;
end

reg          [7:0]    rx_data;
//当处于工作状态中时，按照接收时钟的节拍获取数据
always @ (posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        num <= 4'd0;
        rx_data <= 8'd0;
    end else if(bps_en) begin
        if(bps_clk) begin
            num <= num + 1'b1;
            if(num<=4'd8) rx_data[num-1] <= uart_rx1; //先接受低位再接收高位，8 位有
效数据

```

```

        end else if(num == 4'd9) begin                                //完成一次 UART 接收操作后，将
        获取的数据输出
            num <= 4'd0;
        end
    end else begin
        num <= 4'd0;
    end
end
end

//将接收的数据输出，同时控制输出有效信号产生脉冲
always @ (posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        rx_data_out <= 8'd0;
        rx_data_valid <= 1'b0;
    end else if(num == 4'd9) begin
        rx_data_out <= rx_data;
        rx_data_valid <= 1'b1;
    end else begin
        rx_data_out <= rx_data_out;
        rx_data_valid <= 1'b0;
    end
end
end

endmodule

```

(5) uart_s。

```

// *****
// File name :   uart_s
// Module name :   Uart_Tx
//
// Author       :   STEP
// Date         :   2023/3/31
// Version      :   V 1.0
// Description   :   uart 串口的发送模块。
//
// Modification history
//-----
// $Log$

```

```
//
// *****

module Uart_Tx
(
input          clk,          //系统时钟 12MHz
input          rst_n,        //系统复位，低有效

output reg     bps_en,        //发送时钟使能
input          bps_clk,       //发送时钟输入
input          step_data_valid,
input          [7:0] step_data,
input          tx_data_valid, //发送数据有效脉冲
input          [7:0] tx_data_in, //要发送的数据
output reg     uart_tx        //UART 发送输出
);

reg [3:0] num;
reg [9:0] tx_data_r; //融合了起始位和停止位的数据

reg [7:0] tx_data;
reg en_flag;
always @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        tx_data <= 8'd0;
        en_flag <= 1'b0;
    end
    else if(step_data_valid)begin
        tx_data <= step_data;
        en_flag <= 1'b1;
    end
    else if(tx_data_valid)begin
        tx_data <= tx_data_in;
        en_flag <= 1'b1;
    end
end
```

```

else begin
    tx_data <= 8'd0;
    en_flag <= 1'b0;
end
end

//驱动发送数据操作
always @ (posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        bps_en <= 1'b0;
        tx_data_r <= 10'd0;
    end else if(en_flag && (!bps_en))begin
        bps_en <= 1'b1;        //当检测到接收时钟使能信号的下降沿，表明接收完成，需要发送数据，使能发送时钟使能信号
        tx_data_r <= {1'b1,tx_data,1'b0};
    end else if(num==4'd10) begin
        bps_en <= 1'b0;        //一次 UART 发送需要 10 个时钟信号，然后结束
    end
end

//当处于工作状态中时，按照发送时钟的节拍发送数据
always @ (posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        num <= 1'b0;
        uart_tx <= 1'b1;
    end else if(bps_en) begin
        if(bps_clk) begin
            num <= num + 1'b1;
            uart_tx <= tx_data_r[num];
        end else if(num>=4'd10) begin
            num <= 4'd0;
        end
    end
end

endmodule

```

(6) Baud.v。

```
// *****
// File name : Baud
// Module name : Baud
//
// Author      : STEP
// Date        : 2023/3/31
// Version     : V 1.0
// Description  : 产生波特率节拍, 给 uart 发送和接收模块产生相应的波特率节拍信号。
//
// Modification history
//-----
// $Log$
//
// *****

module Baud #
(
parameter          BPS_PARA = 1250 //12MHz 时钟时参数 1250 对应 9600 的波特
率
)
(
input              clk,      //系统时钟
input              rst_n,    //系统复位, 低有效
input              bps_en,   //接收或发送时钟使能
output reg         bps_clk   //接收或发送时钟输出
);

reg [12:0] cnt;
//计数器计数满足波特率时钟要求
always @ (posedge clk or negedge rst_n) begin
    if(!rst_n)
        cnt <= 1'b0;
    else if((cnt >= BPS_PARA-1)||(!bps_en)) //当时钟信号不使能 (bps_en 为低电平) 时, 计
数器清零并停止计数
        cnt <= 1'b0;                      //当时钟信号使能时, 计数器对系统时钟计
数, 周期为 BPS_PARA 个系统时钟周期
end
```

```

        else
            cnt <= cnt + 1'b1;
        end

//产生相应波特率的时钟节拍，接收模块将以此节拍进行 UART 数据接收
always @ (posedge clk or negedge rst_n) begin
    if(!rst_n)
        bps_clk <= 1'b0;
    else if(cnt == (BPS_PARA>>1)) //右移一位等于除以 2，终值 BPS_PARA 为数据更替点，
    中值数据稳定，做采样点
        bps_clk <= 1'b1;
    else
        bps_clk <= 1'b0;
    end
endmodule

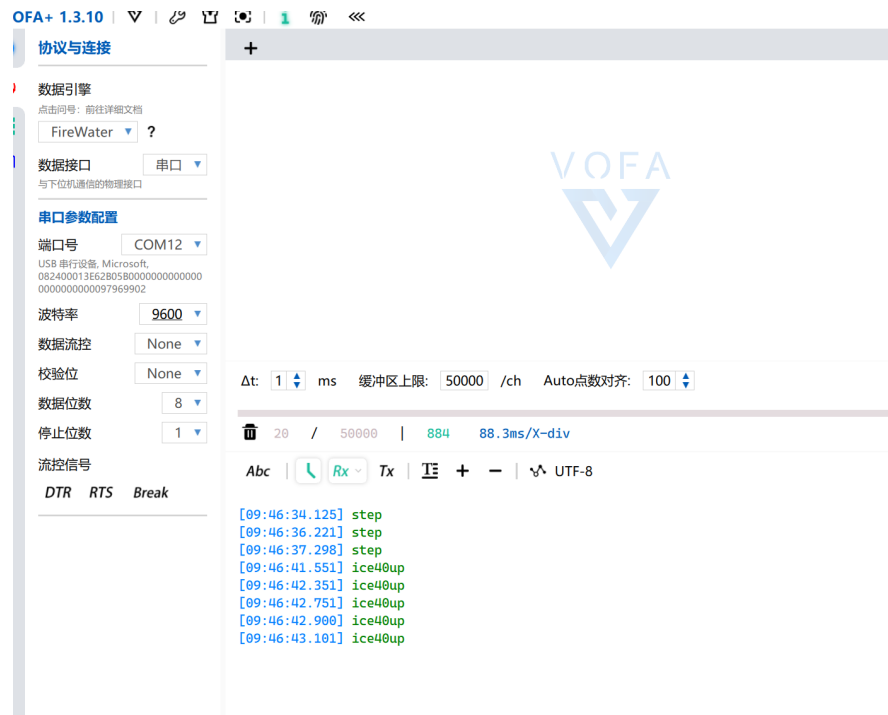
```

3.2.4 实验步骤

1. 打开 Ridian, 建立工程。
2. 新建 Verilog HDL 设计文件，并键入设计代码，包括添加需要调用的模块文件。
3. 构建并输出编程文件，烧写至 FPGA 的 Flash 之中。

3.2.5 实验现象

实验结果如下：



将程序烧录之后，按复位键，FPGA 向上位机发送“step”。在串口助手内输入“ice40up”，则串口向 FPGA 发送 “ice40up”。

3.3 实验三 DS18B20 与 UART 通讯实验

3.3.1 实验任务

在这个实验里我们将学习使用 DS18B20 采集温度，然后将采集的温度通过 UART 串口发送给上位机。

3.3.2 实验原理

UART 串口通讯的原理在实验二已经介绍，DS18B20 温度传感器的原理见章节 。将 DS18B20 采集到的数据乘以 0.0625，得到的就是采集到的实时温度，再将这个数据转换为 bcd 码，再转换为 ASCII 码通过 UART 串口发送给上位机。

3.3.3 Verilog HDL 建模描述

(1) 顶层文件 step.v。

```
// *****  
// File name : step  
// Module name : step  
//  
// Author : STEP  
// Date : 2023/3/31  
// Version : V 1.0  
// Description : 顶层模块。  
//  
// Modification history  
//-----  
// $Log$  
//  
// *****  
  
module step  
(  
    input rst_n, // system reset, active low
```

```

        inout          one_wire,
        output         fpga_tx  //UART 发送输出
    );

    HSOSC #(
        .CLKHF_DIV ("0b10")
    ) OSCInst0 (
        .CLKHFEN (1'b1    ),
        .CLKHFPU (1'b1    ),
        .CLKHF    (clk)
    );

    wire[15:0]    data_out;
    //Drive DS18B20Z to get temperature code
    DS18B20Z DS18B20Z_uut
    (
        .clk          (clk          ),  // system clock
        .rst_n         (rst_n        ),  // system reset, active low
        .one_wire       (one_wire     ),  // ds18b20z one-wire-bus
        .data_out       (data_out     )   // ds18b20z data_out
    );

    // judge sign of temperature
    wire temperature_flag = data_out[15:11]? 1'b0:1'b1;
    // complement if negative
    wire [10:0] temperature_code = temperature_flag? data_out[10:0]:(~data_out[10:0])+1'b1;
    // translate temperature_code to real temperature
    wire [20:0] bin_code = temperature_code * 16'd625;
    wire [24:0] bcd_code; //十位[23:20],个位[19:16],小数位[14:12]
    //Translate binary code to bcd code
    bin_to_bcd bin_to_bcd_uut
    (
        .rst_n          (rst_n        ),  // system reset, active low
        .bin_code        (bin_code     ),  // binary code
        .bcd_code        (bcd_code     )   // bcd code
    );

```

```

wire rx_data_valid;
wire [7:0]    rx_data;
wire tx_data_valid;
wire [7:0]    tx_data;
//Uart_Bus module
Uart_Bus u1
(
.clk                (clk                ), //系统时钟 12MHz
.rst_n              (rst_n              ), //系统复位，低有效
//负责 FPGA 接收 UART 芯片的数据
.uart_rx            (fpga_rx            ), //UART 接收输入
.rx_data_valid      (rx_data_valid), //接收数据有效脉冲
.rx_data_out        (rx_data            ), //接收到的数据      8 位
//负责 FPGA 发送数据给 UART 芯片
.tx_data_valid      (tx_data_valid),
.tx_data_in         (tx_data            ),
.uart_tx            (fpga_tx            )
);

control control_s
(
.clk                (clk                ), // system clock
.rst_n              (rst_n              ), // system reset, active low
.temp_in            (bcd_code[23:12]    ), // ds18b20z data_out
.rx_data_valid      (rx_data_valid),
.rx_data_in         (rx_data            ),
.tx_data_valid      (tx_data_valid),
.tx_data_out        (tx_data            )
);

endmodule

```

(2) center_control.v。

```
// *****
// File name : center_control
// Module name : control
//
// Author : STEP
// Date : 2023/3/31
// Version : V 1.0
// Description : 将 BCD 码转换为 ASCII 码。
//
// Modification history
//-----
// $Log$
//
// *****

module control
(
    input          clk,          // system clock
    input          rst_n,        // system reset, active low
    input [11:0] temp_in,
    input          rx_data_valid,
    input [7:0] rx_data_in,

    output reg tx_data_valid,
    output reg [7:0] tx_data_out,
    output reg [15:0] cycle
);

reg temp_tx_flag;

always @(posedge clk or negedge rst_n) begin
    if(!rst_n)
        temp_tx_flag<=1'b0;
    else
```

```

        temp_tx_flag<=1'b1;
    end

    reg [23:0] cnt_uart;
    always @(posedge clk or negedge rst_n)begin //计时一秒
        if(!rst_n) cnt_uart <= 1'b0;
        else if(cnt_uart >= 24'd11_999_999) cnt_uart <= 1'b0;
        else if (temp_tx_flag) cnt_uart <= cnt_uart + 1'b1;
        else cnt_uart <= cnt_uart;
    end

    always @(posedge clk or negedge rst_n)begin //*****每秒发一个
    温度时间数据，字符串显示*****//
        if(!rst_n) begin
            tx_data_valid <= 1'b0;
            tx_data_out <= 1'b0;
        end else begin
            if(temp_tx_flag)begin //发送温度数据
                if(cnt_uart == 24'd9_099_999 ) begin //温度高位
                    tx_data_valid <= 1'b1;
                    tx_data_out <= {4'd3,temp_in[11:8]};
                end
                else if(cnt_uart == 24'd9_599_999 ) begin //温度低位
                    tx_data_valid <= 1'b1;
                    tx_data_out <= {4'd3,temp_in[7:4]};
                end
                else if(cnt_uart == 24'd10_099_999 ) begin //""
                    tx_data_valid <= 1'b1;
                    tx_data_out <= 8'd46;
                end
                else if(cnt_uart == 24'd10_599_999 ) begin //温度.高位
                    tx_data_valid <= 1'b1;
                    tx_data_out <= {4'd3,temp_in[3:0]};
                end
            end
        end
    end

```

```

        else if(cnt_uart == 24'd11_099_999 ) begin //温度.低位
            tx_data_valid <= 1'b1;
            tx_data_out <= {4'd3,4'b0};
        end
        else if(cnt_uart == 24'd11_599_999 ) begin //"C"
            tx_data_valid <= 1'b1;
            tx_data_out <= 8'd67;
        end
        else if(cnt_uart == 24'd11_999_999 ) begin //换行
            tx_data_valid <= 1'b1;
            tx_data_out <= 8'd10;
        end
        else begin
            tx_data_valid <= 1'b0; tx_data_out <= tx_data_out;
        end
        end else begin
            tx_data_valid <= 1'b0; tx_data_out <= tx_data_out;
        end
    end
end
endmodule

```

(3) uart_bus.v。

```

// *****
// File name :   uart_bus
// Module name :   Uart_Bus
//
// Author       :   STEP
// Date        :   2023/3/31
// Version     :   V 1.0
// Description  :   uart 串口的顶层模块。
//
// Modification history
//-----
// $Log$
//
// *****

```

```

module Uart_Bus #
(
parameter          BPS_PARA = 1250      //12MHz 时钟时参数 1250 对应 9600 的
波特率
)
(
input              clk,                //系统时钟 12MHz
input              rst_n,              //系统复位，低有效

input              uart_rx,            //UART 接收输入
output             rx_data_valid, //接收数据有效脉冲
output [7:0]       rx_data_out, //接收到的数据

input              tx_data_valid, //发送数据有效脉冲
input [7:0]        tx_data_in,     //要发送的数据
output             uart_tx          //UART 发送输出
);

//////////////////////////////////UART 接收功能模块例化
//////////////////////////////////

wire              bps_en_rx,bps_clk_rx;

//UART 接收波特率时钟控制模块 例化
Baud #
(
.BPS_PARA          (BPS_PARA          )
)
Baud_rx
(
.clk                (clk                ), //系统时钟 12MHz
.rst_n              (rst_n              ), //系统复位，低有效
.bps_en             (bps_en_rx          ), //接收时钟使能
.bps_clk            (bps_clk_rx         ) //接收时钟输出
);

//UART 接收数据模块 例化

```

```

Uart_Rx Uart_Rx_uut
(
.clk                (clk                ), //系统时钟 12MHz
.rst_n              (rst_n              ), //系统复位，低有效

.bps_en             (bps_en_rx          ), //接收时钟使能
.bps_clk            (bps_clk_rx         ), //接收时钟输入

.uart_rx            (uart_rx            ), //UART 接收输入
.rx_data_valid      (rx_data_valid),    //接收数据有效脉冲
.rx_data_out        (rx_data_out )      //接收到的数据
);

//////////////////////////UART 发送功能模块例化
//////////////////////////
wire                bps_en_tx,bps_clk_tx;

//UART 发送波特率时钟控制模块 例化
Baud #
(
.BPS_PARA           (BPS_PARA          )
)
Baud_tx
(
.clk                (clk                ), //系统时钟 12MHz
.rst_n              (rst_n              ), //系统复位，低有效
.bps_en             (bps_en_tx         ), //发送时钟使能
.bps_clk            (bps_clk_tx        ) //发送时钟输出
);

//UART 发送数据模块 例化
Uart_Tx Uart_Tx_uut
(
.clk                (clk                ), //系统时钟 12MHz
.rst_n              (rst_n              ), //系统复位，低有效

.bps_en             (bps_en_tx         ), //发送时钟使能

```



```
.bps_clk          (bps_clk_tx      ), //发送时钟输入

.tx_data_valid    (tx_data_valid), //发送数据有效脉冲
.tx_data_in       (tx_data_in      ), //要发送的数据
.uart_tx          (uart_tx        ) //UART 发送输出
);

endmodule
```

(4) uart_r。

```
// *****
// File name : uart_r
// Module name : Uart_Rx
//
// Author      : STEP
// Date        : 2023/3/31
// Version     : V 1.0
// Description  : uart 串口的接收模块。
//
// Modification history
//-----
// $Log$
//
// *****

module Uart_Rx
(
input          clk,          //系统时钟 12MHz
input          rst_n,        //系统复位，低有效

output reg     bps_en,       //接收时钟使能
input          bps_clk,      //接收时钟输入

input          uart_rx,      //UART 接收输入
output reg     rx_data_valid, //接收数据有效脉冲
output reg     [7:0] rx_data_out //接收到的数据
);
```

```

reg  uart_rx0,uart_rx1,uart_rx2;
//多级延时锁存去除亚稳态
always @ (posedge clk) begin
    uart_rx0 <= uart_rx;
    uart_rx1 <= uart_rx0;
    uart_rx2 <= uart_rx1;
end

//检测 UART 接收输入信号的下降沿
wireneg_uart_rx = uart_rx2 & ~uart_rx1;

reg          [3:0]    num;
//接收时钟使能信号的控制
always @ (posedge clk or negedge rst_n) begin
    if(!rst_n)
        bps_en <= 1'b0;
    else if(neg_uart_rx && (!bps_en))    //当空闲状态（bps_en 为低电平）时检测到 UART
接收信号下降沿，进入工作状态（bps_en 为高电平），控制时钟模块产生接收时钟
        bps_en <= 1'b1;
    else if(num==4'd9)                  //当完成一次 UART 接收操作后，退出工作状
态，恢复空闲状态
        bps_en <= 1'b0;
end

reg          [7:0]    rx_data;
//当处于工作状态中时，按照接收时钟的节拍获取数据
always @ (posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        num <= 4'd0;
        rx_data <= 8'd0;
    end else if(bps_en) begin
        if(bps_clk) begin
            num <= num + 1'b1;
            if(num<=4'd8) rx_data[num-1] <= uart_rx1; //先接受低位再接收高位，8 位有
效数据
        end else if(num == 4'd9) begin                //完成一次 UART 接收操作后，将

```

获取的数据输出

```

        num <= 4'd0;
    end
end else begin
    num <= 4'd0;
end
end
end

//将接收的数据输出，同时控制输出有效信号产生脉冲
always @ (posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        rx_data_out <= 8'd0;
        rx_data_valid <= 1'b0;
    end else if(num == 4'd9) begin
        rx_data_out <= rx_data;
        rx_data_valid <= 1'b1;
    end else begin
        rx_data_out <= rx_data_out;
        rx_data_valid <= 1'b0;
    end
end
end

endmodule

```

(5) uart_s.v。

```

// *****
// File name   :   uart_s
// Module name  :   Uart_Tx
//
// Author      :   STEP
// Date        :   2023/3/31
// Version     :   V 1.0
// Description  :   uart 串口的发送模块。
//
// Modification history
//-----
// $Log$
//

```

```
// *****

module Uart_Tx
(
input          clk,          //系统时钟 12MHz
input          rst_n,        //系统复位，低有效

output reg     bps_en,       //发送时钟使能
input          bps_clk,      //发送时钟输入

input          tx_data_valid, //发送数据有效脉冲
input          [7:0] tx_data_in, //要发送的数据
output reg     uart_tx       //UART 发送输出
);

reg          [3:0]  num;
reg          [9:0]  tx_data_r; //融合了起始位和停止位的数据
//驱动发送数据操作
always @ (posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        bps_en <= 1'b0;
        tx_data_r <= 10'd0;
    end else if(tx_data_valid && (!bps_en))begin
        bps_en <= 1'b1; //当检测到接收时钟使能信号的下降沿，表明接收完成，需要发送数据，使能发送时钟使能信号
        tx_data_r <= {1'b1,tx_data_in,1'b0};
    end else if(num==4'd10) begin
        bps_en <= 1'b0; //一次 UART 发送需要 10 个时钟信号，然后结束
    end
end

//当处于工作状态中时，按照发送时钟的节拍发送数据
always @ (posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        num <= 1'b0;
        uart_tx <= 1'b1;
    end else if(bps_en) begin
```

```

        if(bps_clk) begin
            num <= num + 1'b1;
            uart_tx <= tx_data_r[num];
        end else if(num>=4'd10) begin
            num <= 4'd0;
        end
    end
end
endmodule

```

(6) Baud.v。

```

// *****
// File name : Baud
// Module name : Baud
//
// Author : STEP
// Date : 2023/3/31
// Version : V 1.0
// Description : 产生波特率节拍, 给 uart 发送和接收模块产生相应的波特率节拍信号。
//
// Modification history
//-----
// $Log$
//
// *****

module Baud #
(
parameter          BPS_PARA = 1250 //12MHz 时钟时参数 1250 对应 9600 的波特
率
)
(
input              clk,          //系统时钟
input              rst_n,        //系统复位, 低有效
input              bps_en,       //接收或发送时钟使能
output reg         bps_clk       //接收或发送时钟输出
);

```

```

reg          [12:0]  cnt;
//计数器计数满足波特率时钟要求
always @ (posedge clk or negedge rst_n) begin
    if(!rst_n)
        cnt <= 1'b0;
    else if((cnt >= BPS_PARA-1)||(!bps_en)) //当时钟信号不使能（bps_en 为低电平）时，计数器清零并停止计数
        cnt <= 1'b0;                      //当时钟信号使能时，计数器对系统时钟计数，周期为 BPS_PARA 个系统时钟周期
    else
        cnt <= cnt + 1'b1;
end

//产生相应波特率的时钟节拍，接收模块将以此节拍进行 UART 数据接收
always @ (posedge clk or negedge rst_n) begin
    if(!rst_n)
        bps_clk <= 1'b0;
    else if(cnt == (BPS_PARA>>1)) //右移一位等于除以 2，终值 BPS_PARA 为数据更替点，中值数据稳定，做采样点
        bps_clk <= 1'b1;
    else
        bps_clk <= 1'b0;
end

endmodule

```

(7) temperture.v。

```

// *****
// File name   : temperture
// Module name  : DS18B20Z
//
// Author      : STEP
// Date        : 2023/3/31
// Version     : V 1.0
// Description  : 温度采集模块，负责 FPGA 与 DS18B20 的通讯。
//
// Modification history

```

```
//-----
// $Log$
//
// *****

module DS18B20Z
(
    input          clk,          // system clock
    input          rst_n,        // system reset, active low
    inout          one_wire,     // ds18b20z one-wire-bus
    output reg [15:0] data_out   // ds18b20z data_out
);

    localparam IDLE    = 3'd0;
    localparam MAIN    = 3'd1;
    localparam INIT    = 3'd2;
    localparam WRITE   = 3'd3;
    localparam READ    = 3'd4;
    localparam DELAY   = 3'd5;

    //generate clk_1mhz clock
    reg          clk_1mhz;
    reg [2:0]    cnt_1mhz;
    always@(posedge clk or negedge rst_n) begin
        if(!rst_n) begin
            cnt_1mhz <= 3'd0;
            clk_1mhz <= 1'b0;
        end else if(cnt_1mhz >= 3'd5) begin
            cnt_1mhz <= 3'd0;
            clk_1mhz <= ~clk_1mhz;
        end else begin
            cnt_1mhz <= cnt_1mhz + 1'b1;
        end
    end

    reg          one_wire_buffer;
    reg [3:0]    cnt_main;
```

```

reg    [7:0]    data_wr;
reg    [7:0]    data_wr_buffer;
reg    [2:0]    cnt_init;
reg    [19:0]   cnt_delay;
reg    [19:0]   num_delay;
reg    [5:0]    cnt_write;
reg    [5:0]    cnt_read;
reg    [15:0]   temperature;
reg    [7:0]    temperature_buffer;
reg    [2:0]    state = IDLE;
reg    [2:0]    state_back = IDLE;

always@(posedge clk_1mhz or negedge rst_n) begin
    if(!rst_n) begin
        state <= IDLE;
        state_back <= IDLE;
        cnt_main <= 4'd0;
        cnt_init <= 3'd0;
        cnt_write <= 6'd0;
        cnt_read <= 6'd0;
        cnt_delay <= 20'd0;
        one_wire_buffer <= 1'bz;
        temperature <= 16'h0;
    end else begin
        case(state)
            IDLE:begin
                state <= MAIN;
                state_back <= MAIN;
                cnt_main <= 4'd0;
                cnt_init <= 3'd0;
                cnt_write <= 6'd0;
                cnt_read <= 6'd0;
                cnt_delay <= 20'd0;
                one_wire_buffer <= 1'bz;
            end
            MAIN:begin
                if(cnt_main >= 4'd11) cnt_main <= 1'b0;
                else cnt_main <= cnt_main + 1'b1;
            end
        endcase
    end
end

```



```

        case(cnt_main)
            4'd0: begin state <= INIT; end
            4'd1: begin data_wr <= 8'hcc;state <= WRITE; end
            4'd2: begin data_wr <= 8'h44;state <= WRITE; end
            4'd3: begin num_delay <= 20'd750000;state <=
DELAY;state_back <= MAIN; end

            4'd4: begin state <= INIT; end
            4'd5: begin data_wr <= 8'hcc;state <= WRITE; end
            4'd6: begin data_wr <= 8'hbe;state <= WRITE; end

            4'd7: begin state <= READ; end
            4'd8: begin temperature[7:0] <= temperature_buffer; end

            4'd9: begin state <= READ; end
            4'd10: begin temperature[15:8] <= temperature_buffer; end

            4'd11: begin state <= IDLE;data_out <= temperature; end
            default: state <= IDLE;
        endcase
    end
INIT:begin
    if(cnt_init >= 3'd6) cnt_init <= 1'b0;
    else cnt_init <= cnt_init + 1'b1;
    case(cnt_init)
        3'd0: begin one_wire_buffer <= 1'b0; end
        3'd1: begin num_delay <= 20'd500;state <=
DELAY;state_back <= INIT; end
        3'd2: begin one_wire_buffer <= 1'bz; end
        3'd3: begin num_delay <= 20'd100;state <=
DELAY;state_back <= INIT; end
        3'd4: begin if(one_wire) state <= IDLE; else state <= INIT;
    end
        3'd5: begin num_delay <= 20'd400;state <=
DELAY;state_back <= INIT; end
        3'd6: begin state <= MAIN; end
        default: state <= IDLE;
    endcase
end

```

```

        endcase
    end
    WRITE:begin
        if(cnt_write >= 6'd50) cnt_write <= 1'b0;
        else cnt_write <= cnt_write + 1'b1;
        case(cnt_write)
            //lock data_wr
            6'd0: begin data_wr_buffer <= data_wr; end
            //write bit 0
            6'd1: begin one_wire_buffer <= 1'b0; end
            6'd2: begin num_delay <= 20'd2;state <= DELAY;state_back
<= WRITE; end

            6'd3: begin one_wire_buffer <= data_wr_buffer[0]; end
            6'd4: begin num_delay <= 20'd80;state <= DELAY;state_back
<= WRITE; end

            6'd5: begin one_wire_buffer <= 1'bz; end
            6'd6: begin num_delay <= 20'd2;state <= DELAY;state_back
<= WRITE; end

            //write bit 1
            6'd7: begin one_wire_buffer <= 1'b0; end
            6'd8: begin num_delay <= 20'd2;state <= DELAY;state_back
<= WRITE; end

            6'd9: begin one_wire_buffer <= data_wr_buffer[1]; end
            6'd10: begin num_delay <= 20'd80;state <=
DELAY;state_back <= WRITE; end

            6'd11: begin one_wire_buffer <= 1'bz; end
            6'd12: begin num_delay <= 20'd2;state <= DELAY;state_back
<= WRITE; end

            //write bit 2
            6'd13: begin one_wire_buffer <= 1'b0; end
            6'd14: begin num_delay <= 20'd2;state <= DELAY;state_back
<= WRITE; end

            6'd15: begin one_wire_buffer <= data_wr_buffer[2]; end
            6'd16: begin num_delay <= 20'd80;state <=
DELAY;state_back <= WRITE; end

            6'd17: begin one_wire_buffer <= 1'bz; end
            6'd18: begin num_delay <= 20'd2;state <= DELAY;state_back

```

```

<= WRITE; end

                                //write bit 3
6'd19: begin one_wire_buffer <= 1'b0; end
6'd20: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

6'd21: begin one_wire_buffer <= data_wr_buffer[3]; end
6'd22: begin num_delay <= 20'd80;state <=
DELAY;state_back <= WRITE; end
6'd23: begin one_wire_buffer <= 1'bz; end
6'd24: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

                                //write bit 4
6'd25: begin one_wire_buffer <= 1'b0; end
6'd26: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

6'd27: begin one_wire_buffer <= data_wr_buffer[4]; end
6'd28: begin num_delay <= 20'd80;state <=
DELAY;state_back <= WRITE; end
6'd29: begin one_wire_buffer <= 1'bz; end
6'd30: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

                                //write bit 5
6'd31: begin one_wire_buffer <= 1'b0; end
6'd32: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

6'd33: begin one_wire_buffer <= data_wr_buffer[5]; end
6'd34: begin num_delay <= 20'd80;state <=
DELAY;state_back <= WRITE; end
6'd35: begin one_wire_buffer <= 1'bz; end
6'd36: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

                                //write bit 6
6'd37: begin one_wire_buffer <= 1'b0; end
6'd38: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

6'd39: begin one_wire_buffer <= data_wr_buffer[6]; end
6'd40: begin num_delay <= 20'd80;state <=

```

```

DELAY;state_back <= WRITE; end

        6'd41: begin one_wire_buffer <= 1'bz; end
        6'd42: begin num_delay <= 20'd2;state <= DELAY;state_back
<= WRITE; end

                //write bit 7
        6'd43: begin one_wire_buffer <= 1'b0; end
        6'd44: begin num_delay <= 20'd2;state <= DELAY;state_back
<= WRITE; end

        6'd45: begin one_wire_buffer <= data_wr_buffer[7]; end
        6'd46: begin num_delay <= 20'd80;state <=
DELAY;state_back <= WRITE; end

        6'd47: begin one_wire_buffer <= 1'bz; end
        6'd48: begin num_delay <= 20'd2;state <= DELAY;state_back
<= WRITE; end

                //back to main
        6'd49: begin num_delay <= 20'd80;state <=
DELAY;state_back <= WRITE; end

        6'd50: begin state <= MAIN; end
        default: state <= IDLE;
    endcase
end
READ:begin
    if(cnt_read >= 6'd48) cnt_read <= 1'b0;
    else cnt_read <= cnt_read + 1'b1;
    case(cnt_read)
        //read bit 0
        6'd0: begin one_wire_buffer <= 1'b0; end
        6'd1: begin num_delay <= 20'd2;state <= DELAY;state_back
<= READ; end

        6'd2: begin one_wire_buffer <= 1'bz; end
        6'd3: begin num_delay <= 20'd10;state <= DELAY;state_back
<= READ; end

        6'd4: begin temperature_buffer[0] <= one_wire; end
        6'd5: begin num_delay <= 20'd55;state <= DELAY;state_back
<= READ; end

        //read bit 1
        6'd6: begin one_wire_buffer <= 1'b0; end

```

```

        6'd7: begin num_delay <= 20'd2;state <= DELAY;state_back
<= READ; end

        6'd8: begin one_wire_buffer <= 1'bz; end
        6'd9: begin num_delay <= 20'd10;state <= DELAY;state_back
<= READ; end

        6'd10: begin temperature_buffer[1] <= one_wire; end
        6'd11: begin num_delay <= 20'd55;state <=
DELAY;state_back <= READ; end

        //read bit 2
        6'd12: begin one_wire_buffer <= 1'b0; end
        6'd13: begin num_delay <= 20'd2;state <= DELAY;state_back
<= READ; end

        6'd14: begin one_wire_buffer <= 1'bz; end
        6'd15: begin num_delay <= 20'd10;state <=
DELAY;state_back <= READ; end

        6'd16: begin temperature_buffer[2] <= one_wire; end
        6'd17: begin num_delay <= 20'd55;state <=
DELAY;state_back <= READ; end

        //read bit 3
        6'd18: begin one_wire_buffer <= 1'b0; end
        6'd19: begin num_delay <= 20'd2;state <= DELAY;state_back
<= READ; end

        6'd20: begin one_wire_buffer <= 1'bz; end
        6'd21: begin num_delay <= 20'd10;state <=
DELAY;state_back <= READ; end

        6'd22: begin temperature_buffer[3] <= one_wire; end
        6'd23: begin num_delay <= 20'd55;state <=
DELAY;state_back <= READ; end

        //read bit 4
        6'd24: begin one_wire_buffer <= 1'b0; end
        6'd25: begin num_delay <= 20'd2;state <= DELAY;state_back
<= READ; end

        6'd26: begin one_wire_buffer <= 1'bz; end
        6'd27: begin num_delay <= 20'd10;state <=
DELAY;state_back <= READ; end

        6'd28: begin temperature_buffer[4] <= one_wire; end
        6'd29: begin num_delay <= 20'd55;state <=

```

```

DELAY;state_back <= READ; end

                                //read bit 5
                                6'd30: begin one_wire_buffer <= 1'b0; end
                                6'd31: begin num_delay <= 20'd2;state <= DELAY;state_back
<= READ; end

                                6'd32: begin one_wire_buffer <= 1'bz; end
                                6'd33: begin num_delay <= 20'd10;state <=
DELAY;state_back <= READ; end

                                6'd34: begin temperature_buffer[5] <= one_wire; end
                                6'd35: begin num_delay <= 20'd55;state <=
DELAY;state_back <= READ; end

                                //read bit 6
                                6'd36: begin one_wire_buffer <= 1'b0; end
                                6'd37: begin num_delay <= 20'd2;state <= DELAY;state_back
<= READ; end

                                6'd38: begin one_wire_buffer <= 1'bz; end
                                6'd39: begin num_delay <= 20'd10;state <=
DELAY;state_back <= READ; end

                                6'd40: begin temperature_buffer[6] <= one_wire; end
                                6'd41: begin num_delay <= 20'd55;state <=
DELAY;state_back <= READ; end

                                //read bit 7
                                6'd42: begin one_wire_buffer <= 1'b0; end
                                6'd43: begin num_delay <= 20'd2;state <= DELAY;state_back
<= READ; end

                                6'd44: begin one_wire_buffer <= 1'bz; end
                                6'd45: begin num_delay <= 20'd10;state <=
DELAY;state_back <= READ; end

                                6'd46: begin temperature_buffer[7] <= one_wire; end
                                6'd47: begin num_delay <= 20'd55;state <=
DELAY;state_back <= READ; end

                                //back to main
                                6'd48: begin state <= MAIN; end
                                default: state <= IDLE;
                                endcase
                                end
                                DELAY:begin

```

```

        if(cnt_delay >= num_delay) begin
            cnt_delay <= 1'b0;
            state <= state_back;
        end else cnt_delay <= cnt_delay + 1'b1;
    end
endcase
end
end

assign one_wire = one_wire_buffer;

endmodule

```

(8) bin_to_bcd.v。

```

// *****
// File name : bin_to_bcd
// Module name : bcd_8421
//
// Author : STEP
// Date : 2023/3/31
// Version : V 1.0
// Description : 二进制转换为 BCD 码。
//
// Modification history
//-----
// $Log$
//
// *****

module bin_to_bcd #
(
    parameter B_SIZE = 21
)
(
    input rst_n, // system reset, active low
    input [B_SIZE-1:0] bin_code, // binary code
    output reg [B_SIZE+3:0] bcd_code // bcd code
);

```

```

reg      [2*B_SIZE+3:0]      shift_reg;
always@(bin_code or rst_n)begin
    shift_reg= {25'h0,bin_code};
    if(!rst_n) bcd_code <= 0;
    else begin
        repeat(B_SIZE)//repeat B_SIZE times
            begin
                if (shift_reg[24:21] >= 5) shift_reg[24:21] = shift_reg[24:21] + 2'b11;
                if (shift_reg[28:25] >= 5) shift_reg[28:25] = shift_reg[28:25] + 2'b11;
                if (shift_reg[32:29] >= 5) shift_reg[32:29] = shift_reg[32:29] + 2'b11;
                if (shift_reg[36:33] >= 5) shift_reg[36:33] = shift_reg[36:33] + 2'b11;
                if (shift_reg[40:37] >= 5) shift_reg[40:37] = shift_reg[40:37] + 2'b11;
                if (shift_reg[44:41] >= 5) shift_reg[44:41] = shift_reg[44:41] + 2'b11;
                shift_reg = shift_reg << 1;
            end
        bcd_code<=shift_reg[45:21];
    end
end
endmodule

```

3.3.4 实验步骤

- 1.打开 Ridian, 建立工程。
- 2.新建 Verilog HDL 设计文件, 并键入设计代码, 包括添加需要调用的模块文件。
- 3.构建并输出编程文件, 烧写至 FPGA 的 Flash 之中。

3.3.5 实验现象

实验结果如下:



将程序烧录之后，每秒 FPGA 都会向上位机发送当前温度，一开始温度为 27.9 摄氏度，手指放上去之后，就变为 31.5 摄氏度。

第四章 DS18B20 温度传感器实验

实验简介

本教程以实验为主，结合相应的实验案例，以详细的讲解和实验步骤让大家尽快熟悉 FPGA 开发的基本流程、Ridiant 软件平台的使用方法以及本节实验相关模块的工作原理。

- 熟悉和掌握 FPGA 开发的基本流程
- 掌握 Ridiant 软件平台的基本使用方法
- 了解使用 DS18B220 的原理
- 掌握使用 Verilog HDL 语言基于 FPGA 实现 DS18B20 测量温度的原理及实现方法

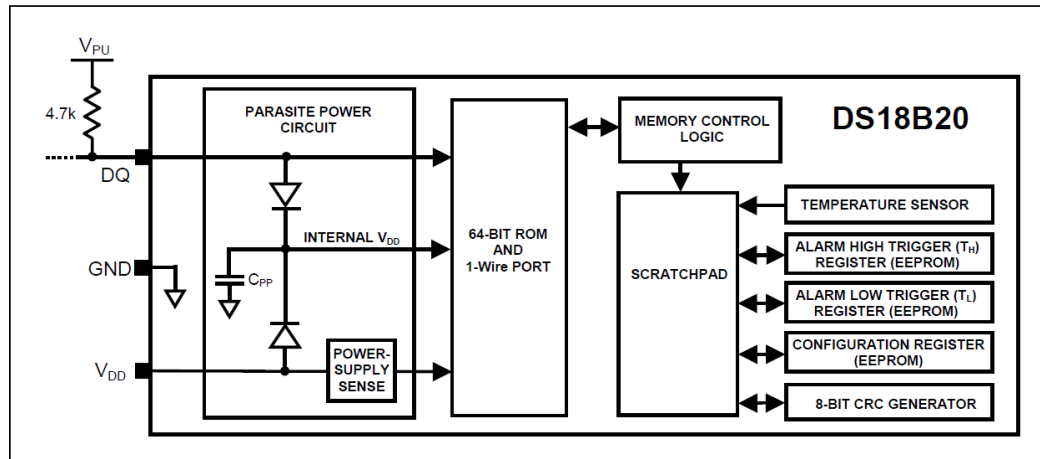
背景知识

温度传感器（temperature transducer）是指能感受温度并转换成可用输出信号的传感器，是各种传感器中最常用的一种。早期的温度传感器是采集热敏电阻等元器件的电压，再根据给定公式计算出环境温度。随着现代仪器的发展，美国 DALLAS 半导体公司推出的数字化温度传感器 DS18B20 采用单总线协议，即与 FPGA 接口仅需占用一个 I/O 端口，无须任何外部元件，直接将环境温度转化成数字信号，以数字码方式串行输出，从而大大简化了传感器与 FPGA 的接口设计。

DS18B20 测量温度的范围是 -55°C ~ $+125^{\circ}\text{C}$ ，精度为 $\pm 0.5^{\circ}\text{C}$ 。外部环境的温度会转化为 9~12 位数据形式以单总线的方式传输。它的工作电压为 3~5.5V。

DS18B20 采用 1-Wire 总线协议，该总线协议仅需要一个控制信号进行通信。该控制信号线需要一个唤醒的上拉电阻以防止连接在该总线上的口是 3 态或者高阻态（DQ 信号线是在 DS18B20 上）。在该总线系统中，微控制器（主设备）通过每个设备的 64 为序列号来识别该总线上的设备。因为每个设备都有一个独一无二的序列号，挂在一个总线上的设备理论上是可以无限个的。

DS18B20 的另外一个特性就是可以无需外部电源供电。当数据线 DQ 为高的时候由其作为设备供电。总线拉高的时候为内部电容（Spp）充电，当总线拉低是由该电容向设备供电。这种由 1-Wire 总线为设备供电的方式称为“寄生电源”。此外，DS18B20 也可以由外部电源通过 VDD 供电。DS18B20 内部框图如下所示：



DS18B20 的核心功能是直接温度—数字测量，上电后工作在低功耗闲置状态下，其温度转换可由用户自定义为 9、10、11、12 位精度分别为 0.5°C 、 0.25°C 、 0.125°C 、 0.0625°C 分辨率，上电默认为 12 位转换精度。温度数据以一个 16 位标志扩展二进制补码数的形式存储在温度寄存器中，符号标志位(S)温度的正负极性：正数则 $S=0$ ，负数则 $S=1$ 。如果 DS18B20 被定义为 12 位的转换精度，温度寄存器中的所有位都将包含有效数据。若为 11 位转换精度，则 BIT 0 为未定义的。若为 10 位转换精度，则 BIT 1 和 BIT 0 为未定义的。若为 9 位转换精度，则 BIT 2、BIT 1 和 BIT 0 为未定义的。下表为在 12 位转换精度下温度输出数据与相对应温度之间的关系表。

	BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
LS BYTE	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}
	BIT 15	BIT 14	BIT 13	BIT 12	BIT 11	BIT 10	BIT 9	BIT 8
MS BYTE	S	S	S	S	S	2^6	2^5	2^4

S = SIGN

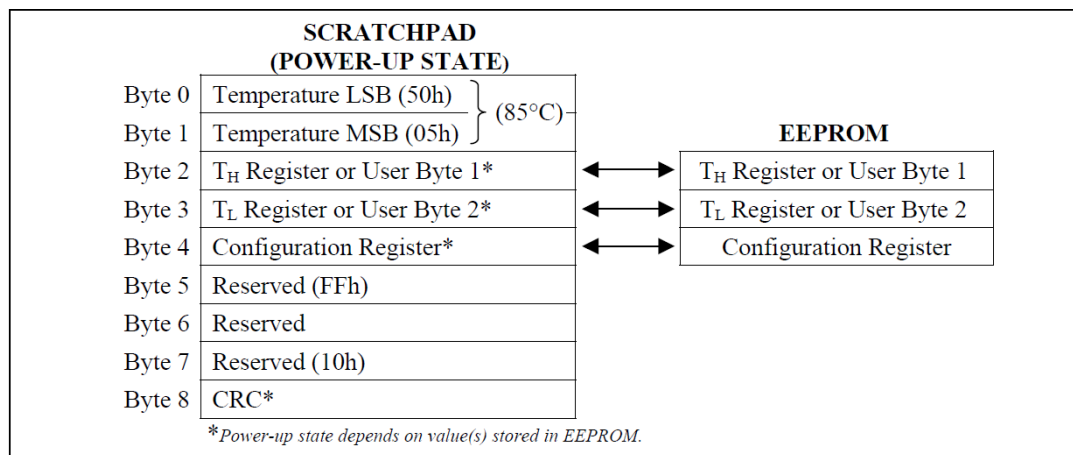
下表为一些在 12 位输出时的温度和对应的输出数据的例子。

TEMPERATURE ($^{\circ}\text{C}$)	DIGITAL OUTPUT (BINARY)	DIGITAL OUTPUT (HEX)
+125	0000 0111 1101 0000	07D0h
+85*	0000 0101 0101 0000	0550h
+25.0625	0000 0001 1001 0001	0191h
+10.125	0000 0000 1010 0010	00A2h
+0.5	0000 0000 0000 1000	0008h
0	0000 0000 0000 0000	0000h
-0.5	1111 1111 1111 1000	FFF8h
-10.125	1111 1111 0101 1110	FF5Eh
-25.0625	1111 1110 0110 1111	FE6Fh
-55	1111 1100 1001 0000	FC90h

*The power-on reset value of the temperature register is $+85^{\circ}\text{C}$.

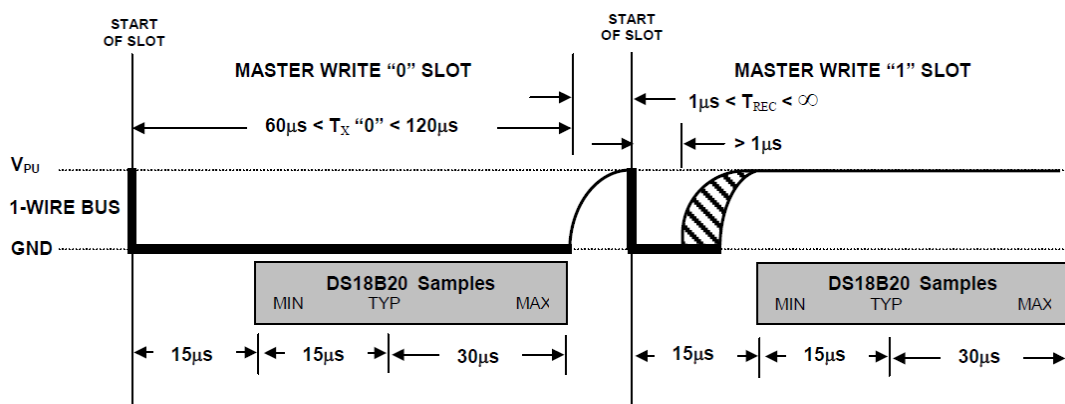
DS18B20 存储器包含了 SRAM 暂存寄存器、过温和低温（TH 和 TL）温度报警寄存器、配置寄存器的非易失性 EEPROM。当温度报警功能没有用到的时候，过温和低温（TH 和 TL）温度报警寄存器可以当做通用功能的存储单元。

SRAM 暂存寄存器中的 Byte 0 和 Byte 1 分别作为温度寄存器的低字节和高字节，同时这两个字节是只读的；Byte 2 和 Byte 3 作为过温和低温（TH 和 TL）温度报警寄存器；Byte 4 保存着配置寄存器的数据；Byte 5、6、7 作为内部使用的字节而保留使用，不可被写入；Byte 8 存储着该暂存寄存器中 Byte 0 至 Byte 7 的循环冗余校验（CRC）值，并且只读不可写入，存储器组织结构如下图所示：



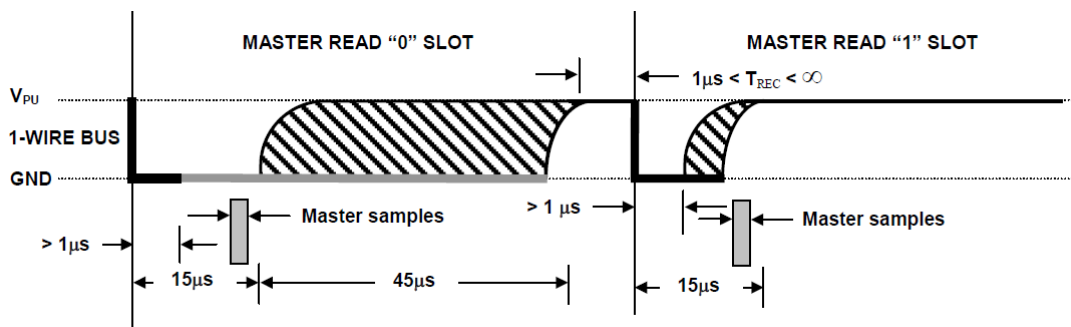
在了解了一些 DS18B20 的基础知识之后，那么我们该如何使用单总线方式发送和接受数据呢？

写时隙有两种：写 0，写 1。下图为写时隙的时序图，所有的写时隙至少需要 60us，并且两次写时隙之间至少需要 1us 的恢复时间。两种写时隙均在主机拉低数据总线后来时。写 0：数据总线拉低保持至少 60us 即可；写 1：在主机拉低数据总线后，必须在 15us 之内拉高数据总线。因为单总线器件会在写时隙开始后的 15~60us 采样，如在此器件数据总线为高电平，则写 1；如为低电平，则写 0。

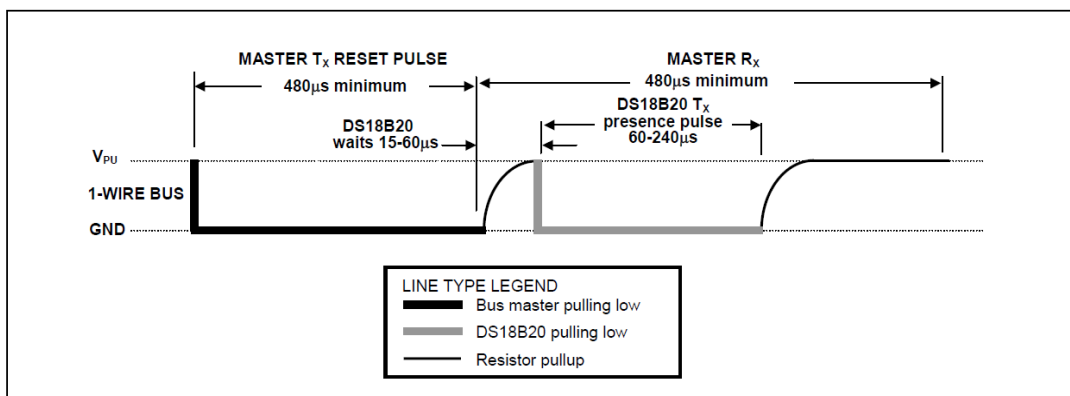


对于读时隙，单总线器件仅在主机发出读数据命令后，才向主机传输数据。主机发出读数据命令后，必须马上产生读时隙，以便从机能够传输数据。读时隙也有两种：读 0，读 1。下图为读时隙的时序图，所有的读时隙至少需要 60us，并且两次读时隙之间至少需要 1us

的恢复时间。每个读时隙由主机发起，至少拉低 1us 的数据总线。由于从机发送的数据有效时间是 15us，所以主机必须在 15us 时间内采样数据总线的状态。读 0：在 15us 采样数据总线状态为低电平；读 1：在 15us 内采样数据总线状态为高电平。



下图为 DS18B20 的初始化时序，主机拉低数据总线并保持 480us~960us，此时为主机发送复位脉冲。然后主机释放总线，保持 15~60us。然后 DS18B20 会拉低数据总线并保持 60~240us，此时是 DS18B20 向主机发送的应答信号。从主机释放总线到 DS18B20 发送应答信号至少要 480us。等到应答完成后，初始化结束。



对于简单的使用 FPGA 从 DS18B20 读取温度，其实要经历以下过程：为：初始化→发跳过 ROM 命令（CCH）→发开始转换命令（44H）→延时→初始化→发送跳过 ROM 命令（CCH）→发读存储器命令（BEH）→连续读出两个字节数据(即温度)→结束或开始下一循环。

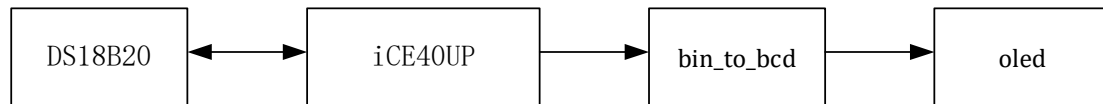
实验环境

- 硬件环境：STEP FPGA 实验平台
- 软件环境：Ridiant 开发环境

4.1 实验四 DS18B20 温度传感器实验

4.1.1 实验任务

通过 FPGA 向 DS18B20 读取温度，并在 OLED 显示温度。



4.1.2 实验原理

根据上述电路图，可以分割任务。首先 FPGA 向 DS18B20 发送采集温度的指令，其次 DS18B20 采集温度并把数据发送给 FPGA，然后 FPGA 将温度传感器返回的二进制数据转换为 BCD 码，在通过 SPI 发送给 OLED 显示出来。

整个程序有四个模块：

Thermometer：顶层模块。

DS18B20Z：温度采集模块，负责 FPGA 与 DS18B20 的通讯。

bin_to_bcd：二进制转换为 BCD 码。

oled：负责将采集的温度在 OLED 显示出来。

4.1.3 Verilog HDL 建模描述

（1）设计文件 Thermometer.v。

```

// *****
// File name : Thermometer
// Module name : Thermometer
//
// Author : STEP
// Date : 2023/3/31
// Version : V 1.0
// Description : 顶层模块。
//
// Modification history
//-----
// $Log$
//
  
```

```
// *****

module Thermometer
(
    input                rst_n,          // system reset, active low
    inout                one_wire,       // ds18b20z one-wire-bus
    output               oled_rst, //OLED 复位信号
    output               oled_dcn, //OLED 数据指令控制
    output               oled_clk, //OLED 时钟信号
    output               oled_dat //OLED 数据信号
);
    wire                clk;             // system clock
    wire[15:0]          data_out;

    wire [3:0] unit   ;                  //温度数据的个位
    wire [3:0] ten    ;                  //温度数据的十位
    wire [3:0] hun     ;                  //温度数据的百位
    wire [3:0] tho     ;                  //温度数据的千位
    wire [3:0] t_tho   ;                  //温度数据的万位
    wire [3:0] h_hun   ;                  //温度数据的十万位


    //内部 12M 时钟
    HSOSC #(
        .CLKHF_DIV ("0b10")
    ) OSCInst0 (
        .CLKHFEN (1'b1   ),
        .CLKHFPU (1'b1   ),
        .CLKHF    (clk)
    );
    //Drive DS18B20Z to get temperature code
    DS18B20Z DS18B20Z_uut
    (
        .clk                (clk          ), // system clock
        .rst_n              (rst_n        ), // system reset, active low
        .one_wire            (one_wire     ), // ds18b20z one-wire-bus

```



```

.data_out      (data_out      )    // ds18b20z data_out
);

// judge sign of temperature
wire temperature_flag = data_out[15:11]? 1'b0:1'b1;
// complement if negative
wire [10:0] temperature_code = temperature_flag? data_out[10:0]:(~data_out[10:0])+1'b1;
// translate temperature_code to real temperature
wire [20:0] bin_code = temperature_code * 16'd625;

//Translate binary code to bcd code
bcd_8421 u_bcd_8421
(
  .sys_clk    (clk),
  .sys_rst_n  (rst_n      ),    // system reset, active low
  .data      (bin_code    ),    // binary code
  .unit      (unit ),
  .ten       (ten  ),
  .hun       (hun  ),
  .tho       (tho  ),
  .t_tho     (t_tho),
  .h_hun     (h_hun)

);

//Display with Segment tube
OLED12864 u_OLED12864
(
  .clk        (clk      ),
  .rst_n      (rst_n    ),
  .unit       (unit ),
  .ten        (ten  ),
  .hun        (hun  ),
  .tho        (tho  ),
  .t_tho      (t_tho),
  .h_hun      (h_hun),
  .oled_csn   (),

```

```
.oled_rst      (oled_rst ),
.oled_dcn      (oled_dcn ),
.oled_clk      (oled_clk ),
.oled_dat      (oled_dat )
);
endmodule
```

(2) 设计文件 DS18B20Z.v

```
// *****
// File name : DS18B20Z
// Module name : DS18B20Z
//
// Author      : STEP
// Date        : 2023/3/31
// Version     : V 1.0
// Description  : 温度采集模块，负责 FPGA 与 DS18B20 的通讯。
//
// Modification history
//-----
// $Log$
//
// *****

module DS18B20Z
(
    input          clk,          // system clock
    input          rst_n,        // system reset, active low
    inout          one_wire,     // ds18b20z one-wire-bus
    output reg [15:0] data_out   // ds18b20z data_out
);

    localparam IDLE = 3'd0;
    localparam MAIN = 3'd1;
    localparam INIT= 3'd2;
```

```

localparam  WRITE  = 3'd3;
localparam  READ   = 3'd4;
localparam  DELAY  = 3'd5;

//generate clk_1mhz clock
reg          clk_1mhz;
reg  [2:0]    cnt_1mhz;
always@(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        cnt_1mhz <= 3'd0;
        clk_1mhz <= 1'b0;
    end else if(cnt_1mhz >= 3'd5) begin
        cnt_1mhz <= 3'd0;
        clk_1mhz <= ~clk_1mhz;
    end else begin
        cnt_1mhz <= cnt_1mhz + 1'b1;
    end
end

reg          one_wire_buffer;
reg  [3:0]    cnt_main;
reg  [7:0]    data_wr;
reg  [7:0]    data_wr_buffer;
reg  [2:0]    cnt_init;
reg  [19:0]   cnt_delay;
reg  [19:0]   num_delay;
reg  [5:0]    cnt_write;
reg  [5:0]    cnt_read;
reg  [15:0]   temperature;
reg  [7:0]    temperature_buffer;
reg  [2:0]    state = IDLE;
reg  [2:0]    state_back = IDLE;
always@(posedge clk_1mhz or negedge rst_n) begin
    if(!rst_n) begin
        state <= IDLE;
        state_back <= IDLE;
        cnt_main <= 4'd0;
    end
end

```

```

cnt_init <= 3'd0;
cnt_write <= 6'd0;
cnt_read <= 6'd0;
cnt_delay <= 20'd0;
one_wire_buffer <= 1'bz;
temperature <= 16'h0;
end else begin
  case(state)
    IDLE:begin
      state <= MAIN;
      state_back <= MAIN;
      cnt_main <= 4'd0;
      cnt_init <= 3'd0;
      cnt_write <= 6'd0;
      cnt_read <= 6'd0;
      cnt_delay <= 20'd0;
      one_wire_buffer <= 1'bz;
    end
    MAIN:begin
      if(cnt_main >= 4'd11) cnt_main <= 1'b0;
      else cnt_main <= cnt_main + 1'b1;
      case(cnt_main)
        4'd0: begin state <= INIT; end
        4'd1: begin data_wr <= 8'hcc;state <= WRITE; end
        4'd2: begin data_wr <= 8'h44;state <= WRITE; end
        4'd3: begin num_delay <= 20'd750000;state <=
DELAY;state_back <= MAIN; end

        4'd4: begin state <= INIT; end
        4'd5: begin data_wr <= 8'hcc;state <= WRITE; end
        4'd6: begin data_wr <= 8'hbe;state <= WRITE; end

        4'd7: begin state <= READ; end
        4'd8: begin temperature[7:0] <= temperature_buffer; end

        4'd9: begin state <= READ; end
        4'd10: begin temperature[15:8] <= temperature_buffer; end

```

```

        4'd11: begin state <= IDLE; data_out <= temperature; end
        default: state <= IDLE;
    endcase
end
INIT: begin
    if(cnt_init >= 3'd6) cnt_init <= 1'b0;
    else cnt_init <= cnt_init + 1'b1;
    case(cnt_init)
        3'd0: begin one_wire_buffer <= 1'b0; end
        3'd1: begin num_delay <= 20'd500; state <= DELAY; state_back
<= INIT; end
        3'd2: begin one_wire_buffer <= 1'bz; end
        3'd3: begin num_delay <= 20'd100; state <= DELAY; state_back
<= INIT; end
        3'd4: begin if(one_wire) state <= IDLE; else state <= INIT; end
        3'd5: begin num_delay <= 20'd400; state <= DELAY; state_back
<= INIT; end
        3'd6: begin state <= MAIN; end
        default: state <= IDLE;
    endcase
end
WRITE: begin
    if(cnt_write >= 6'd50) cnt_write <= 1'b0;
    else cnt_write <= cnt_write + 1'b1;
    case(cnt_write)
        //lock data_wr
        6'd0: begin data_wr_buffer <= data_wr; end
        //write bit 0
        6'd1: begin one_wire_buffer <= 1'b0; end
        6'd2: begin num_delay <= 20'd2; state <= DELAY; state_back
<= WRITE; end
        6'd3: begin one_wire_buffer <= data_wr_buffer[0]; end
        6'd4: begin num_delay <= 20'd80; state <= DELAY; state_back
<= WRITE; end
        6'd5: begin one_wire_buffer <= 1'bz; end
        6'd6: begin num_delay <= 20'd2; state <= DELAY; state_back

```

```

<= WRITE; end

                                //write bit 1
6'd7: begin one_wire_buffer <= 1'b0; end
6'd8: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

6'd9: begin one_wire_buffer <= data_wr_buffer[1]; end
6'd10: begin num_delay <= 20'd80;state <= DELAY;state_back

<= WRITE; end

6'd11: begin one_wire_buffer <= 1'bz; end
6'd12: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

                                //write bit 2
6'd13: begin one_wire_buffer <= 1'b0; end
6'd14: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

6'd15: begin one_wire_buffer <= data_wr_buffer[2]; end
6'd16: begin num_delay <= 20'd80;state <= DELAY;state_back

<= WRITE; end

6'd17: begin one_wire_buffer <= 1'bz; end
6'd18: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

                                //write bit 3
6'd19: begin one_wire_buffer <= 1'b0; end
6'd20: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

6'd21: begin one_wire_buffer <= data_wr_buffer[3]; end
6'd22: begin num_delay <= 20'd80;state <= DELAY;state_back

<= WRITE; end

6'd23: begin one_wire_buffer <= 1'bz; end
6'd24: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

                                //write bit 4
6'd25: begin one_wire_buffer <= 1'b0; end
6'd26: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

6'd27: begin one_wire_buffer <= data_wr_buffer[4]; end
6'd28: begin num_delay <= 20'd80;state <= DELAY;state_back

```

```

<= WRITE; end

        6'd29: begin one_wire_buffer <= 1'bz; end
        6'd30: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

        //write bit 5
        6'd31: begin one_wire_buffer <= 1'b0; end
        6'd32: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

        6'd33: begin one_wire_buffer <= data_wr_buffer[5]; end
        6'd34: begin num_delay <= 20'd80;state <= DELAY;state_back

<= WRITE; end

        6'd35: begin one_wire_buffer <= 1'bz; end
        6'd36: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

        //write bit 6
        6'd37: begin one_wire_buffer <= 1'b0; end
        6'd38: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

        6'd39: begin one_wire_buffer <= data_wr_buffer[6]; end
        6'd40: begin num_delay <= 20'd80;state <= DELAY;state_back

<= WRITE; end

        6'd41: begin one_wire_buffer <= 1'bz; end
        6'd42: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

        //write bit 7
        6'd43: begin one_wire_buffer <= 1'b0; end
        6'd44: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

        6'd45: begin one_wire_buffer <= data_wr_buffer[7]; end
        6'd46: begin num_delay <= 20'd80;state <= DELAY;state_back

<= WRITE; end

        6'd47: begin one_wire_buffer <= 1'bz; end
        6'd48: begin num_delay <= 20'd2;state <= DELAY;state_back

<= WRITE; end

        //back to main
        6'd49: begin num_delay <= 20'd80;state <= DELAY;state_back

<= WRITE; end

```

```

        6'd50: begin state <= MAIN; end
        default: state <= IDLE;
    endcase
end
READ:begin
    if(cnt_read >= 6'd48) cnt_read <= 1'b0;
    else cnt_read <= cnt_read + 1'b1;
    case(cnt_read)
        //read bit 0
        6'd0: begin one_wire_buffer <= 1'b0; end
        6'd1: begin num_delay <= 20'd2;state <= DELAY;state_back
<= READ; end

        6'd2: begin one_wire_buffer <= 1'bz; end
        6'd3: begin num_delay <= 20'd10;state <= DELAY;state_back
<= READ; end

        6'd4: begin temperature_buffer[0] <= one_wire; end
        6'd5: begin num_delay <= 20'd55;state <= DELAY;state_back
<= READ; end

        //read bit 1
        6'd6: begin one_wire_buffer <= 1'b0; end
        6'd7: begin num_delay <= 20'd2;state <= DELAY;state_back
<= READ; end

        6'd8: begin one_wire_buffer <= 1'bz; end
        6'd9: begin num_delay <= 20'd10;state <= DELAY;state_back
<= READ; end

        6'd10: begin temperature_buffer[1] <= one_wire; end
        6'd11: begin num_delay <= 20'd55;state <= DELAY;state_back
<= READ; end

        //read bit 2
        6'd12: begin one_wire_buffer <= 1'b0; end
        6'd13: begin num_delay <= 20'd2;state <= DELAY;state_back
<= READ; end

        6'd14: begin one_wire_buffer <= 1'bz; end
        6'd15: begin num_delay <= 20'd10;state <= DELAY;state_back
<= READ; end

        6'd16: begin temperature_buffer[2] <= one_wire; end
        6'd17: begin num_delay <= 20'd55;state <= DELAY;state_back

```



```

<= READ; end

                                //read bit 3
6'd18: begin one_wire_buffer <= 1'b0; end
6'd19: begin num_delay <= 20'd2;state <= DELAY;state_back

<= READ; end

6'd20: begin one_wire_buffer <= 1'bz; end
6'd21: begin num_delay <= 20'd10;state <= DELAY;state_back

<= READ; end

6'd22: begin temperature_buffer[3] <= one_wire; end
6'd23: begin num_delay <= 20'd55;state <= DELAY;state_back

<= READ; end

                                //read bit 4
6'd24: begin one_wire_buffer <= 1'b0; end
6'd25: begin num_delay <= 20'd2;state <= DELAY;state_back

<= READ; end

6'd26: begin one_wire_buffer <= 1'bz; end
6'd27: begin num_delay <= 20'd10;state <= DELAY;state_back

<= READ; end

6'd28: begin temperature_buffer[4] <= one_wire; end
6'd29: begin num_delay <= 20'd55;state <= DELAY;state_back

<= READ; end

                                //read bit 5
6'd30: begin one_wire_buffer <= 1'b0; end
6'd31: begin num_delay <= 20'd2;state <= DELAY;state_back

<= READ; end

6'd32: begin one_wire_buffer <= 1'bz; end
6'd33: begin num_delay <= 20'd10;state <= DELAY;state_back

<= READ; end

6'd34: begin temperature_buffer[5] <= one_wire; end
6'd35: begin num_delay <= 20'd55;state <= DELAY;state_back

<= READ; end

                                //read bit 6
6'd36: begin one_wire_buffer <= 1'b0; end
6'd37: begin num_delay <= 20'd2;state <= DELAY;state_back

<= READ; end

6'd38: begin one_wire_buffer <= 1'bz; end
6'd39: begin num_delay <= 20'd10;state <= DELAY;state_back

```

```

<= READ; end

        6'd40: begin temperature_buffer[6] <= one_wire; end
        6'd41: begin num_delay <= 20'd55;state <= DELAY;state_back
<= READ; end

        //read bit 7
        6'd42: begin one_wire_buffer <= 1'b0; end
        6'd43: begin num_delay <= 20'd2;state <= DELAY;state_back
<= READ; end

        6'd44: begin one_wire_buffer <= 1'bz; end
        6'd45: begin num_delay <= 20'd10;state <= DELAY;state_back
<= READ; end

        6'd46: begin temperature_buffer[7] <= one_wire; end
        6'd47: begin num_delay <= 20'd55;state <= DELAY;state_back
<= READ; end

        //back to main
        6'd48: begin state <= MAIN; end
        default: state <= IDLE;

    endcase
end
DELAY:begin
    if(cnt_delay >= num_delay) begin
        cnt_delay <= 1'b0;
        state <= state_back;
    end else cnt_delay <= cnt_delay + 1'b1;
end
endcase
end
end

    assign    one_wire = one_wire_buffer;

endmodule

```

(3) 设计文件 bin_to_bcd.v

```

// *****
// File name   : bin_to_bcd
// Module name  : bcd_8421

```

```
//
// Author      :  STEP
// Date        :  2023/3/31
// Version     :  V 1.0
// Description  :  二进制转换为 BCD 码。
//
// Modification history
//-----
// $Log$
//
// *****

module bcd_8421
(
    input  wire      sys_clk      ,    //系统时钟，频率 50MHz
    input  wire      sys_rst_n    ,    //复位信号，低电平有效
    input  wire [20:0] data        ,    //输入需要转换的数据

    output reg [3:0] unit  ,//个位
    output reg [3:0] ten   ,//十位
    output reg [3:0] hun   ,//百位
    output reg [3:0] tho   ,//千位
    output reg [3:0] t_tho ,//万位
    output reg [3:0] h_hun //十万位
);
reg [24:0] bcd_code;
parameter B_SIZE = 21;

reg [45:0] shift_reg;
reg [4:0] cnt_shift ;    //移位判断计数器

always@(posedge sys_clk or negedge sys_rst_n)begin
    if(!sys_rst_n)
        cnt_shift <= 5'd0;
    else if(cnt_shift==5'd22)
        cnt_shift<=5'd0;
    else
```

```

        cnt_shift<=cnt_shift+1'b1;
end

always@(posedge sys_clk or negedge sys_rst_n)begin
    shift_reg= {25'h0,data};
    if(!sys_rst_n) bcd_code <= 0;
    else begin
        repeat(B_SIZE)//repeat B_SIZE times
            begin
                //BCD 码各位数据作满 5 加 3 操作
                if (shift_reg[24:21] >= 5) shift_reg[24:21] = shift_reg[24:21] + 2'b11;
                if (shift_reg[28:25] >= 5) shift_reg[28:25] = shift_reg[28:25] + 2'b11;
                if (shift_reg[32:29] >= 5) shift_reg[32:29] = shift_reg[32:29] + 2'b11;
                if (shift_reg[36:33] >= 5) shift_reg[36:33] = shift_reg[36:33] + 2'b11;
                if (shift_reg[40:37] >= 5) shift_reg[40:37] = shift_reg[40:37] + 2'b11;
                if (shift_reg[44:41] >= 5) shift_reg[44:41] = shift_reg[44:41] + 2'b11;
                shift_reg = shift_reg << 1'b1;
            end
        bcd_code<=shift_reg[45:21];
    end
end

always@(posedge sys_clk or negedge sys_rst_n)begin
    if(!sys_rst_n) begin
        unit   <=4'd0;
        ten    <=4'd0;
        hun    <=4'd0;
        tho    <=4'd0;
        t_tho  <=4'd0;
        h_hun  <=4'd0;
    end
    else if(cnt_shift==5'd22)begin
        unit   <=bcd_code[3:0]   ;
        ten    <=bcd_code[7:4]   ;
        hun    <=bcd_code[11:8]  ;
        tho    <=bcd_code[15:12] ;
    end
end

```

```

        t_tho <=bcd_code[19:16];
        h_hun <=bcd_code[23:20];
    end
else begin
        unit  <=unit  ;
        ten   <=ten   ;
        hun   <=hun   ;
        tho   <=tho   ;
        t_tho <=t_tho ;
        h_hun <=h_hun ;
    end

end

endmodule

```

(4) 设计文件 oled.v

```

// *****
// File name : Thermometer
// Module name : Thermometer
//
// Author      : STEP
// Date        : 2023/3/31
// Version     : V 1.0
// Description  : OLED12832_Driver, 使用 8*8 点阵字库, 每行显示 128/8=16 个字符。
//
// Modification history
//-----
// $Log$
//
// *****

module OLED12864
(
    input          clk,      //12MHz 系统时钟
    input          rst_n,    //系统复位, 低有效

```

```

//input      [7:0]    dis_dat,
// input      debug,
input [3:0] unit ,
input [3:0] ten ,
input [3:0] hun ,
input [3:0] tho ,
input [3:0] t_tho ,
input [3:0] h_hun ,

output reg    oled_csn, //OLCD 液晶屏使能
output reg    oled_rst, //OLCD 液晶屏复位
output reg    oled_dcn, //OLCD 数据指令控制
output reg    oled_clk, //OLCD 时钟信号
output reg    oled_dat //OLCD 数据信号
);

localparam INIT_DEPTH = 16'd23; //LCD 初始化的命令的数量
localparam IDLE = 7'h1, MAIN = 7'h2, INIT = 7'h4, SCAN = 7'h8, WRITE = 7'h10, DELAY =
7'h20;
localparam HIGH = 1'b1, LOW = 1'b0;
localparam DATA = 1'b1, CMD = 1'b0;

reg [7:0]      cmd [22:0]; //命令寄存器
reg [39:0]     mem [122:0]; //ASCII 字符
reg [7:0]      y_p, x_ph, x_pl;
reg [(8*21-1):0] char; //ASCII 字符
reg [7:0]      num; //ASCII 字符显示编号
reg [7:0]      char_reg; //ASCII 字符对应寄存器
reg [4:0]      cnt_main, cnt_init, cnt_scan, cnt_write;
reg [15:0]     num_delay, cnt_delay, cnt;
reg [6:0]      state, state_back;
wire [(8*8-1):0] dis_dat_buff; //dis_dat 缓存

assign dis_dat_buff = {4'd0, h_hun, 4'd0, t_tho, " ", 4'd0, tho, 4'd0, hun, "C"}; //

```

```

always @ (posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        cnt_main <= 1'b0; cnt_init <= 1'b0; cnt_scan <= 1'b0; cnt_write <= 1'b0;
        y_p <= 1'b0; x_ph <= 1'b0; x_pl <= 1'b0;
        num <= 1'b0; char <= 1'b0; char_reg <= 1'b0;
        num_delay <= 16'd5; cnt_delay <= 1'b0; cnt <= 1'b0;
        oled_csn <= HIGH; oled_rst <= HIGH; oled_dcn <= CMD; oled_clk <= HIGH;
oled_dat <= LOW;
        state <= IDLE; state_back <= IDLE;
    end else begin
        case(state)
            IDLE:begin
                cnt_main <= 1'b0; cnt_init <= 1'b0; cnt_scan <= 1'b0; cnt_write <=
1'b0;
                y_p <= 1'b0; x_ph <= 1'b0; x_pl <= 1'b0;
                num <= 1'b0; char <= 1'b0; char_reg <= 1'b0;
                num_delay <= 16'd5; cnt_delay <= 1'b0; cnt <= 1'b0;
                oled_csn <= HIGH; oled_rst <= HIGH; oled_dcn <= CMD; oled_clk
<= HIGH; oled_dat <= LOW;
                state <= MAIN; state_back <= MAIN;
            end
            MAIN:begin
                if(cnt_main >= 5'd10) cnt_main <= 5'd9;//接下来执行空操作，实
现数据只刷新一次
                else cnt_main <= cnt_main + 1'b1;
                case(cnt_main)    //MAIN 状态
                    5'd0 :    begin state <= INIT; end

                    5'd1 :    begin y_p <= 8'hb0; x_ph <= 8'h10; x_pl <= 8'h00;
num <= 5'd16; char <= "
";state <= SCAN; end

                    5'd2 :    begin y_p <= 8'hb1; x_ph <= 8'h10; x_pl <= 8'h00;
num <= 5'd16; char <= "
";state <= SCAN; end

                    5'd3 :    begin y_p <= 8'hb2; x_ph <= 8'h10; x_pl <= 8'h00;
num <= 5'd16; char <= "
";state <= SCAN; end

                    5'd4 :    begin y_p <= 8'hb3; x_ph <= 8'h10; x_pl <= 8'h00;
num <= 5'd16; char <= "
";state <= SCAN; end

```

```

                    5'd5 :   begin y_p <= 8'hb4; x_ph <= 8'h10; x_pl <= 8'h00;
num <= 5'd16; char <= "                    ";state <= SCAN; end
                    5'd6 :   begin y_p <= 8'hb5; x_ph <= 8'h10; x_pl <= 8'h00;
num <= 5'd16; char <= "                    ";state <= SCAN; end
                    5'd7 :   begin y_p <= 8'hb6; x_ph <= 8'h10; x_pl <= 8'h00;
num <= 5'd16; char <= "                    ";state <= SCAN; end
                    5'd8 :   begin y_p <= 8'hb7; x_ph <= 8'h10; x_pl <= 8'h00;
num <= 5'd16; char <= "                    ";state <= SCAN; end
                    5'd9 :   begin y_p <= 8'hb3; x_ph <= 8'h12; x_pl <= 8'h08;
num <= 5'd5;  char <= "Temp:";state <= SCAN; end
                    5'd10:   begin y_p <= 8'hb5; x_ph <= 8'h12; x_pl <= 8'h08;
num <= 5'd6;  char <= dis_dat_buff;state <= SCAN; end
                    default: state <= IDLE;    //如果你需要动态刷新一些信息,
此行应该取消注释
                    endcase
                end
            INIT:begin    //初始化状态
                case(cnt_init)
                    5'd0:    begin oled_rst <= LOW; cnt_init <= cnt_init + 1'b1;
end //复位有效
                    5'd1:    begin num_delay <= 16'd25000; state <= DELAY;
state_back <= INIT; cnt_init <= cnt_init + 1'b1; end //延时大于 3us
                    5'd2:    begin oled_rst <= HIGH; cnt_init <= cnt_init + 1'b1;
end //复位恢复
                    5'd3:    begin num_delay <= 16'd25000; state <= DELAY;
state_back <= INIT; cnt_init <= cnt_init + 1'b1; end //延时大于 220us
                    5'd4:    begin
                                if(cnt>=INIT_DEPTH) begin //当 25 条指令及
数据发出后, 配置完成
                                    cnt <= 1'b0;
                                    cnt_init <= cnt_init + 1'b1;
                                end else begin
                                    cnt <= cnt + 1'b1; num_delay <= 16'd5;
                                    oled_dcn <= CMD; char_reg <= cmd[cnt];
state <= WRITE; state_back <= INIT;
                                end
                            end
            end

```



```

5'd5:    begin cnt_init <= 1'b0; state <= MAIN; end //初始化
完成, 返回 MAIN 状态

        default: state <= IDLE;
    endcase
end
SCAN:begin //刷屏状态, 从 RAM 中读取数据刷屏
    if(cnt_scan == 5'd11) begin
        if(num) cnt_scan <= 5'd3;
        else cnt_scan <= cnt_scan + 1'b1;
    end
    else if(cnt_scan == 5'd12) cnt_scan <= 1'b0;
    else cnt_scan <= cnt_scan + 1'b1;
    case(cnt_scan)
        5'd 0:    begin oled_dcn <= CMD; char_reg <= y_p; state <=
WRITE; state_back <= SCAN; end //定位列页地址
        5'd 1:    begin oled_dcn <= CMD; char_reg <= x_pl; state <=
WRITE; state_back <= SCAN; end//定位行地址低位
        5'd 2:    begin oled_dcn <= CMD; char_reg <= x_ph; state <=
WRITE; state_back <= SCAN; end//定位行地址高位

        5'd 3:    begin num <= num - 1'b1;end
        5'd 4:    begin oled_dcn <= DATA; char_reg <= 8'h00; state
<= WRITE; state_back <= SCAN; end //将 5*8 点阵编程 8*8
        5'd 5:    begin oled_dcn <= DATA; char_reg <= 8'h00; state
<= WRITE; state_back <= SCAN; end //将 5*8 点阵编程 8*8
        5'd 6:    begin oled_dcn <= DATA; char_reg <= 8'h00; state
<= WRITE; state_back <= SCAN; end //将 5*8 点阵编程 8*8
        5'd 7:    begin oled_dcn <= DATA; char_reg <=
mem[char[(num*8)+:8]][39:32]; state <= WRITE; state_back <= SCAN; end
        5'd 8:    begin oled_dcn <= DATA; char_reg <=
mem[char[(num*8)+:8]][31:24]; state <= WRITE; state_back <= SCAN; end
        5'd 9:    begin oled_dcn <= DATA; char_reg <=
mem[char[(num*8)+:8]][23:16]; state <= WRITE; state_back <= SCAN; end
        5'd10:    begin oled_dcn <= DATA; char_reg <=
mem[char[(num*8)+:8]][15: 8]; state <= WRITE; state_back <= SCAN; end
        5'd11:    begin oled_dcn <= DATA; char_reg <=
mem[char[(num*8)+:8]][ 7: 0]; state <= WRITE; state_back <= SCAN; end

```

```

5'd12:  begin state <= MAIN; end
        default: state <= IDLE;
    endcase
end
WRITE:begin //WRITE 状态，将数据按照 SPI 时序发送给屏幕
    if(cnt_write >= 5'd17) cnt_write <= 1'b0;
    else cnt_write <= cnt_write + 1'b1;
    case(cnt_write)
        5'd 0:  begin oled_csn <= LOW; end //9 位数据最高位为命令数据控制位
        5'd 1:  begin oled_clk <= LOW; oled_dat <= char_reg[7];
    end //先发高位数据
        5'd 2:  begin oled_clk <= HIGH; end
        5'd 3:  begin oled_clk <= LOW; oled_dat <= char_reg[6];
    end
        5'd 4:  begin oled_clk <= HIGH; end
        5'd 5:  begin oled_clk <= LOW; oled_dat <= char_reg[5];
    end
        5'd 6:  begin oled_clk <= HIGH; end
        5'd 7:  begin oled_clk <= LOW; oled_dat <= char_reg[4];
    end
        5'd 8:  begin oled_clk <= HIGH; end
        5'd 9:  begin oled_clk <= LOW; oled_dat <= char_reg[3];
    end
        5'd10:  begin oled_clk <= HIGH; end
        5'd11:  begin oled_clk <= LOW; oled_dat <= char_reg[2];
    end
        5'd12:  begin oled_clk <= HIGH; end
        5'd13:  begin oled_clk <= LOW; oled_dat <= char_reg[1];
    end
        5'd14:  begin oled_clk <= HIGH; end
        5'd15:  begin oled_clk <= LOW; oled_dat <= char_reg[0];
    end //后发低位数据
        5'd16:  begin oled_clk <= HIGH; end
        5'd17:  begin oled_csn <= HIGH; state <= DELAY; end //
        default: state <= IDLE;
    endcase

```

```

        end
        DELAY:begin //延时状态
            if(cnt_delay >= num_delay) begin
                cnt_delay <= 16'd0; state <= state_back;
            end else cnt_delay <= cnt_delay + 1'b1;
        end
        end
        default:state <= IDLE;
    endcase
end
end
end

```

//OLED 配置指令数据

```

always@(posedge rst_n)
begin
    cmd[0 ] = {8'hae};
    cmd[1 ] = {8'hd5};
    cmd[2 ] = {8'h80};
    cmd[3 ] = {8'ha8};
    cmd[4 ] = {8'h3f};
    cmd[5 ] = {8'hd3};
    cmd[6 ] = {8'h00};
    cmd[7 ] = {8'h40};
    cmd[8 ] = {8'h8d};
    cmd[9 ] = {8'h14};
    cmd[10] = {8'h20};
    cmd[11] = {8'h02};
    cmd[12] = {8'hc8};
    cmd[13] = {8'ha1};
    cmd[14] = {8'hda};
    cmd[15] = {8'h12};
    cmd[16] = {8'h81};
    cmd[17] = {8'hcf};
    cmd[18] = {8'hd9};
    cmd[19] = {8'hf1};
    cmd[20] = {8'hdb};
    cmd[21] = {8'h40};
    cmd[22] = {8'haf};
end

```

```
end
```

```
//5*8 点阵字库数据
```

```
always@(posedge rst_n)
```

```
begin
```

```
    mem[ 0] = {8'h3E, 8'h51, 8'h49, 8'h45, 8'h3E}; // 48 0
    mem[ 1] = {8'h00, 8'h42, 8'h7F, 8'h40, 8'h00}; // 49 1
    mem[ 2] = {8'h42, 8'h61, 8'h51, 8'h49, 8'h46}; // 50 2
    mem[ 3] = {8'h21, 8'h41, 8'h45, 8'h4B, 8'h31}; // 51 3
    mem[ 4] = {8'h18, 8'h14, 8'h12, 8'h7F, 8'h10}; // 52 4
    mem[ 5] = {8'h27, 8'h45, 8'h45, 8'h45, 8'h39}; // 53 5
    mem[ 6] = {8'h3C, 8'h4A, 8'h49, 8'h49, 8'h30}; // 54 6
    mem[ 7] = {8'h01, 8'h71, 8'h09, 8'h05, 8'h03}; // 55 7
    mem[ 8] = {8'h36, 8'h49, 8'h49, 8'h49, 8'h36}; // 56 8
    mem[ 9] = {8'h06, 8'h49, 8'h49, 8'h29, 8'h1E}; // 57 9
    mem[10] = {8'h7C, 8'h12, 8'h11, 8'h12, 8'h7C}; // 65 A
    mem[11] = {8'h7F, 8'h49, 8'h49, 8'h49, 8'h36}; // 66 B
    mem[12] = {8'h3E, 8'h41, 8'h41, 8'h41, 8'h22}; // 67 C
    mem[13] = {8'h7F, 8'h41, 8'h41, 8'h22, 8'h1C}; // 68 D
    mem[14] = {8'h7F, 8'h49, 8'h49, 8'h49, 8'h41}; // 69 E
    mem[15] = {8'h7F, 8'h09, 8'h09, 8'h09, 8'h01}; // 70 F
```

```
    mem[32] = {8'h00, 8'h00, 8'h00, 8'h00, 8'h00}; // 32 sp
    mem[33] = {8'h00, 8'h00, 8'h2f, 8'h00, 8'h00}; // 33 !
    mem[34] = {8'h00, 8'h07, 8'h00, 8'h07, 8'h00}; // 34
    mem[35] = {8'h14, 8'h7f, 8'h14, 8'h7f, 8'h14}; // 35 #
    mem[36] = {8'h24, 8'h2a, 8'h7f, 8'h2a, 8'h12}; // 36 $
    mem[37] = {8'h62, 8'h64, 8'h08, 8'h13, 8'h23}; // 37 %
    mem[38] = {8'h36, 8'h49, 8'h55, 8'h22, 8'h50}; // 38 &
    mem[39] = {8'h00, 8'h05, 8'h03, 8'h00, 8'h00}; // 39 '
    mem[40] = {8'h00, 8'h1c, 8'h22, 8'h41, 8'h00}; // 40 (
    mem[41] = {8'h00, 8'h41, 8'h22, 8'h1c, 8'h00}; // 41 )
    mem[42] = {8'h14, 8'h08, 8'h3E, 8'h08, 8'h14}; // 42 *
    mem[43] = {8'h08, 8'h08, 8'h3E, 8'h08, 8'h08}; // 43 +
    mem[44] = {8'h00, 8'h00, 8'hA0, 8'h60, 8'h00}; // 44 ,
    mem[45] = {8'h08, 8'h08, 8'h08, 8'h08, 8'h08}; // 45 -
```

```

mem[ 46] = {8'h00, 8'h60, 8'h60, 8'h00, 8'h00}; // 46  .
mem[ 47] = {8'h20, 8'h10, 8'h08, 8'h04, 8'h02}; // 47  /
mem[ 48] = {8'h3E, 8'h51, 8'h49, 8'h45, 8'h3E}; // 48  0
mem[ 49] = {8'h00, 8'h42, 8'h7F, 8'h40, 8'h00}; // 49  1
mem[ 50] = {8'h42, 8'h61, 8'h51, 8'h49, 8'h46}; // 50  2
mem[ 51] = {8'h21, 8'h41, 8'h45, 8'h4B, 8'h31}; // 51  3
mem[ 52] = {8'h18, 8'h14, 8'h12, 8'h7F, 8'h10}; // 52  4
mem[ 53] = {8'h27, 8'h45, 8'h45, 8'h45, 8'h39}; // 53  5
mem[ 54] = {8'h3C, 8'h4A, 8'h49, 8'h49, 8'h30}; // 54  6
mem[ 55] = {8'h01, 8'h71, 8'h09, 8'h05, 8'h03}; // 55  7
mem[ 56] = {8'h36, 8'h49, 8'h49, 8'h49, 8'h36}; // 56  8
mem[ 57] = {8'h06, 8'h49, 8'h49, 8'h29, 8'h1E}; // 57  9
mem[ 58] = {8'h00, 8'h36, 8'h36, 8'h00, 8'h00}; // 58  :
mem[ 59] = {8'h00, 8'h56, 8'h36, 8'h00, 8'h00}; // 59  ;
mem[ 60] = {8'h08, 8'h14, 8'h22, 8'h41, 8'h00}; // 60  <
mem[ 61] = {8'h14, 8'h14, 8'h14, 8'h14, 8'h14}; // 61  =
mem[ 62] = {8'h00, 8'h41, 8'h22, 8'h14, 8'h08}; // 62  >
mem[ 63] = {8'h02, 8'h01, 8'h51, 8'h09, 8'h06}; // 63  ?
mem[ 64] = {8'h32, 8'h49, 8'h59, 8'h51, 8'h3E}; // 64  @
mem[ 65] = {8'h7C, 8'h12, 8'h11, 8'h12, 8'h7C}; // 65  A
mem[ 66] = {8'h7F, 8'h49, 8'h49, 8'h49, 8'h36}; // 66  B
mem[ 67] = {8'h3E, 8'h41, 8'h41, 8'h41, 8'h22}; // 67  C
mem[ 68] = {8'h7F, 8'h41, 8'h41, 8'h22, 8'h1C}; // 68  D
mem[ 69] = {8'h7F, 8'h49, 8'h49, 8'h49, 8'h41}; // 69  E
mem[ 70] = {8'h7F, 8'h09, 8'h09, 8'h09, 8'h01}; // 70  F
mem[ 71] = {8'h3E, 8'h41, 8'h49, 8'h49, 8'h7A}; // 71  G
mem[ 72] = {8'h7F, 8'h08, 8'h08, 8'h08, 8'h7F}; // 72  H
mem[ 73] = {8'h00, 8'h41, 8'h7F, 8'h41, 8'h00}; // 73  I
mem[ 74] = {8'h20, 8'h40, 8'h41, 8'h3F, 8'h01}; // 74  J
mem[ 75] = {8'h7F, 8'h08, 8'h14, 8'h22, 8'h41}; // 75  K
mem[ 76] = {8'h7F, 8'h40, 8'h40, 8'h40, 8'h40}; // 76  L
mem[ 77] = {8'h7F, 8'h02, 8'h0C, 8'h02, 8'h7F}; // 77  M
mem[ 78] = {8'h7F, 8'h04, 8'h08, 8'h10, 8'h7F}; // 78  N
mem[ 79] = {8'h3E, 8'h41, 8'h41, 8'h41, 8'h3E}; // 79  O
mem[ 80] = {8'h7F, 8'h09, 8'h09, 8'h09, 8'h06}; // 80  P
mem[ 81] = {8'h3E, 8'h41, 8'h51, 8'h21, 8'h5E}; // 81  Q
mem[ 82] = {8'h7F, 8'h09, 8'h19, 8'h29, 8'h46}; // 82  R

```

```

mem[ 83] = {8'h46, 8'h49, 8'h49, 8'h49, 8'h31}; // 83  S
mem[ 84] = {8'h01, 8'h01, 8'h7F, 8'h01, 8'h01}; // 84  T
mem[ 85] = {8'h3F, 8'h40, 8'h40, 8'h40, 8'h3F}; // 85  U
mem[ 86] = {8'h1F, 8'h20, 8'h40, 8'h20, 8'h1F}; // 86  V
mem[ 87] = {8'h3F, 8'h40, 8'h38, 8'h40, 8'h3F}; // 87  W
mem[ 88] = {8'h63, 8'h14, 8'h08, 8'h14, 8'h63}; // 88  X
mem[ 89] = {8'h07, 8'h08, 8'h70, 8'h08, 8'h07}; // 89  Y
mem[ 90] = {8'h61, 8'h51, 8'h49, 8'h45, 8'h43}; // 90  Z
mem[ 91] = {8'h00, 8'h7F, 8'h41, 8'h41, 8'h00}; // 91  [
mem[ 92] = {8'h55, 8'h2A, 8'h55, 8'h2A, 8'h55}; // 92  .
mem[ 93] = {8'h00, 8'h41, 8'h41, 8'h7F, 8'h00}; // 93  ]
mem[ 94] = {8'h04, 8'h02, 8'h01, 8'h02, 8'h04}; // 94  ^
mem[ 95] = {8'h40, 8'h40, 8'h40, 8'h40, 8'h40}; // 95  _
mem[ 96] = {8'h00, 8'h01, 8'h02, 8'h04, 8'h00}; // 96  '
mem[ 97] = {8'h20, 8'h54, 8'h54, 8'h54, 8'h78}; // 97  a
mem[ 98] = {8'h7F, 8'h48, 8'h44, 8'h44, 8'h38}; // 98  b
mem[ 99] = {8'h38, 8'h44, 8'h44, 8'h44, 8'h20}; // 99  c
mem[100] = {8'h38, 8'h44, 8'h44, 8'h48, 8'h7F}; // 100 d
mem[101] = {8'h38, 8'h54, 8'h54, 8'h54, 8'h18}; // 101 e
mem[102] = {8'h08, 8'h7E, 8'h09, 8'h01, 8'h02}; // 102 f
mem[103] = {8'h18, 8'hA4, 8'hA4, 8'hA4, 8'h7C}; // 103 g
mem[104] = {8'h7F, 8'h08, 8'h04, 8'h04, 8'h78}; // 104 h
mem[105] = {8'h00, 8'h44, 8'h7D, 8'h40, 8'h00}; // 105 i
mem[106] = {8'h40, 8'h80, 8'h84, 8'h7D, 8'h00}; // 106 j
mem[107] = {8'h7F, 8'h10, 8'h28, 8'h44, 8'h00}; // 107 k
mem[108] = {8'h00, 8'h41, 8'h7F, 8'h40, 8'h00}; // 108 l
mem[109] = {8'h7C, 8'h04, 8'h18, 8'h04, 8'h78}; // 109 m
mem[110] = {8'h7C, 8'h08, 8'h04, 8'h04, 8'h78}; // 110 n
mem[111] = {8'h38, 8'h44, 8'h44, 8'h44, 8'h38}; // 111 o
mem[112] = {8'hFC, 8'h24, 8'h24, 8'h24, 8'h18}; // 112 p
mem[113] = {8'h18, 8'h24, 8'h24, 8'h18, 8'hFC}; // 113 q
mem[114] = {8'h7C, 8'h08, 8'h04, 8'h04, 8'h08}; // 114 r
mem[115] = {8'h48, 8'h54, 8'h54, 8'h54, 8'h20}; // 115 s
mem[116] = {8'h04, 8'h3F, 8'h44, 8'h40, 8'h20}; // 116 t
mem[117] = {8'h3C, 8'h40, 8'h40, 8'h20, 8'h7C}; // 117 u
mem[118] = {8'h1C, 8'h20, 8'h40, 8'h20, 8'h1C}; // 118 v
mem[119] = {8'h3C, 8'h40, 8'h30, 8'h40, 8'h3C}; // 119 w

```

```
mem[120] = {8'h44, 8'h28, 8'h10, 8'h28, 8'h44}; // 120 x
mem[121] = {8'h1C, 8'hA0, 8'hA0, 8'hA0, 8'h7C}; // 121 y
mem[122] = {8'h44, 8'h64, 8'h54, 8'h4C, 8'h44}; // 122 z

end

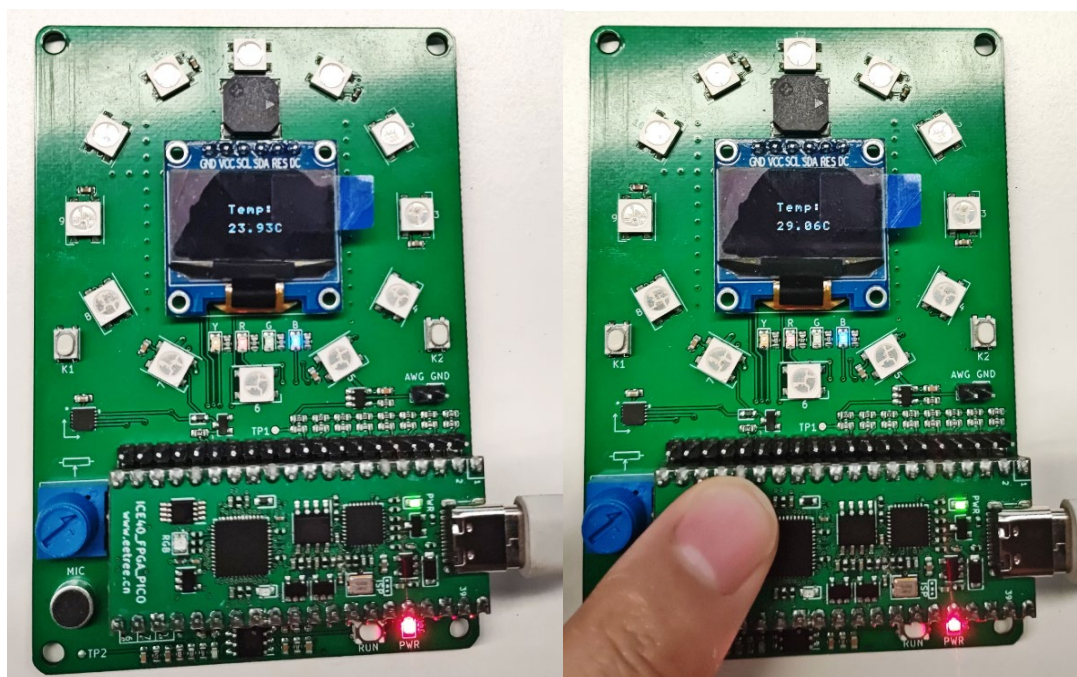
endmodule
```

4.1.4 实验步骤

1. 打开 Ridian, 建立工程。
2. 新建 Verilog HDL 设计文件, 并键入设计代码, 包括添加需要调用的模块文件。
3. 构建并输出编程文件, 烧写至 FPGA 的 Flash 之中。

4.1.5 实验现象

实验结果如下:



左图为测得的环境温度为 23.93 摄氏度, 右图为手指放到温度传感器上的温度为 29.06 摄氏度。

第五章 基于比较器的数字电压表实验

实验简介

本教程以实验为主，结合相应的实验案例，以详细的讲解和实验步骤让大家尽快熟悉 FPGA 开发的基本流程、Ridian 软件平台的使用方法以及本节实验相关模块的工作原理。

- 熟悉和掌握 FPGA 开发的基本流程
- 掌握 Ridian 软件平台的基本使用方法
- 了解使用 SigmaDelta ADC 的原理
- 掌握使用 Verilog HDL 语言基于 FPGA 实现 ADC 数字电压表的原理及实现方法

背景知识

多数 FPGA 芯片上没有 ADC 的功能，而一些应用则需要用到 ADC 对一些模拟信号，比如直流电压等进行量化，有没有特别简单、低成本的实现方法呢？

在要求转换速率不高的情况下，完全可以借助一颗高速比较器（成本只有几毛钱）来实现对模拟信号的量化，Lattice 的官网上一篇文章就介绍了如何制作一个简易的 Sigma Delta ADC，如果 FPGA 能够提供 LVDS 的接口，连外部的高速比较器都可以省掉。由于我们的小脚丫 FPGA 核心模块在设计的时候没有考虑到 LVDS 的应用场景，所以还是需要搭配一个高速的比较器来实现 Lattice 官网上推荐的简易 Sigma Delta ADC 的功能。

工作原理：

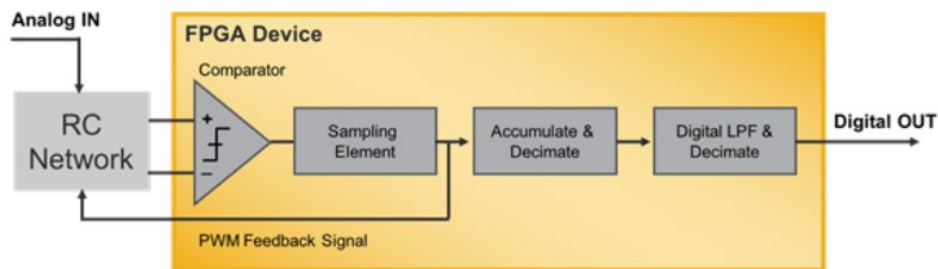


图 1 简易 Sigma Delta ADC 的工作原理

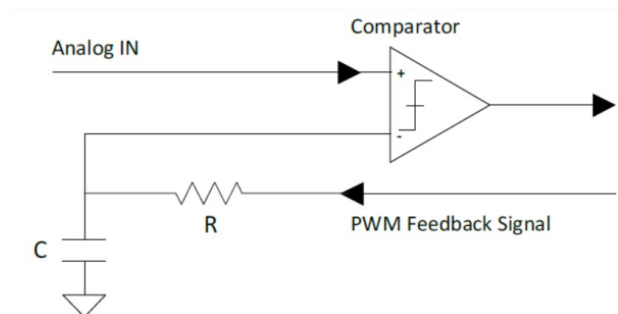


图 2 直接连接 - 被测模拟信号的幅度范围为 0-3.3V

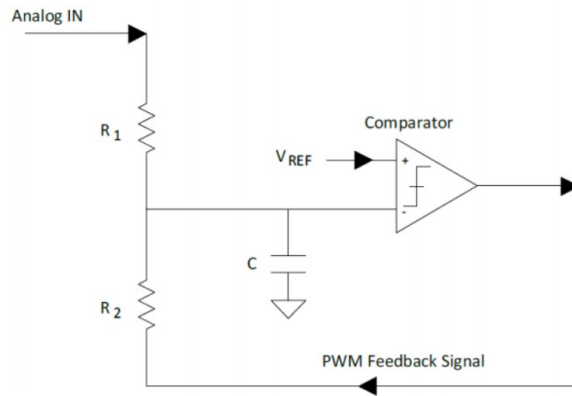


图 3：通过电阻分压网络输入，并在比较器+端提供参考电压，则被采集模拟信号的电压变化范围可以扩展

Operation Frequency	Output Sample Rate	Input Frequency (Hz)	8-Bit SSD SNR	8-Bit SSD ENOB1	10-Bit SSD SNR	10-Bit SSD ENOB*	Operation Frequency
62.5 MHz	7.63 KHz	50	47.0	7.12	54.9	8.60	62.5 MHz
62.5 MHz	7.63 KHz	1000	46.7	7.18	52.8	8.25	62.5 MHz
62.5 MHz	7.63 KHz	3800	42.5	6.74	53.1	8.53	62.5 MHz

Note: ENOB = Equivalent Number of Bits of resolution.

图 4：简易 Sigma Delta ADC 的性能与逻辑电路的工作频率

Device	Tool/Coding Language	Speed Grade	Utilization (LUTs)	fMAX (MHz)	I/Os	Architecture Resources
iCE40 UltraPlus ⁵	Verilog-LSE	NA	99	>150	13	NA
	Verilog-Syn	NA	101	>150	13	NA
MachXO2™ 2	Verilog-LSE	-6	49	>150	13	NA
	Verilog-Syn	-6	61	>150	13	NA
	VHDL-LSE	-6	49	>150	13	NA
	VHDL-Syn	-6	61	>150	13	NA
MachXO3	Verilog-LSE	-5	46	>150	13	NA
	Verilog-Syn	-5	48	>150	13	NA
	VHDL-LSE	-5	46	>150	13	NA
	VHDL-Syn	-5	48	>150	13	NA
LatticeXP2™ 4	Verilog-Syn	-5	62	>150	13	NA
	VHDL-Syn	-5	62	>150	13	NA

图 5：在不同的 FPGA 平台上消耗的逻辑资源

实验环境

- 硬件环境：STEP FPGA 实验平台
- 软件环境：Ridiant 开发环境

5.1 实验五 基于比较器的数字电压表实验

5.1.1 实验任务

利用 ADC 制作一个数字电压表,旋转电位计可以产生 0-3.3V 的电压,利用板上的串行 ADC 对电压进行转换,将电压值在板上的 OLED 屏幕上显示出来。



5.1.2 实验原理

按照项目要求来实现,我们要制作的是一个数字电压表,而电压值是一个模拟量,所以我们需要使用 ADC (模数转换器) 把模拟信号转换为数字信号,根据开发板的原理图可以看出来,板上的旋转电位计与高速比较器 (TP1961) 正向输入端相连。我们通过高速比较器和 FPGA 逻辑实现的 Sigma DeltaADC 对旋转电位计的模拟量进行采集,将采集量进行进制转换,然后实时的显示在 OLED 屏幕上,这样就可以基本完成项目的要求了。

整个程序有六个模块:

ADC_top: 顶层模块。

sigmadelta_adc: SSD 模数转换器。

box_ave: 箱式滤波器。

bintobcd: 二进制转换为 bcd 码。

OLEDshow: OLED 驱动模块。

my_pll: 锁相环模块,将 12MHZ 的时钟转换为 48MHZ 时钟。

5.1.3 Verilog HDL 建模描述

(1) 设计文件 adc_top.v

```

// *****
// File name : adc_top
// Module name : ADC_top
//
// Author : STEP
// Date : 2023/3/31
// Version : V 1.0
  
```

```

// Description    : 顶层模块。
//
// Modification history
//-----
// $Log$
//
// *****

module ADC_top (
    i_rst_in, // 低电平复位
    i_analog_cmp, // 比较器输出, 可用 lvds 替代
    // o_digital_out, // 滤波器并行输出
    o_analog_out, // 反馈路径, RC 滤波后进入比较器
    o_sample_rdy, // 输出数据有效
    oled_rst, // OLCD 液晶屏复位
    oled_dcn, // OLCD 数据指令控制 DC
    oled_clk, // OLCD 时钟信号 D0
    oled_dat // OLCD 数据信号 D1
);

parameter
    ADC_WIDTH = 8, // ADC Convertor Bit Precision
    ACCUM_BITS = 10, // 2^ACCUM_BITS is decimation rate of
    accumulator // 累加器位宽, analog_cmp 进入此处, 必须大于 ADC 精度? 2^ACCUM_BITS 是信号
    累加器大小
    LPF_DEPTH_BITS = 3, // 2^LPF_DEPTH_BITS is decimation rate of
    averager // 累加器位宽, analog_cmp 进入此处, 必须大于 ADC 精度? 2^ACCUM_BITS 是信号累加
    器大小
    INPUT_TOPOLOGY = 0; // 0: DIRECT: Analog input directly connected to
    + input of comparator // 反馈参数配置
    // 1: NETWORK: Analog input connected
    through R divider to - input of comp.
    /*
    *0: DIRECT: Analog input directly
    connected to + input of comparitor
    *模拟信号+端输入, 反馈信号 RC 滤波
    后从-输入
    *适合测量 0-VCC 的模拟信号

```

through R divider to - input of comp.

接积分器

1: NETWORK:Analog input connected

*模拟信号与反馈信号经过加法器，连

*扩大了电压测量范围

*/

```
wire [ADC_WIDTH-1:0] o_digital_out; // connected to I2C register.
```

```
wire i_clk_in; // 12Mhz on Control Demo board
```

```
//input ports
```

```
input i_rst_in;
```

```
input i_analog_cmp; // from LVDS buffer or external comparitor
```

```
//output ports
```

```
output o_analog_out; // feedback to RC network
```

```
output o_sample_rdy;
```

```
output oled_rst; //OLCD 液晶屏复位
```

```
output oled_dcn; //OLCD 数据指令控制 DC
```

```
output oled_clk; //OLCD 时钟信号 D0
```

```
output oled_dat; //OLCD 数据信号 D1
```

```
HSOSC #(  
.CLKHF_DIV ("0b10")  
) OSCInst0 (  
.CLKHFEN (1'b1 ),  
.CLKHFPU (1'b1 ),  
.CLKHF (i_clk_in)  
);
```

```
//*****
```

```
//
```

```
// Internal Wire & Reg Signals
```

```
//
```

```
//*****
```

```
wire sys_clk;
```

```

        assign sys_clk = i_clk_in;

/*
HSOSC
#(
    .CLKHF_DIV ("0b10") //Clock divider selection.0b10=12MHz
) u_HSOSC (
    .CLKHFEN (1'b1),
    .CLKHFPU (1'b1),
    .CLKHF    (sys_clk)
);
*/

    wire analog_out_i;
    wire sample_rdy_i;
    wire [ADC_WIDTH-1:0]    digital_out_i;
    wire [ADC_WIDTH-1:0]    digital_out_abs;
    wire sys_rst_n,clk_gen_locked,clk_pwm_adc;

    //将 ADC 采样数据按规则转换为电压数据（乘以 0.0129=3.3/2^8），这里我们直接
    乘以 129，得到的数据经过 BCD 转码后小数点左移 4 位即可
    wire [15:0]    bin_code = digital_out_abs * 16'd130;
    wire [19:0]    bcd_code;

    //*****
    //
    //  锁相环倍频
    //
    //*****

    assign sys_rst_n = i_rst_in & clk_gen_locked;
my_pll _(.ref_clk_i( sys_clk),
        .rst_n_i( i_rst_in),
        .lock_o( clk_gen_locked),
        .outcore_o( ),
        .outglobal_o( clk_pwm_adc));

    //*****
    //
    //  SSD ADC using onboard LVDS buffer or external comparator
    //
    //*****

```

```

sigmadelta_adc #(
    .ADC_WIDTH(ADC_WIDTH),
    .ACCUM_BITS(ACCUM_BITS),
    .LPF_DEPTH_BITS(LPF_DEPTH_BITS)
)

u1(
    .clk(clk_pwm_adc),
    .rstn(sys_rst_n),
    .analog_cmp(i_analog_cmp),
    .digital_out(digital_out_i),
    .analog_out(analog_out_i),
    .sample_rdy(sample_rdy_i)
);

//*****
//
// 将处理后的 ADC 数据进行 BCD 转码，例化
//
//*****
bintobcd u2(
    .rst_n          (sys_rst_n),  //系统复位，低有效
    .bin_code        (bin_code),  //需要进行 BCD 转码的二进制数据
    .bcd_code         (bcd_code)  //转码后的 BCD 码型数据输出
);

//*****
//
// OLED 显示电压
//
//*****
OLEDshow u3(
    .clk      (clk_pwm_adc),
    .rst_n    (sys_rst_n),

    .data     (bcd_code[19:12]),

    //oled_csn(oled_csn),
    .oled_rst(oled_rst),
    .oled_dcn(oled_dcn),

```



```

        .oled_clk(oled_clk),
        .oled_dat(oled_dat)

    );

//*****

//
//  output assignments
//
//*****

assign digital_out_abs = INPUT_TOPOLOGY ? ~digital_out_i : digital_out_i;
assign o_digital_out   = digital_out_abs;
assign o_analog_out    = analog_out_i;
assign o_sample_rdy    = sample_rdy_i;

endmodule

```

(2) 设计文件 sigmadelta_adc_ice40.v

```

// *****
// File name   : sigmadelta_adc_ice40
// Module name  : sigmadelta_adc
//
// Author      : STEP
// Date        : 2023/3/31
// Version     : V 1.0
// Description  : SSD 模数转换器。
//
// Modification history
//-----
// $Log$
//
// *****

//*****
//
//  SSD Top Level Module
//

```

```

//*****

module sigmadelta_adc (
    clk,
    rstn,
    digital_out,
    analog_cmp,
    analog_out,
    sample_rdy);

parameter
ADC_WIDTH = 8,           // ADC Convertor Bit Precision// ADC 精度，单位 bit
ACCUM_BITS = 10,         // 2^ACCUM_BITS is decimation rate of accumulator// 累加器位
                        宽，analog_cmp 进入此处，必须大于 ADC 精度
LPF_DEPTH_BITS = 3;      // 2^LPF_DEPTH_BITS is decimation rate of averager // 均值滤波
                        器深度，2^LPF_DEPTH_BITS 是抽取速率

//input ports
input  clk;               // sample rate clock
input  rstn;              // async reset, asserted low
input  analog_cmp ;       // input from LVDS buffer (comparator)

//output ports
output analog_out;        // feedback to comparator input RC circuit
output sample_rdy;        // digital_out is ready
output [ADC_WIDTH-1:0] digital_out; // digital output word of ADC

//*****
//
// Internal Wire & Reg Signals 内部连线，寄存器声明
//

```

```

//*****

reg [ACCUM_BITS-1:0]    sigma;          // running accumulator value
reg [ADC_WIDTH-1:0]     accum;          // latched accumulator value
reg [ACCUM_BITS-1:0]    counter;        // decimation counter for accumulator
reg                     rollover;       // decimation counter terminal count
reg                     accum_rdy;      // latched accumulator value 'ready'


//*****
//
//  SSD 'Analog' Input - PWM
//
//  External Comparator Generates High/Low Value
//
//*****
// For the ice40 there is a register in the component

reg      delta;          // captured comparitor output
always @ (posedge clk)
begin
    delta <= analog_cmp;    // capture comparitor output
end

assign analog_out = delta;    // feedback to comparitor LPF
assign analog_out = delta;    // feedback to comparitor LPF


//*****
// 将 01 序列转换为 N 位占空比
//  Accumulator Stage
//
//  Adds PWM positive pulses over accumulator period
//
//*****

```

```

always @ (posedge clk or negedge rstn)
begin
    if( ~rstn )
    begin
        sigma      <= 0;
        accum      <= 0;
        accum_rdy   <= 0;
    end else begin
        if (rollover) begin
            // latch top ADC_WIDTH bits of sigma accumulator (drop LSBs)
            accum <= sigma[ACCUM_BITS-1:ACCUM_BITS-ADC_WIDTH];
            sigma <= delta;          // reset accumulator, prime with current delta value
        end else begin
            if (&sigma != 1'b1)      // if not saturated
                sigma <= sigma + delta; // accumulate
        end
        accum_rdy <= rollover;      // latch 'rdy' (to align with accum)
    end
end

//*****
// 低通滤波器，均值滤波
//  Box filter Average
//
//  Acts as simple decimating Low-Pass Filter
//
//*****

box_ave #(
    .ADC_WIDTH(ADC_WIDTH),
    .LPF_DEPTH_BITS(LPF_DEPTH_BITS))
box_ave (
    .clk(clk),
    .rstn(rstn),

```

```

        .sample(accum_rdy),
        .raw_data_in(accum),
        .ave_data_out(digital_out),
        .data_out_valid(sample_rdy)
    );

//*****
//
// Sample Control - Accumulator Timing 采样转换时序控制
//
//*****

always @(posedge clk or negedge rstn)
begin
    if( ~rstn ) begin
        counter <= 0;
        rollover <= 0;
    end
    else begin
        counter <= counter + 1;        // running count
        rollover <= &counter;         // assert 'rollover' when counter is all 1's
    end
end

endmodule

```

(3) 设计文件 box_ave.v

```

// *****
// File name : box_ave
// Module name : box_ave
//
// Author      : STEP
// Date        : 2023/3/31
// Version     : V 1.0
// Description  : 箱式滤波器平均值。
//
// Modification history
//-----

```

```

// $Log$
//
// *****

module box_ave (
    clk,
    rstn,
    sample,
    raw_data_in,
    ave_data_out,
    data_out_valid);

parameter
ADC_WIDTH = 8,           // ADC Convertor Bit Precision// ADC 精度, 单位 bit
LPF_DEPTH_BITS = 4;      // 2^LPF_DEPTH_BITS is decimation rate of averager// 低通滤波器
                          // 深度, 2^LPF_DEPTH_BITS 是抽取率

//input ports
input  clk;               // sample rate clock // 时钟
input  rstn;              // async reset, asserted low// 复位, 低电平有效
input  sample;            // raw_data_in is good on rising edge, // 输入有
                          // 效, 高电平
input  [ADC_WIDTH-1:0] raw_data_in; // raw_data input// 数据输入

//output ports
output [ADC_WIDTH-1:0]  ave_data_out; // ave data output// 滤波后输出
output data_out_valid; // ave_data_out is valid, single pulse// 输出数据有
                          // 效

reg [ADC_WIDTH-1:0] ave_data_out;
//*****
//内部连线, 寄存器声明
// Internal Wire & Reg Signals
//
//*****
reg [ADC_WIDTH+LPF_DEPTH_BITS-1:0] accum; // accumulator// 累加寄存器

```

```

reg [LPF_DEPTH_BITS-1:0]          count;          // decimation count// 计数器
reg [ADC_WIDTH-1:0]               raw_data_d1;      // pipeline register// 流水线寄存器

reg sample_d1, sample_d2;          // pipeline registers// 数据有效打拍
reg result_valid;                  // accumulator result 'valid'// 结果有效寄存器
wire accumulate;                   // sample rising edge detected
wire latch_result;                 // latch accumulator result// 锁存结果数据

//*****
//
// Rising Edge Detection and data alignment pipelines
//
//*****
always @(posedge clk or negedge rstn)
begin
    if( ~rstn ) begin
        sample_d1 <= 0;
        sample_d2 <= 0;
        raw_data_d1 <= 0;
        result_valid <= 0;
    end else begin
        sample_d1 <= sample;          // capture 'sample' input// 各种信号打拍
        sample_d2 <= sample_d1;       // delay for edge detection
        raw_data_d1 <= raw_data_in;    // pipeline
        result_valid <= latch_result;  // pipeline for alignment with result
    end
end

assign    accumulate = sample_d1 && !sample_d2;    // 'sample' rising_edge detect
assign    latch_result = accumulate && (count == 0); // latch accum. per decimation count//
计数器为 0 且新的数据到达，数据锁存指示

//*****
//累加时序控制

```

```

// Accumulator Depth counter
//
//*****
always @(posedge clk or negedge rstn)
begin
    if( ~rstn ) begin
        count <= 0;
    end else begin
        if (accumulate)    count <= count + 1;          // incr. count per each sample// 计数器
+1
    end
end

//*****
//数据累加
// Accumulator
//
//*****
always @(posedge clk or negedge rstn)
begin
    if( ~rstn ) begin
        accum <= 0;
    end else begin
        if (accumulate)
            if(count == 0)                // reset accumulator// 累加结束,
                accum <= raw_data_d1;      // prime with first value// 写入新数据
            else
                accum <= accum + raw_data_d1; // accumulate // 累加中
    end
end

//*****
////// 锁存结果
// 把累加器的值进行移位，代替除法
// Latch Result
//

```



```
// ave = (summation of 'n' samples)/'n' is right shift when 'n' is power of two
//
//*****
always @(posedge clk or negedge rstn)
begin
    if( ~rstn ) begin
        ave_data_out <= 0;
    end else if (latch_result) begin // at end of decimation period...// 如果锁存结果
        ave_data_out <= accum >> LPF_DEPTH_BITS; // ... save accumulator/n result// 右移
        LPF_DEPTH_BITS 位, 等效/(2^LPF_DEPTH_BITS)
    end
end

assign data_out_valid = result_valid; // output assignment// 输出数据有效信号

endmodule
```

(4) 设计文件 bintobcd.v

```
// *****
// File name : bintobcd
// Module name : bintobcd
//
// Author : STEP
// Date : 2023/3/31
// Version : V 1.0
// Description : 二进制转换为 bcd 码。
//
// Modification history
//-----
// $Log$
//
// *****

module bintobcd
(
input rst_n, //系统复位, 低有效
input [15:0] bin_code, //需要进行 BCD 转码的二进制数据
output reg [19:0] bcd_code //转码后的 BCD 码型数据输出
)
```

```

);

/*
此模块为了将 ADC 采样的数据转换为我们常用的十进制显示而存在，
主要知识涉及数学中不同制式数据的转换，详细原理这里不做介绍，去百度搜索<FPGA 二进制转
BCD 码>可得
*/

reg      [35:0]      shift_reg;
always@(bin_code or rst_n)begin
    shift_reg = {20'h0,bin_code};
    if(!rst_n) bcd_code = 0;
    else begin
        repeat(16) begin //循环 16 次
            //BCD 码各位数据作满 5 加 3 操作，
            if (shift_reg[19:16] >= 5) shift_reg[19:16] = shift_reg[19:16] + 2'b11;
            if (shift_reg[23:20] >= 5) shift_reg[23:20] = shift_reg[23:20] + 2'b11;
            if (shift_reg[27:24] >= 5) shift_reg[27:24] = shift_reg[27:24] + 2'b11;
            if (shift_reg[31:28] >= 5) shift_reg[31:28] = shift_reg[31:28] + 2'b11;
            if (shift_reg[35:32] >= 5) shift_reg[35:32] = shift_reg[35:32] + 2'b11;
            shift_reg = shift_reg << 1;
        end
        bcd_code = shift_reg[35:16];
    end
end
endmodule

```

(5) 设计文件 OLEDshow.v

```

// *****
// File name : OLEDshow
// Module name : OLEDshow
//
// Author : STEP
// Date : 2023/3/31
// Version : V 1.0
// Description : OLED12832_Driver, 使用 8*8 点阵字库，每行显示 128/8=16 个字符。
//

```

```

// Modification history
//-----
// $Log$
//
// *****

module OLEDshow
(
    input          clk,      //12MHz 系统时钟
    input          rst_n,    //系统复位，低有效

    input [7:0]     data,     //

    output reg      oled_csn, //OLED 液晶屏使能
    output reg      oled_rst, //OLED 液晶屏复位
    output reg      oled_dcn, //OLED 数据指令控制
    output reg      oled_clk, //OLED 时钟信号
    output reg      oled_dat //OLED 数据信号
);

    localparam INIT_DEPTH = 16'd23; //LCD 初始化的命令的数量
    localparam IDLE = 7'h1, MAIN = 7'h2, INIT = 7'h4, SCAN = 7'h8, WRITE = 7'h10, DELAY =
7'h20, CHINESE = 7'h40;
    localparam HIGH = 1'b1, LOW = 1'b0;
    localparam DATA = 1'b1, CMD = 1'b0;

    reg [7:0]        cmd [22:0];
    reg [39:0]       mem [122:0];
    reg [127:0]      mem_hanzi[16:0];

    reg [7:0]        mem_hanzi_num;
    reg [7:0]        y_p, x_ph, x_pl;
    reg [(8*21-1):0] char;
    reg [7:0]        num, char_reg;
    reg [4:0]        cnt_main, cnt_init, cnt_scan, cnt_write;

```

```

reg [5:0]          cnt_chinese;
reg [15:0]         num_delay, cnt_delay, cnt;
reg [6:0]          state, state_back;

always @ (posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        cnt_main <= 1'b0; cnt_init <= 1'b0; cnt_scan <= 1'b0; cnt_write <= 1'b0; cnt_chinese <= 1'b0;
        y_p <= 1'b0; x_ph <= 1'b0; x_pl <= 1'b0;
        num <= 1'b0; char <= 1'b0; char_reg <= 1'b0;
        num_delay <= 16'd5; cnt_delay <= 1'b0; cnt <= 1'b0;
        oled_csn <= HIGH; oled_rst <= HIGH; oled_dcn <= CMD; oled_clk <= HIGH; oled_dat <=
LOW;

        state <= IDLE; state_back <= IDLE;
    end else begin
        case(state)
            IDLE:begin
                cnt_main <= 1'b0; cnt_init <= 1'b0; cnt_scan <= 1'b0; cnt_write <= 1'b0;
                y_p <= 1'b0; x_ph <= 1'b0; x_pl <= 1'b0;
                num <= 1'b0; char <= 1'b0; char_reg <= 1'b0;
                num_delay <= 16'd5; cnt_delay <= 1'b0; cnt <= 1'b0; mem_hanzi_num <= 8'd0;
                oled_csn <= HIGH; oled_rst <= HIGH; oled_dcn <= CMD; oled_clk <= HIGH;
oled_dat <= LOW;

                state <= MAIN; state_back <= MAIN;
            end
            MAIN:begin
                if(cnt_main >= 5'd14) cnt_main <= 5'd13; //接下来执行空操作，实现数据只
刷新一次

                else cnt_main <= cnt_main + 1'b1;
                case(cnt_main)    //MAIN 状态
                    5'd0 :    begin state <= INIT; end

                    5'd1 :    begin y_p <= 8'hb0; x_ph <= 8'h10; x_pl <= 8'h00; num <=
5'd16; char <= "
"; state <= SCAN; end //y_p 是行， x_ph 是列起始地址高位
                    5'd2 :    begin y_p <= 8'hb1; x_ph <= 8'h10; x_pl <= 8'h00; num <=
5'd16; char <= "
"; state <= SCAN; end

```

```

5'd3:   begin y_p <= 8'hb2; x_ph <= 8'h10; x_pl <= 8'h00; num <=
5'd16; char <= "           ";state <= SCAN; end
5'd4:   begin y_p <= 8'hb3; x_ph <= 8'h10; x_pl <= 8'h00; num <=
5'd16; char <= "voltage:  ";state <= SCAN; end
5'd5:   begin y_p <= 8'hb4; x_ph <= 8'h10; x_pl <= 8'h00; num <=
5'd16; char <= "           ";state <= SCAN; end
5'd6:   begin y_p <= 8'hb5; x_ph <= 8'h10; x_pl <= 8'h00; num <=
5'd16; char <= "         . V ";state <= SCAN; end
5'd7:   begin y_p <= 8'hb6; x_ph <= 8'h10; x_pl <= 8'h00; num <=
5'd16; char <= "           ";state <= SCAN; end
5'd8:   begin y_p <= 8'hb7; x_ph <= 8'h10; x_pl <= 8'h00; num <=
5'd16; char <= "           ";state <= SCAN; end
5'd9:   begin y_p <= 8'hb0; x_ph <= 8'h12; x_pl <= 8'h00;
mem_hanzi_num <= 8'd0; state <= CHINESE; end
5'd10:  begin y_p <= 8'hb0; x_ph <= 8'h13; x_pl <= 8'h00;
mem_hanzi_num <= 8'd2; state <= CHINESE; end
5'd11:  begin y_p <= 8'hb0; x_ph <= 8'h14; x_pl <= 8'h00;
mem_hanzi_num <= 8'd4; state <= CHINESE; end
5'd12:  begin y_p <= 8'hb0; x_ph <= 8'h15; x_pl <= 8'h00;
mem_hanzi_num <= 8'd6; state <= CHINESE; end

5'd13:  begin y_p <= 8'hb5; x_ph <= 8'h13; x_pl <= 8'h00; num <= 5'd1;
char <=data[7:4];state <= SCAN; end
5'd14:  begin y_p <= 8'hb5; x_ph <= 8'h14; x_pl <= 8'h00; num <= 5'd1;
char <=data[3:0];state <= SCAN; end

//default: state <= IDLE; //如果你需要动态刷新一些信息，此行应该
取消注释

endcase
end
INIT:begin //初始化状态
case(cnt_init)
5'd0:   begin oled_rst <= LOW; cnt_init <= cnt_init + 1'b1; end // 复
位有效
5'd1:   begin num_delay <= 16'd25000; state <= DELAY; state_back <=
INIT; cnt_init <= cnt_init + 1'b1; end //延时大于 3us

```

```

5'd2:    begin oled_rst <= HIGH; cnt_init <= cnt_init + 1'b1; end // 复
位恢复

5'd3:    begin num_delay <= 16'd25000; state <= DELAY; state_back <=
INIT; cnt_init <= cnt_init + 1'b1; end //延时大于 220us

5'd4:    begin
            if(cnt>=INIT_DEPTH) begin //当 25 条指令及数据发出
后，配置完成

                cnt <= 1'b0;
                cnt_init <= cnt_init + 1'b1;
            end else begin
                cnt <= cnt + 1'b1; num_delay <= 16'd5;
                oled_dcn <= CMD; char_reg <= cmd[cnt]; state <=
WRITE; state_back <= INIT;

            end
        end
5'd5:    begin cnt_init <= 1'b0; state <= MAIN; end //初始化完成，返回
MAIN 状态

        default: state <= IDLE;
    endcase
end
SCAN:begin //刷屏状态，从 RAM 中读取数据刷屏
    if(cnt_scan == 5'd11) begin
        if(num) cnt_scan <= 5'd3;
        else cnt_scan <= cnt_scan + 1'b1;
    end
    else if(cnt_scan == 5'd12) cnt_scan <= 1'b0;
    else cnt_scan <= cnt_scan + 1'b1;
    case(cnt_scan)
        5'd 0:    begin oled_dcn <= CMD; char_reg <= y_p; state <= WRITE;
state_back <= SCAN; end //定位列页地址
        5'd 1:    begin oled_dcn <= CMD; char_reg <= x_pl; state <= WRITE;
state_back <= SCAN; end //定位行地址低位
        5'd 2:    begin oled_dcn <= CMD; char_reg <= x_ph; state <= WRITE;
state_back <= SCAN; end //定位行地址高位

        5'd 3:    begin num <= num - 1'b1;end
        5'd 4:    begin oled_dcn <= DATA; char_reg <= 8'h00; state <= WRITE;

```

```

state_back <= SCAN; end    //将 5*8 点阵编程 8*8
                    5'd 5:  begin oled_dcn <= DATA; char_reg <= 8'h00; state <= WRITE;
state_back <= SCAN; end    //将 5*8 点阵编程 8*8
                    5'd 6:  begin oled_dcn <= DATA; char_reg <= 8'h00; state <= WRITE;
state_back <= SCAN; end    //将 5*8 点阵编程 8*8
                    5'd 7:  begin      oled_dcn      <=      DATA;      char_reg      <=
mem[char[(num*8)+:8]][39:32]; state <= WRITE; state_back <= SCAN; end
                    5'd 8:  begin      oled_dcn      <=      DATA;      char_reg      <=
mem[char[(num*8)+:8]][31:24]; state <= WRITE; state_back <= SCAN; end
                    5'd 9:  begin      oled_dcn      <=      DATA;      char_reg      <=
mem[char[(num*8)+:8]][23:16]; state <= WRITE; state_back <= SCAN; end
                    5'd10: begin      oled_dcn      <=      DATA;      char_reg      <=
mem[char[(num*8)+:8]][15: 8]; state <= WRITE; state_back <= SCAN; end
                    5'd11: begin      oled_dcn      <=      DATA;      char_reg      <=
mem[char[(num*8)+:8]][ 7: 0]; state <= WRITE; state_back <= SCAN; end
                    5'd12: begin state <= MAIN; end
                        default: state <= IDLE;
                    endcase
                end
            end

        CHINESE:begin    //显示汉字

                        if(cnt_chinese == 6'd38) cnt_chinese <= 1'b0;
                        else cnt_chinese <= cnt_chinese+1'b1;
                        case(cnt_chinese)
                            6'd 0:  begin oled_dcn <= CMD; char_reg <= y_p; state <= WRITE;
state_back <= CHINESE; end    //定位列页地址
                            6'd 1:  begin oled_dcn <= CMD; char_reg <= x_pl; state <= WRITE;
state_back <= CHINESE; end//定位行地址低位
                            6'd 2:  begin oled_dcn <= CMD; char_reg <= x_ph; state <= WRITE;
state_back <= CHINESE; end//定位行地址高位

                            6'd3 :  begin      oled_dcn      <=      DATA;      char_reg      <=
mem_hanzi[mem_hanzi_num][127:120]; state <= WRITE; state_back <= CHINESE; end

```

```

        6'd4:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num][119:112]; state <= WRITE; state_back <= CHINESE; end

        6'd5:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num][111:104]; state <= WRITE; state_back <= CHINESE; end

        6'd6:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num][103:96] ; state <= WRITE; state_back <= CHINESE; end

        6'd7:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num][95:88]  ; state <= WRITE; state_back <= CHINESE; end

        6'd8:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num][87:80]  ; state <= WRITE; state_back <= CHINESE; end

        6'd9:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num][79:72]  ; state <= WRITE; state_back <= CHINESE; end

        6'd10: begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num][71:64]  ; state <= WRITE; state_back <= CHINESE; end

        6'd11: begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num][63:56];  state <= WRITE; state_back <= CHINESE; end

        6'd12: begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num][55:48];  state <= WRITE; state_back <= CHINESE; end

        6'd13: begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num][47:40];  state <= WRITE; state_back <= CHINESE; end

        6'd14: begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num][39:32];  state <= WRITE; state_back <= CHINESE; end

        6'd15: begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num][31:24];  state <= WRITE; state_back <= CHINESE; end

        6'd16: begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num][23:16];  state <= WRITE; state_back <= CHINESE; end

        6'd17: begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num][15: 8];  state <= WRITE; state_back <= CHINESE; end

        6'd18: begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num][ 7: 0];  state <= WRITE; state_back <= CHINESE; end


        6'd19: begin oled_dcn <= CMD; char_reg <= y_p+1; state <= WRITE;
state_back <= CHINESE; end //定位列页地址

        6'd20: begin oled_dcn <= CMD; char_reg <= x_pl; state <= WRITE;
state_back <= CHINESE; end//定位行地址低位

        6'd21: begin oled_dcn <= CMD; char_reg <= x_ph; state <= WRITE;
state_back <= CHINESE; end//定位行地址高位

```



```

        6'd22:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num+1][127:120]; state <= WRITE; state_back <= CHINESE; end
        6'd23:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num+1][119:112]; state <= WRITE; state_back <= CHINESE; end
        6'd24:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num+1][111:104]; state <= WRITE; state_back <= CHINESE; end
        6'd25:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num+1][103:96] ; state <= WRITE; state_back <= CHINESE; end
        6'd26:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num+1][95:88]  ; state <= WRITE; state_back <= CHINESE; end
        6'd27:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num+1][87:80]  ; state <= WRITE; state_back <= CHINESE; end
        6'd28:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num+1][79:72]  ; state <= WRITE; state_back <= CHINESE; end
        6'd29:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num+1][71:64]  ; state <= WRITE; state_back <= CHINESE; end
        6'd30:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num+1][63:56];  state <= WRITE; state_back <= CHINESE; end
        6'd31:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num+1][55:48];  state <= WRITE; state_back <= CHINESE; end
        6'd32:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num+1][47:40];  state <= WRITE; state_back <= CHINESE; end
        6'd33:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num+1][39:32];  state <= WRITE; state_back <= CHINESE; end
        6'd34:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num+1][31:24];  state <= WRITE; state_back <= CHINESE; end
        6'd35:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num+1][23:16];  state <= WRITE; state_back <= CHINESE; end
        6'd36:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num+1][15: 8];  state <= WRITE; state_back <= CHINESE; end
        6'd37:  begin    oled_dcn    <=    DATA;    char_reg    <=
mem_hanzi[mem_hanzi_num+1][ 7: 0];  state <= WRITE; state_back <= CHINESE; end

        6'd38:  begin state <= MAIN; end
        default: state <= IDLE;
    endcase
end

```

位

发高位数据

发低位数据

```

WRITE:begin //WRITE 状态，将数据按照 SPI 时序发送给屏幕
    if(cnt_write >= 5'd17) cnt_write <= 1'b0;
    else cnt_write <= cnt_write + 1'b1;
    case(cnt_write)
        5'd 0:    begin oled_csn <= LOW; end // 9 位数据最高位为命令数据控制
        5'd 1:    begin oled_clk <= LOW; oled_dat <= char_reg[7]; end    // 先
        5'd 2:    begin oled_clk <= HIGH; end
        5'd 3:    begin oled_clk <= LOW; oled_dat <= char_reg[6]; end
        5'd 4:    begin oled_clk <= HIGH; end
        5'd 5:    begin oled_clk <= LOW; oled_dat <= char_reg[5]; end
        5'd 6:    begin oled_clk <= HIGH; end
        5'd 7:    begin oled_clk <= LOW; oled_dat <= char_reg[4]; end
        5'd 8:    begin oled_clk <= HIGH; end
        5'd 9:    begin oled_clk <= LOW; oled_dat <= char_reg[3]; end
        5'd10:    begin oled_clk <= HIGH; end
        5'd11:    begin oled_clk <= LOW; oled_dat <= char_reg[2]; end
        5'd12:    begin oled_clk <= HIGH; end
        5'd13:    begin oled_clk <= LOW; oled_dat <= char_reg[1]; end
        5'd14:    begin oled_clk <= HIGH; end
        5'd15:    begin oled_clk <= LOW; oled_dat <= char_reg[0]; end    // 后
        5'd16:    begin oled_clk <= HIGH; end
        5'd17:    begin oled_csn <= HIGH; state <= DELAY; end    //
        default: state <= IDLE;
    endcase
end
DELAY:begin //延时状态
    if(cnt_delay >= num_delay) begin
        cnt_delay <= 16'd0; state <= state_back;
    end else cnt_delay <= cnt_delay + 1'b1;
end
default:state <= IDLE;

```

```
        endcase
    end
end

//OLED 配置指令数据
always@(posedge rst_n)
    begin
        cmd[0 ] = {8'hae};
        cmd[1 ] = {8'hd5};
        cmd[2 ] = {8'h80};
        cmd[3 ] = {8'ha8};
        cmd[4 ] = {8'h3f};
        cmd[5 ] = {8'hd3};
        cmd[6 ] = {8'h00};
        cmd[7 ] = {8'h40};
        cmd[8 ] = {8'h8d};
        cmd[9 ] = {8'h14};
        cmd[10] = {8'h20};
        cmd[11] = {8'h02};
        cmd[12] = {8'hc8};
        cmd[13] = {8'ha1};
        cmd[14] = {8'hda};
        cmd[15] = {8'h12};
        cmd[16] = {8'h81};
        cmd[17] = {8'hcf};
        cmd[18] = {8'hd9};
        cmd[19] = {8'hf1};
        cmd[20] = {8'hdb};
        cmd[21] = {8'h40};
        cmd[22] = {8'haf};

    end

//5*8 点阵字库数据
always@(posedge rst_n)
    begin
        mem[ 0] = {8'h3E, 8'h51, 8'h49, 8'h45, 8'h3E};    // 48  0
```

```

mem[ 1] = {8'h00, 8'h42, 8'h7F, 8'h40, 8'h00}; // 49 1
mem[ 2] = {8'h42, 8'h61, 8'h51, 8'h49, 8'h46}; // 50 2
mem[ 3] = {8'h21, 8'h41, 8'h45, 8'h4B, 8'h31}; // 51 3
mem[ 4] = {8'h18, 8'h14, 8'h12, 8'h7F, 8'h10}; // 52 4
mem[ 5] = {8'h27, 8'h45, 8'h45, 8'h45, 8'h39}; // 53 5
mem[ 6] = {8'h3C, 8'h4A, 8'h49, 8'h49, 8'h30}; // 54 6
mem[ 7] = {8'h01, 8'h71, 8'h09, 8'h05, 8'h03}; // 55 7
mem[ 8] = {8'h36, 8'h49, 8'h49, 8'h49, 8'h36}; // 56 8
mem[ 9] = {8'h06, 8'h49, 8'h49, 8'h29, 8'h1E}; // 57 9
mem[10] = {8'h7C, 8'h12, 8'h11, 8'h12, 8'h7C}; // 65 A
mem[11] = {8'h7F, 8'h49, 8'h49, 8'h49, 8'h36}; // 66 B
mem[12] = {8'h3E, 8'h41, 8'h41, 8'h41, 8'h22}; // 67 C
mem[13] = {8'h7F, 8'h41, 8'h41, 8'h22, 8'h1C}; // 68 D
mem[14] = {8'h7F, 8'h49, 8'h49, 8'h49, 8'h41}; // 69 E
mem[15] = {8'h7F, 8'h09, 8'h09, 8'h09, 8'h01}; // 70 F

mem[32] = {8'h00, 8'h00, 8'h00, 8'h00, 8'h00}; // 32 sp
mem[33] = {8'h00, 8'h00, 8'h2f, 8'h00, 8'h00}; // 33 !
mem[34] = {8'h00, 8'h07, 8'h00, 8'h07, 8'h00}; // 34
mem[35] = {8'h14, 8'h7f, 8'h14, 8'h7f, 8'h14}; // 35 #
mem[36] = {8'h24, 8'h2a, 8'h7f, 8'h2a, 8'h12}; // 36 $
mem[37] = {8'h62, 8'h64, 8'h08, 8'h13, 8'h23}; // 37 %
mem[38] = {8'h36, 8'h49, 8'h55, 8'h22, 8'h50}; // 38 &
mem[39] = {8'h00, 8'h05, 8'h03, 8'h00, 8'h00}; // 39 '
mem[40] = {8'h00, 8'h1c, 8'h22, 8'h41, 8'h00}; // 40 (
mem[41] = {8'h00, 8'h41, 8'h22, 8'h1c, 8'h00}; // 41 )
mem[42] = {8'h14, 8'h08, 8'h3E, 8'h08, 8'h14}; // 42 *
mem[43] = {8'h08, 8'h08, 8'h3E, 8'h08, 8'h08}; // 43 +
mem[44] = {8'h00, 8'h00, 8'hA0, 8'h60, 8'h00}; // 44 ,
mem[45] = {8'h08, 8'h08, 8'h08, 8'h08, 8'h08}; // 45 -
mem[46] = {8'h00, 8'h60, 8'h60, 8'h00, 8'h00}; // 46 .
mem[47] = {8'h20, 8'h10, 8'h08, 8'h04, 8'h02}; // 47 /
mem[48] = {8'h3E, 8'h51, 8'h49, 8'h45, 8'h3E}; // 48 0
mem[49] = {8'h00, 8'h42, 8'h7F, 8'h40, 8'h00}; // 49 1
mem[50] = {8'h42, 8'h61, 8'h51, 8'h49, 8'h46}; // 50 2
mem[51] = {8'h21, 8'h41, 8'h45, 8'h4B, 8'h31}; // 51 3
mem[52] = {8'h18, 8'h14, 8'h12, 8'h7F, 8'h10}; // 52 4

```

```

mem[ 53] = {8'h27, 8'h45, 8'h45, 8'h45, 8'h39}; // 53 5
mem[ 54] = {8'h3C, 8'h4A, 8'h49, 8'h49, 8'h30}; // 54 6
mem[ 55] = {8'h01, 8'h71, 8'h09, 8'h05, 8'h03}; // 55 7
mem[ 56] = {8'h36, 8'h49, 8'h49, 8'h49, 8'h36}; // 56 8
mem[ 57] = {8'h06, 8'h49, 8'h49, 8'h29, 8'h1E}; // 57 9
mem[ 58] = {8'h00, 8'h36, 8'h36, 8'h00, 8'h00}; // 58 :
mem[ 59] = {8'h00, 8'h56, 8'h36, 8'h00, 8'h00}; // 59 ;
mem[ 60] = {8'h08, 8'h14, 8'h22, 8'h41, 8'h00}; // 60 <
mem[ 61] = {8'h14, 8'h14, 8'h14, 8'h14, 8'h14}; // 61 =
mem[ 62] = {8'h00, 8'h41, 8'h22, 8'h14, 8'h08}; // 62 >
mem[ 63] = {8'h02, 8'h01, 8'h51, 8'h09, 8'h06}; // 63 ?
mem[ 64] = {8'h32, 8'h49, 8'h59, 8'h51, 8'h3E}; // 64 @
mem[ 65] = {8'h7C, 8'h12, 8'h11, 8'h12, 8'h7C}; // 65 A
mem[ 66] = {8'h7F, 8'h49, 8'h49, 8'h49, 8'h36}; // 66 B
mem[ 67] = {8'h3E, 8'h41, 8'h41, 8'h41, 8'h22}; // 67 C
mem[ 68] = {8'h7F, 8'h41, 8'h41, 8'h22, 8'h1C}; // 68 D
mem[ 69] = {8'h7F, 8'h49, 8'h49, 8'h49, 8'h41}; // 69 E
mem[ 70] = {8'h7F, 8'h09, 8'h09, 8'h09, 8'h01}; // 70 F
mem[ 71] = {8'h3E, 8'h41, 8'h49, 8'h49, 8'h7A}; // 71 G
mem[ 72] = {8'h7F, 8'h08, 8'h08, 8'h08, 8'h7F}; // 72 H
mem[ 73] = {8'h00, 8'h41, 8'h7F, 8'h41, 8'h00}; // 73 I
mem[ 74] = {8'h20, 8'h40, 8'h41, 8'h3F, 8'h01}; // 74 J
mem[ 75] = {8'h7F, 8'h08, 8'h14, 8'h22, 8'h41}; // 75 K
mem[ 76] = {8'h7F, 8'h40, 8'h40, 8'h40, 8'h40}; // 76 L
mem[ 77] = {8'h7F, 8'h02, 8'h0C, 8'h02, 8'h7F}; // 77 M
mem[ 78] = {8'h7F, 8'h04, 8'h08, 8'h10, 8'h7F}; // 78 N
mem[ 79] = {8'h3E, 8'h41, 8'h41, 8'h41, 8'h3E}; // 79 O
mem[ 80] = {8'h7F, 8'h09, 8'h09, 8'h09, 8'h06}; // 80 P
mem[ 81] = {8'h3E, 8'h41, 8'h51, 8'h21, 8'h5E}; // 81 Q
mem[ 82] = {8'h7F, 8'h09, 8'h19, 8'h29, 8'h46}; // 82 R
mem[ 83] = {8'h46, 8'h49, 8'h49, 8'h49, 8'h31}; // 83 S
mem[ 84] = {8'h01, 8'h01, 8'h7F, 8'h01, 8'h01}; // 84 T
mem[ 85] = {8'h3F, 8'h40, 8'h40, 8'h40, 8'h3F}; // 85 U
mem[ 86] = {8'h1F, 8'h20, 8'h40, 8'h20, 8'h1F}; // 86 V
mem[ 87] = {8'h3F, 8'h40, 8'h38, 8'h40, 8'h3F}; // 87 W
mem[ 88] = {8'h63, 8'h14, 8'h08, 8'h14, 8'h63}; // 88 X
mem[ 89] = {8'h07, 8'h08, 8'h70, 8'h08, 8'h07}; // 89 Y

```

```

mem[ 90] = {8'h61, 8'h51, 8'h49, 8'h45, 8'h43}; // 90  Z
mem[ 91] = {8'h00, 8'h7F, 8'h41, 8'h41, 8'h00}; // 91  [
mem[ 92] = {8'h55, 8'h2A, 8'h55, 8'h2A, 8'h55}; // 92  .
mem[ 93] = {8'h00, 8'h41, 8'h41, 8'h7F, 8'h00}; // 93  ]
mem[ 94] = {8'h04, 8'h02, 8'h01, 8'h02, 8'h04}; // 94  ^
mem[ 95] = {8'h40, 8'h40, 8'h40, 8'h40, 8'h40}; // 95  _
mem[ 96] = {8'h00, 8'h01, 8'h02, 8'h04, 8'h00}; // 96  '
mem[ 97] = {8'h20, 8'h54, 8'h54, 8'h54, 8'h78}; // 97  a
mem[ 98] = {8'h7F, 8'h48, 8'h44, 8'h44, 8'h38}; // 98  b
mem[ 99] = {8'h38, 8'h44, 8'h44, 8'h44, 8'h20}; // 99  c
mem[100] = {8'h38, 8'h44, 8'h44, 8'h48, 8'h7F}; // 100 d
mem[101] = {8'h38, 8'h54, 8'h54, 8'h54, 8'h18}; // 101 e
mem[102] = {8'h08, 8'h7E, 8'h09, 8'h01, 8'h02}; // 102 f
mem[103] = {8'h18, 8'hA4, 8'hA4, 8'hA4, 8'h7C}; // 103 g
mem[104] = {8'h7F, 8'h08, 8'h04, 8'h04, 8'h78}; // 104 h
mem[105] = {8'h00, 8'h44, 8'h7D, 8'h40, 8'h00}; // 105 i
mem[106] = {8'h40, 8'h80, 8'h84, 8'h7D, 8'h00}; // 106 j
mem[107] = {8'h7F, 8'h10, 8'h28, 8'h44, 8'h00}; // 107 k
mem[108] = {8'h00, 8'h41, 8'h7F, 8'h40, 8'h00}; // 108 l
mem[109] = {8'h7C, 8'h04, 8'h18, 8'h04, 8'h78}; // 109 m
mem[110] = {8'h7C, 8'h08, 8'h04, 8'h04, 8'h78}; // 110 n
mem[111] = {8'h38, 8'h44, 8'h44, 8'h44, 8'h38}; // 111 o
mem[112] = {8'hFC, 8'h24, 8'h24, 8'h24, 8'h18}; // 112 p
mem[113] = {8'h18, 8'h24, 8'h24, 8'h18, 8'hFC}; // 113 q
mem[114] = {8'h7C, 8'h08, 8'h04, 8'h04, 8'h08}; // 114 r
mem[115] = {8'h48, 8'h54, 8'h54, 8'h54, 8'h20}; // 115 s
mem[116] = {8'h04, 8'h3F, 8'h44, 8'h40, 8'h20}; // 116 t
mem[117] = {8'h3C, 8'h40, 8'h40, 8'h20, 8'h7C}; // 117 u
mem[118] = {8'h1C, 8'h20, 8'h40, 8'h20, 8'h1C}; // 118 v
mem[119] = {8'h3C, 8'h40, 8'h30, 8'h40, 8'h3C}; // 119 w
mem[120] = {8'h44, 8'h28, 8'h10, 8'h28, 8'h44}; // 120 x
mem[121] = {8'h1C, 8'hA0, 8'hA0, 8'hA0, 8'h7C}; // 121 y
mem[122] = {8'h44, 8'h64, 8'h54, 8'h4C, 8'h44}; // 122 z

end

```

```

//16*16 汉字
always@(posedge rst_n)
begin
    mem_hanzi[0] =
{8'h00,8'h00,8'h00,8'hB0,8'hF0,8'hB0,8'h90,8'h90,8'h74,8'hD4,8'hFC,8'hBC,8'h96,8'hF2,8'h30,8'h00};
    mem_hanzi[1] =
{8'h00,8'h02,8'h01,8'h01,8'h03,8'h02,8'h0B,8'h09,8'h0D,8'h05,8'h07,8'h04,8'h04,8'h18,8'h18,8'h18};// "
硬",0
    mem_hanzi[2] =
{8'h00,8'h00,8'h00,8'h00,8'hC0,8'h48,8'h48,8'hEC,8'hFC,8'hFE,8'hB2,8'h32,8'h30,8'h20,8'h00,8'h00};
    mem_hanzi[3] =
{8'h00,8'h08,8'h08,8'h0C,8'h04,8'h02,8'h11,8'h10,8'h3F,8'h3F,8'h00,8'h01,8'h02,8'h06,8'h0C,8'h0C};// "
禾",1
    mem_hanzi[4] =
{8'h00,8'h00,8'h80,8'hC0,8'h00,8'h2C,8'hD8,8'h50,8'h56,8'hBA,8'h6C,8'h4E,8'h4B,8'h50,8'h30,8'h20};
    mem_hanzi[5] =
{8'h00,8'h00,8'h00,8'h00,8'h04,8'h02,8'h02,8'h22,8'h22,8'h3F,8'h0F,8'h01,8'h01,8'h01,8'h01,8'h00};// "
学",2
    mem_hanzi[6] =
{8'h00,8'h00,8'h00,8'h80,8'h40,8'h20,8'hA8,8'hA8,8'h70,8'h5F,8'h58,8'hD4,8'h94,8'h90,8'hE0,8'hC0};
    mem_hanzi[7] =
{8'h00,8'h00,8'h00,8'h00,8'h20,8'h20,8'h29,8'h15,8'h1F,8'h1F,8'h17,8'h15,8'h10,8'h10,8'h00,8'h00};// "
堂",3

    //此处可以导入需要的字模

end

/*汉字：硬禾学堂
以汉字“硬禾学堂”RAM生成步骤为例：
1、阴码、列行式、逆向生成字模
2、然后把生成的字模作为RAM的元素
*/
endmodule

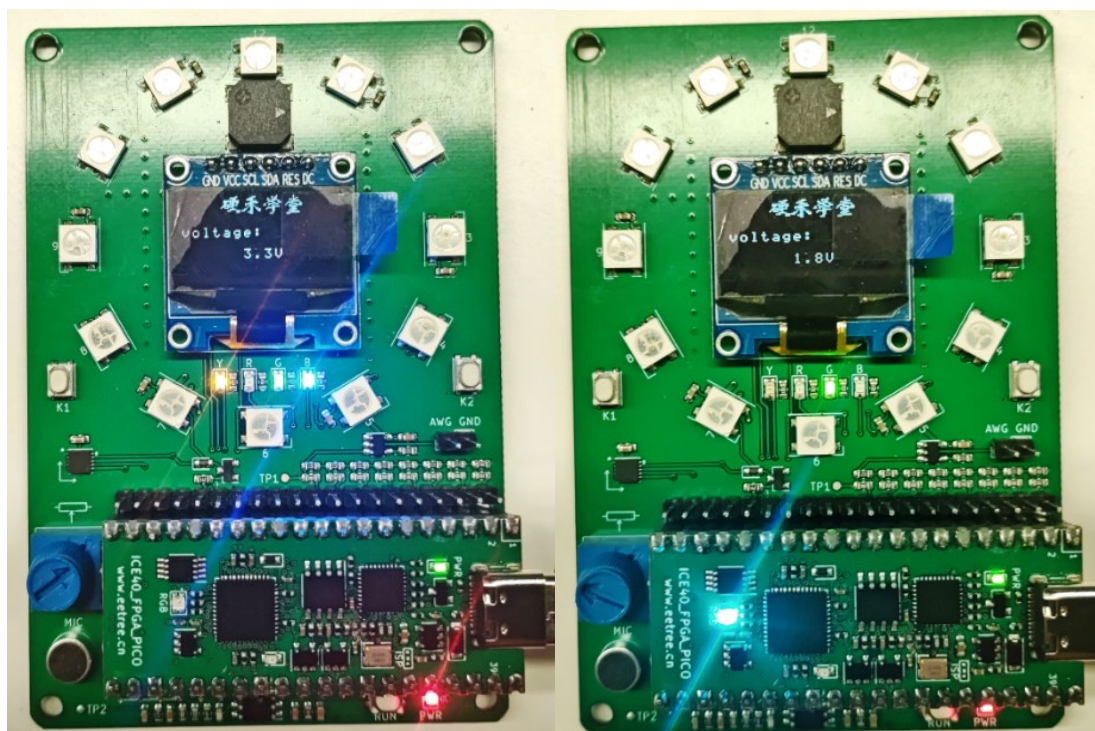
```

5.1.4 实验步骤

1. 打开 Ridian, 建立工程。
2. 新建 Verilog HDL 设计文件, 并键入设计代码, 包括添加需要调用的模块文件。
3. 构建并输出编程文件, 烧写至 FPGA 的 Flash 之中。

5.1.5 实验现象

实验结果如下:



第六章 姿态传感器实验

实验简介

本教程以实验为主，结合相应的实验案例，以详细的讲解和实验步骤让大家尽快熟悉 FPGA 开发的基本流程、Ridiant 软件平台的使用方法以及本节实验相关模块的工作原理。

- 熟悉和掌握 FPGA 开发的基本流程
- 掌握 Ridiant 软件平台的基本使用方法
- 掌握 IIC 读写数据的原理
- 掌握使用 Verilog HDL 语言驱动 MMA7660 姿态传感器

背景知识

MMA7660FC 是具有数字输出的 IIC、低功耗、紧凑型电容式微机械加速度传感器，提供低通滤波器、零重力加速度偏移和增益误差补偿，并可以转化为 6 位数字值，用户可配置输出数据的传输速率。该器件可通过中断引脚(INT)识别传感器的数据变化、产品的朝向和姿态等。MMA7660FC 采用非常小的 3 mm x 3 mm x 0.9 mm DFN 封装。完全兼容 IIC 总线，管脚功能描述如下。

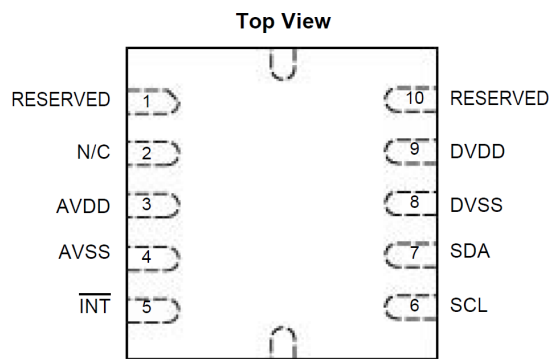


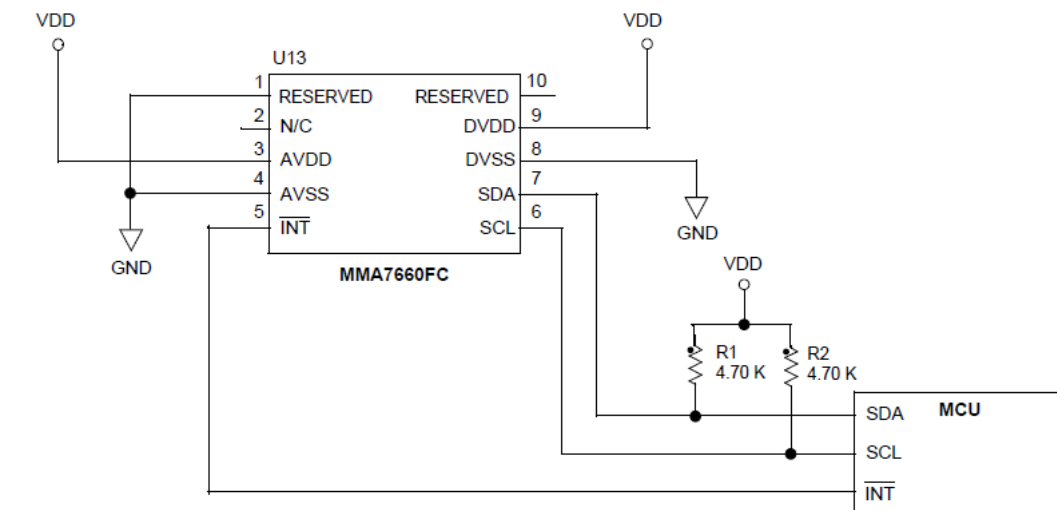
Figure 1. Pinout

Table 1. Pin Description

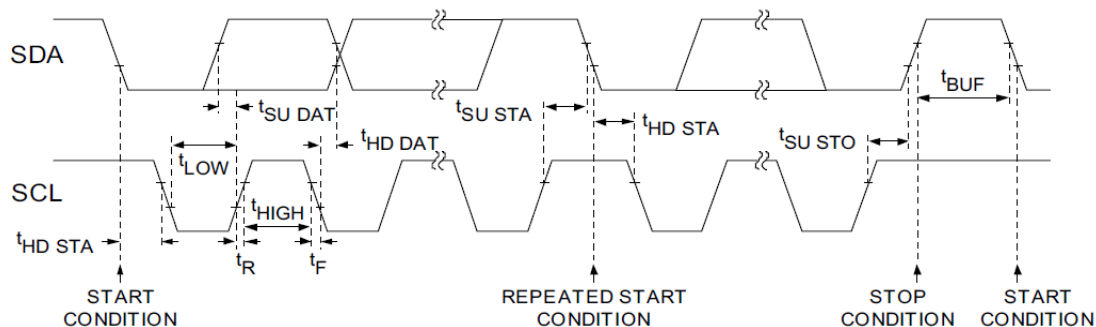
Pin #	Pin Name	Description	Pin Status
1	RESERVED	Connect to AVSS	Input
2	N/C	No Internal Connection, leave unconnected or connect to Ground	Input
3	AVDD	Device Power	Input
4	AVSS	Device Ground	Input
5	INT	Interrupt/Data Ready	Output
6	SCL	I ² C Serial Clock	Input
7	SDA	I ² C Serial Data	Open Drain
8	DVSS	Digital I/O Ground	Input
9	DVDD	Digital I/O Power	Input
10	RESERVED	Connect to AVSS	Input

管脚功能描述

MMA7660 芯片典型电路连接如下：



典型电路连接



MMA7660 时序图

Parameter	Symbol	Min	Typ	Max	Units
Serial Clock Frequency ⁽¹⁾	f_{SCL}			400	kHz
Bus Free Time Between a STOP and a START Condition ⁽²⁾	t_{BUF}	1.3			μs
Hold Time, (Repeated) START Condition ⁽²⁾	$t_{HD, STA}$	0.6			μs
Repeated START Condition Setup Time ⁽²⁾	$t_{SU, STA}$	0.6			μs
STOP Condition Setup Time ⁽²⁾	$t_{SU, STO}$	0.6			μs
Data Hold Time ⁽²⁾	$t_{HD, DAT}$			0.9	μs
Data Setup Time ⁽²⁾	$t_{SU, DAT}$	100			ns
SCL Clock Low Period ⁽²⁾	t_{LOW}	1.3			μs
SCL Clock High Period ⁽²⁾	t_{HIGH}	0.7			μs
Rise Time of Both SDA and SCL Signals, Receiving ⁽²⁾	t_R		$20+0.1C_b$	300	ns
Fall Time of Both SDA and SCL Signals, Receiving ⁽²⁾	t_F		$20+0.1C_b$	300	ns
Fall Time of SDA Transmitting ⁽²⁾	$t_{F, TX}$		$20+0.1C_b$	250	ns
Pulse Width of Spike Suppressed ⁽²⁾	t_{SP}		25		ns
Capacitive Load for Each Bus Line ⁽²⁾	C_b			400	pF

MMA7660 时序参数

通过 MMA7660 时序参数了解，MMA7660 最高支持 IIC 通信 400KHz。

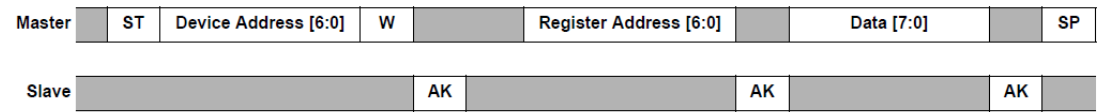
- 分频得到 400KHz 的时钟，程序实现同智能接近系统设计实验。

IIC 时序基本单元（启动、停止、发送、接收、发应答、读应答）协议里统一的，所以基本单元状态的设计也是不需要调整的。

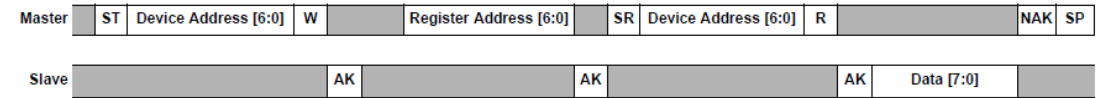
- 启动时序状态设计程序实现同智能接近系统设计实验。
- 发送单元和读应答单元合并，时序状态设计程序实现同智能接近系统设计实验。
- 接收单元和写应答单元合并，时序状态设计程序实现同智能接近系统设计实验。
- 停止时序状态设计程序实现同智能接近系统设计实验。
- MMA7660 出厂芯片器件地址为 7 位 0x4C

对于写操作而言，分为两种，单字节写和多字节写，本实验采用了单字节写；对于读操作而言，则有单字节读和多字节读两种，本实验采用了单字节读方式完成。

单字节写操作时序流程如下：



单字节写



单字节读

实验环境

- 硬件环境：STEP FPGA 实验平台
- 软件环境：Ridiant 开发环境

6.1 实验六 姿态传感器实验

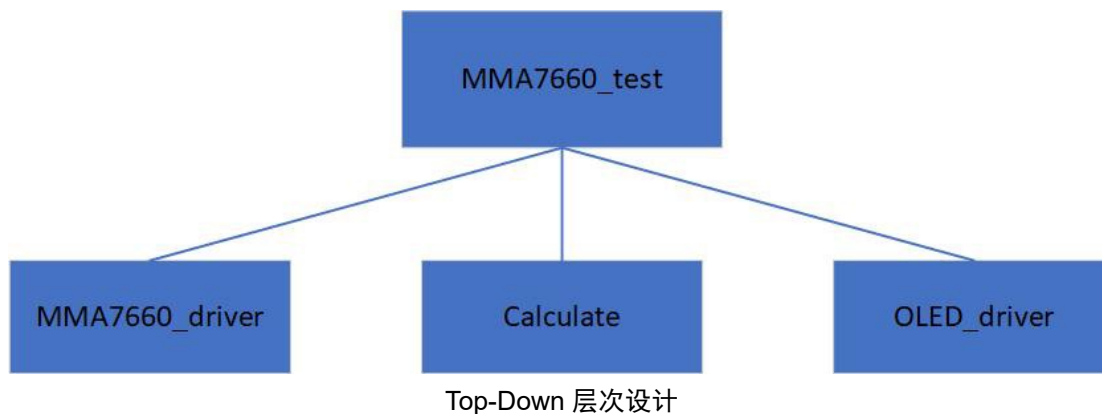
6.1.1 实验任务

基于 FPGA 实现 MMA7660 的驱动，并且将实验结果显示到 OLED 上。

6.1.2 实验原理

根据前面的实验解析我们可以得知，该设计可以拆分成三个功能模块实现，

- MMA7660_driver：姿态传感器 MMA7660 I2C 总线通信驱动模块。
- Calculate：将得到的二进制数码转换成 BCD 码的计算模块。
- OLED_driver：将得到的数据在 OLED 显示屏上显示。



5.1.3 Verilog HDL 建模描述

(1) 设计文件 MMA7660_test.v

```

// *****
// File name : MMA7660_test
// Module name : MMA7660_test
//
// Author      : STEP
// Date       : 2023.4.19
// Version    : V 1.0
// Description : MMA7660 instance module
//
  
```

```

// Modification history
// //
// $Log$
//
// *****_
// *****
//  DEFINE MODULE PORT  //
// *****
module MMA7660_test
(
// INPUT
input          rst_n,    //系统复位，低有效
// OUTPUT
output         i2c_scl,  //I2C 时钟总线
inout          i2c_sda,  //I2C 数据总线
output         oled_rst, //
output         oled_dcn, //
output         oled_clk, //
output         oled_dat
);

//WIRES
wire           [7:0]    xout, yout, zout, tilt, srst, x_data_bcd, y_data_bcd, z_data_bcd;
wire clk;
// *****
// INSTANCE MODULE  //
// *****
HSOSC #(
    .CLKHF_DIV ("0b10")
    ) OSCInst0 (
    .CLKHFEN (1'b1   ),
    .CLKHFPU (1'b1   ),
    .CLKHF   (clk)
    );

MMA7660_driver U_MMA7660_driver

```

```
(
.clk      (clk      ), //系统时钟
.rst_n    (rst_n    ), //系统复位，低有效
.i2c_scl  (i2c_scl  ), //I2C 总线 SCL
.i2c_sda  (i2c_sda  ), //I2C 总线 SDA
.xout     (xout     ), //读取数据
.yout     (yout     ), //读取数据
.zout     (zout     )  //读取数据
);
```

Calculate U_Calculate

```
(
.rst_n      (rst_n),
.xout       (xout),
.yout       (yout),
.zout       (zout),
.x_data_bcd (x_data_bcd),
.y_data_bcd (y_data_bcd),
.z_data_bcd (z_data_bcd)
);
```

OLED_driver U_OLED_driver

```
(
.clk      (clk      ), //系统时钟
.rst_n    (rst_n    ), //系统复位，低有效
.x_data_bcd (x_data_bcd ),
.y_data_bcd (y_data_bcd ),
.z_data_bcd (z_data_bcd ),
.oled_rst (oled_rst ),
.oled_dcn(oled_dcn   ),
.oled_clk (oled_clk),
.oled_dat (oled_dat)
);
```

endmodule

(2) 设计文件 MMA7660_driver.v

```
// *****
// File name : MMA7660_test
// Module name : MMA7660_driver
//
// Author : STEP
// Date : 2023.4.19
// Version : V 1.0
// Description : MMA7660 Driver
//
// Modification history
// //
// $Log$
//
// *****
// *****
// DEFINE MODULE PORT //
// *****
module MMA7660_driver
(
// INPUT
input clk, //系统时钟
input rst_n, //系统复位，低有效
// OUTPUT
output reg [7:0] xout, //读取数据
output reg [7:0] yout,
output reg [7:0] zout,
output i2c_scl, //I2C 总线 SCL
inout i2c_sda //I2C 总线 SDA
);
// *****
// DEFINE PARAMETER //
// *****
parameter CNT_NUM = 15;
localparam IDLE = 4'd0;
localparam MAIN = 4'd1;
localparam MODE1 = 4'd2;
localparam MODE2 = 4'd3;
```



```

localparam  START  =  4'd4;
localparam  WRITE  =  4'd5;
localparam  READ   =  4'd6;
localparam  STOP   =  4'd7;
localparam  DELAY  =  4'd8;
localparam  ACK    =  1'b0;
localparam  NACK   =  1'b1;
// REGS
reg          clk_400khz;
reg  [9:0]    cnt_400khz;
reg scl,sda,ack,ack_flag;
reg [3:0] cnt, cnt_main, cnt_mode1, cnt_mode2, cnt_start, cnt_write, cnt_read, cnt_stop;
reg [7:0] data_wr, dev_addr, dev_addr_r, reg_addr, data_r, reg_wr, read_data;
reg [23:0] cnt_delay, num_delay;
reg [3:0]  state, state_back;
// *****
//   MAIN CODE   //
// *****
//使用计数器分频产生 400KHz 时钟信号 clk_400khz
always@(posedge clk or negedge rst_n)
begin
    if(!rst_n) begin
        cnt_400khz <= 10'd0;
        clk_400khz <= 1'b0;
    end else if(cnt_400khz >= CNT_NUM-1) begin
        cnt_400khz <= 10'd0;
        clk_400khz <= ~clk_400khz;
    end else begin
        cnt_400khz <= cnt_400khz + 1'b1;
    end
end
always@(posedge clk_400khz or negedge rst_n)
begin
    if(!rst_n)
    begin    //如果按键复位，将相关数据初始化
        scl <= 1'd1; sda <= 1'd1; ack <= ACK; ack_flag <= 1'b0; cnt <= 1'b0;
        cnt_main <= 1'b0; cnt_mode1 <= 1'b0; cnt_mode2 <= 1'b0;
    end
end

```

```

cnt_start <= 1'b0; cnt_write <= 1'b0; cnt_read <= 1'b0; cnt_stop <= 1'b0;
cnt_delay <= 1'b0; num_delay <= 24'd48000;
state <= IDLE; state_back <= IDLE;
end else begin
    case(state)
        IDLE:begin //软件自复位，主要用于程序跑飞后的处理
            scl <= 1'd1; sda <= 1'd1; ack <= ACK; ack_flag <= 1'b0; cnt <= 1'b0;
            cnt_main <= 1'b0;
            cnt_start <= 1'b0; cnt_write <= 1'b0; cnt_read <= 1'b0; cnt_stop <= 1'b0;
            cnt_delay <= 1'b0; num_delay <= 24'd48000;
            state <= MAIN; state_back <= IDLE;
        end
        MAIN:begin
            if(cnt_main >= 4'd8)
                cnt_main <= 4'd2; //写完控制指令后循环读数据
            else
                cnt_main <= cnt_main + 1'b1;
            case(cnt_main)
                4'd0: begin dev_addr <= 8'h98; reg_addr <= 8'h07; reg_wr <= 8'h01;
state <= MODE1; end //写入数据
                4'd1: begin num_delay <= 24'd2000; state <= DELAY; end //5ms 延
时
                4'd2: begin dev_addr <= 8'h98; dev_addr_r <= 8'h99; reg_addr <= 8'h01;
state <= MODE2; end //读取数据
                4'd3: begin xout <= read_data; end
                4'd4: begin dev_addr <= 8'h98; dev_addr_r <= 8'h99; reg_addr <= 8'h02;
state <= MODE2; end //读取数据
                4'd5: begin yout <= read_data; end
                4'd6: begin dev_addr <= 8'h98; dev_addr_r <= 8'h99; reg_addr <= 8'h03;
state <= MODE2; end //读取数据
                4'd7: begin zout <= read_data; end //读取数据
                4'd8: begin num_delay <= 32'd48000; state <= DELAY; end
//120ms 延时
                default: state <= IDLE; //如果程序失控，进入 IDLE 自复位状态
            endcase
        end
        MODE1:begin //单次写操作

```

```

if(cnt_mode1 >= 3'd5)
cnt_mode1 <= 1'b0;    //对 START 中的子状态执行控制 cnt_start
else cnt_mode1 <= cnt_mode1 + 1'b1;
state_back <= MODE1;
case(cnt_mode1)
    3'd0:    begin state <= START; end    //I2C 通信时序中的 START
    3'd1:    begin data_wr <= dev_addr; state <= WRITE; end    // 设备地
址
    3'd2:    begin data_wr <= reg_addr; state <= WRITE; end    // 寄存器
地址
    3'd3:    begin data_wr <= reg_wr; state <= WRITE; end //写入数据
    3'd4:    begin state <= STOP; end    //I2C 通信时序中的 STOP
    3'd5:    begin state <= MAIN; end    //返回 MAIN
    default: state <= IDLE; //如果程序失控，进入 IDLE 自复位状态
endcase
end
MODE2:begin//单次读操作
if(cnt_mode2 >= 4'd8)
cnt_mode2 <= 1'b0;    //对 START 中的子状态执行控制 cnt_start
else
cnt_mode2 <= cnt_mode2 + 1'b1;
state_back <= MODE2;
case(cnt_mode2)
    4'd0:    begin state <= START; end    //I2C 通信时序中的 START
    4'd1:    begin data_wr <= dev_addr; state <= WRITE; end    // 设备地
址
    4'd2:    begin data_wr <= reg_addr; state <= WRITE; end    // 寄存器
地址
    4'd3:    begin state <= START; end    //I2C 通信时序中的 START
    4'd4:    begin data_wr <= dev_addr_r; state <= WRITE; end // 设备地
址
    4'd5:    begin ack <= NACK; state <= READ; end    //发送 NACK 信号，
读寄存器数据
    4'd6:    begin read_data <= data_r; end
    4'd7:    begin state <= STOP; end    //I2C 通信时序中的 STOP
    4'd8:    begin state <= MAIN; end    //返回 MAIN
    default: state <= IDLE; //如果程序失控，进入 IDLE 自复位状态

```

```

        endcase
    end
    START:begin //I2C 通信时序中的起始 START
        if(cnt_start >= 3'd5)
            cnt_start <= 1'b0; //对 START 中的子状态执行控制 cnt_start
        else
            cnt_start <= cnt_start + 1'b1;
            case(cnt_start)
                3'd0:    begin sda <= 1'b1; scl <= 1'b1; end    //将 SCL 和 SDA 拉高，
保持 4.7us 以上
                3'd1:    begin sda <= 1'b1; scl <= 1'b1; end    //clk_400khz 每个周期
2.5us，需要两个周期
                3'd2:    begin sda <= 1'b0; end //SDA 拉低到 SCL 拉低，保持 4.0us 以
上
                3'd3:    begin sda <= 1'b0; end //clk_400khz 每个周期 2.5us，需要两
个周期
                3'd4:    begin scl <= 1'b0; end //SCL 拉低，保持 4.7us 以上
                3'd5:    begin scl <= 1'b0; state <= state_back; end //clk_400khz 每个
周期 2.5us，需要两个周期，返回 MAIN
                default: state <= IDLE; //如果程序失控，进入 IDLE 自复位状态
            endcase
        end
    WRITE:begin //I2C 通信时序中的写操作 WRITE 和相应判断操作 ACK
        if(cnt <= 3'd6)
            begin //共需要发送 8bit 的数据，这里控制循环的次数
                if(cnt_write >= 3'd3)
                    begin
                        cnt_write <= 1'b0; cnt <= cnt + 1'b1;
                    end
                else
                    begin
                        cnt_write <= cnt_write + 1'b1; cnt <= cnt;
                    end
                end
            end
        else
            begin
                if(cnt_write >= 3'd7)

```

```

begin
    cnt_write <= 1'b0; cnt <= 1'b0;
end //两个变量都恢复初值
else
begin
    cnt_write <= cnt_write + 1'b1; cnt <= cnt;
end
end
case(cnt_write)
//按照 I2C 的时序传输数据
3'd0:    begin scl <= 1'b0; sda <= data_wr[7-cnt]; end //SCL 拉低，并
控制 SDA 输出对应的位
3'd1:    begin scl <= 1'b1; end //SCL 拉高，保持 4.0us 以上
3'd2:    begin scl <= 1'b1; end //clk_400khz 每个周期 2.5us，需要两
个周期
3'd3:    begin scl <= 1'b0; end //SCL 拉低，准备发送下 1bit 的数据
//获取从设备的响应信号并判断
3'd4:    begin sda <= 1'bz; end //释放 SDA 线，准备接收从设备的响
应信号
3'd5:    begin scl <= 1'b1; end //SCL 拉高，保持 4.0us 以上
3'd6:    begin ack_flag <= i2c_sda; end //获取从设备的响应信号并
判断
3'd7:    begin scl <= 1'b0; if(ack_flag)state <= state; else state <=
state_back; end //SCL 拉低，如果不应答循环写
default: state <= IDLE; //如果程序失控，进入 IDLE 自复位状态
endcase
end
READ:begin //I2C 通信时序中的读操作 READ 和返回 ACK 的操作
if(cnt <= 3'd6) begin //共需要接收 8bit 的数据，这里控制循环的次数
if(cnt_read >= 3'd3) begin cnt_read <= 1'b0; cnt <= cnt + 1'b1; end
else begin cnt_read <= cnt_read + 1'b1; cnt <= cnt; end
end else begin
if(cnt_read >= 3'd7) begin cnt_read <= 1'b0; cnt <= 1'b0; end //两个变
量都恢复初值
else begin cnt_read <= cnt_read + 1'b1; cnt <= cnt; end
end
case(cnt_read)

```

```

//按照 I2C 的时序接收数据
3'd0:   begin scl <= 1'b0; sda <= 1'bz; end   //SCL 拉低，释放 SDA
线，准备接收从设备数据

3'd1:   begin scl <= 1'b1; end   //SCL 拉高，保持 4.0us 以上
3'd2:   begin data_r[7-cnt] <= i2c_sda; end   //读取从设备返回的数
据

3'd3:   begin scl <= 1'b0; end   //SCL 拉低，准备接收下 1bit 的数据
//向从设备发送响应信号
3'd4:   begin sda <= ack; end   //发送响应信号，将前面接收的数据锁
存

3'd5:   begin scl <= 1'b1; end   //SCL 拉高，保持 4.0us 以上
3'd6:   begin scl <= 1'b1; end   //SCL 拉高，保持 4.0us 以上
3'd7:   begin scl <= 1'b0; state <= state_back; end //SCL 拉低，返回
MAIN 状态

default: state <= IDLE; //如果程序失控，进入 IDLE 自复位状态
endcase
end
STOP:begin //I2C 通信时序中的结束 STOP
if(cnt_stop >= 3'd5) cnt_stop <= 1'b0; //对 STOP 中的子状态执行控制
cnt_stop

else cnt_stop <= cnt_stop + 1'b1;
case(cnt_stop)
3'd0:   begin sda <= 1'b0; end //SDA 拉低，准备 STOP
3'd1:   begin sda <= 1'b0; end //SDA 拉低，准备 STOP
3'd2:   begin scl <= 1'b1; end //SCL 提前 SDA 拉高 4.0us
3'd3:   begin scl <= 1'b1; end //SCL 提前 SDA 拉高 4.0us
3'd4:   begin sda <= 1'b1; end //SDA 拉高
3'd5:   begin sda <= 1'b1; state <= state_back; end //完成 STOP 操
作，返回 MAIN 状态

default: state <= IDLE; //如果程序失控，进入 IDLE 自复位状态
endcase
end
DELAY:begin //12ms 延时
if(cnt_delay >= num_delay) begin
cnt_delay <= 1'b0;
state <= MAIN;
end else cnt_delay <= cnt_delay + 1'b1;

```

```

                end
            default;;
        endcase
    end
end
assign  i2c_scl = scl; //对 SCL 端口赋值
assign  i2c_sda = sda; //对 SDA 端口赋值
endmodule

```

(3) 设计文件 OLED_driver.v

```

// *****
// File name : MMA7660_test
// Module name : OLED_driver
//
// Author      : STEP
// Date        : 2023.4.19
// Version     : V 1.0
// Description  : OLED Driver
//
// Modification history
// //
// $Log$
//
// *****
// *****
//  DEFINE MODULE PORT  //
// *****
module OLED_driver
(
// INPUT
input clk, //
input rst_n, //
input  [7:0]  x_data_bcd, //写入数据
input  [7:0]  y_data_bcd,
input  [7:0]  z_data_bcd,
// OUTPUT
output reg oled_rst, //
output reg oled_dcn, //

```

```

output  reg oled_clk, //
output  reg oled_dat
);
// *****
//  DEFINE PARAMETER  //
// *****
localparam INIT_DEPTH = 16'd28; //
localparam IDLE = 6'h1, MAIN = 6'h2, INIT = 6'h4, SCAN = 6'h8, WRITE = 6'h10, DELAY = 6'h20;
localparam HIGH = 1'b1, LOW = 1'b0;
localparam DATA = 1'b1, CMD = 1'b0;
// REGS
reg [7:0] cmd [27:0];
reg [39:0] mem [122:0];
reg [7:0] y_p, x_ph, x_pl;
reg [(8*21-1):0] char;
reg [7:0] num, char_reg;          //
reg [4:0] cnt_main, cnt_init, cnt_scan, cnt_write;
reg [15:0] num_delay, cnt_delay, cnt;
reg [5:0] state, state_back;
// *****
//  MAIN CODE  //
// *****
always@(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        cnt_main <= 1'b0; cnt_init <= 1'b0; cnt_scan <= 1'b0; cnt_write <= 1'b0;
        y_p <= 1'b0; x_ph <= 1'b0; x_pl <= 1'b0;
        num <= 1'b0; char <= 1'b0; char_reg <= 1'b0;
        num_delay <= 16'd5; cnt_delay <= 1'b0; cnt <= 1'b0;
        oled_rst <= HIGH; oled_dcn <= CMD; oled_clk <= HIGH; oled_dat <= LOW;
        state <= IDLE; state_back <= IDLE;
    end else begin
        case(state)
            IDLE:begin
                cnt_main <= 1'b0; cnt_init <= 1'b0; cnt_scan <= 1'b0; cnt_write <= 1'b0;
                y_p <= 1'b0; x_ph <= 1'b0; x_pl <= 1'b0;
                num <= 1'b0; char <= 1'b0; char_reg <= 1'b0;
                num_delay <= 16'd5; cnt_delay <= 1'b0; cnt <= 1'b0;

```



```

oled_rst <= HIGH; oled_dcn <= CMD; oled_clk <= HIGH; oled_dat <= LOW;
state <= MAIN; state_back <= MAIN;

end

MAIN:begin
    if(cnt_main >= 5'd14) cnt_main <= 5'd9;
    else cnt_main <= cnt_main + 1'b1;
    case(cnt_main)    //MAIN 状态
        5'd0:    begin state <= INIT; end
        5'd1:    begin y_p <= 8'hb0; x_ph <= 8'h3 ; x_pl <= 8'h00; num <= 5'd16;
char <= "x:    ";state <= SCAN; end
        5'd2:    begin y_p <= 8'hb1; x_ph <= 8'h3 ; x_pl <= 8'h00; num <= 5'd16;
char <= "y:    ";state <= SCAN; end
        5'd3:    begin y_p <= 8'hb2; x_ph <= 8'h3 ; x_pl <= 8'h00; num <= 5'd16;
char <= "z:    ";state <= SCAN; end
        5'd4:    begin y_p <= 8'hb3; x_ph <= 8'h10; x_pl <= 8'h00; num <=
5'd16; char <= "    ";state <= SCAN; end
        5'd5:    begin y_p <= 8'hb4; x_ph <= 8'h10; x_pl <= 8'h00; num <=
5'd16; char <= "    ";state <= SCAN; end
        5'd6:    begin y_p <= 8'hb5; x_ph <= 8'h10; x_pl <= 8'h00; num <=
5'd16; char <= "    ";state <= SCAN; end
        5'd7:    begin y_p <= 8'hb6; x_ph <= 8'h10; x_pl <= 8'h00; num <=
5'd16; char <= "    ";state <= SCAN; end
        5'd8:    begin y_p <= 8'hb7; x_ph <= 8'h10; x_pl <= 8'h00; num <=
5'd16; char <= "    ";state <= SCAN; end
        5'd9:    begin y_p <= 8'hb0; x_ph <= 8'h12; x_pl <= 8'h08; num <= 5'd 1;
char <= x_data_bcd[7:4]; state <= SCAN; end
        5'd10:    begin y_p <= 8'hb0; x_ph <= 8'h12; x_pl <= 8'h00; num <= 5'd 1;
char <= x_data_bcd[3:0]; state <= SCAN; end
        5'd11:    begin y_p <= 8'hb1; x_ph <= 8'h12; x_pl <= 8'h08; num <= 5'd 1;
char <= y_data_bcd[7:4]; state <= SCAN; end
        5'd12:    begin y_p <= 8'hb1; x_ph <= 8'h12; x_pl <= 8'h00; num <= 5'd 1;
char <= y_data_bcd[3:0]; state <= SCAN; end
        5'd13:    begin y_p <= 8'hb2; x_ph <= 8'h12; x_pl <= 8'h08; num <= 5'd 1;
char <= z_data_bcd[7:4]; state <= SCAN; end
        5'd14:    begin y_p <= 8'hb2; x_ph <= 8'h12; x_pl <= 8'h00; num <= 5'd 1;
char <= z_data_bcd[3:0]; state <= SCAN; end
        default: state <= IDLE;
    endcase
end

```

```

        endcase
    end
    INIT:begin    //
        case(cnt_init)
            5'd0:    begin oled_rst <= LOW; cnt_init <= cnt_init + 1'b1; end    //
            5'd1:    begin num_delay <= 16'd25000; state <= DELAY; state_back <=
INIT; cnt_init <= cnt_init + 1'b1; end    //
            5'd2:    begin oled_rst <= HIGH; cnt_init <= cnt_init + 1'b1; end    //
            5'd3:    begin num_delay <= 16'd25000; state <= DELAY; state_back <=
INIT; cnt_init <= cnt_init + 1'b1; end    //
            5'd4:    begin
                        if(cnt>=INIT_DEPTH) begin //
                            cnt <= 1'b0;
                            cnt_init <= cnt_init + 1'b1;
                        end else begin
                            cnt <= cnt + 1'b1; num_delay <= 16'd5;
                            oled_dcn <= CMD; char_reg <= cmd[cnt]; state <=
WRITE; state_back <= INIT;
                        end
                    end
            5'd5:    begin cnt_init <= 1'b0; state <= MAIN; end //
            default: state <= IDLE;
        endcase
    end
    SCAN:begin    //
        if(cnt_scan == 5'd11) begin
            if(num) cnt_scan <= 5'd3;
            else cnt_scan <= cnt_scan + 1'b1;
        end else if(cnt_scan == 5'd12) cnt_scan <= 1'b0;
        else cnt_scan <= cnt_scan + 1'b1;
        case(cnt_scan)
            5'd 0:    begin oled_dcn <= CMD; char_reg <= y_p; state <= WRITE;
state_back <= SCAN; end        //
            5'd 1:    begin oled_dcn <= CMD; char_reg <= x_pl; state <= WRITE;
state_back <= SCAN; end        //
            5'd 2:    begin oled_dcn <= CMD; char_reg <= x_ph; state <= WRITE;
state_back <= SCAN; end        //

```

```

5'd 3:   begin num <= num - 1'b1;end
5'd 4:   begin oled_dcn <= DATA; char_reg <= 8'h00; state <= WRITE;
state_back <= SCAN; end //
5'd 5:   begin oled_dcn <= DATA; char_reg <= 8'h00; state <= WRITE;
state_back <= SCAN; end //
5'd 6:   begin oled_dcn <= DATA; char_reg <= 8'h00; state <= WRITE;
state_back <= SCAN; end //
5'd 7:   begin      oled_dcn      <=      DATA;      char_reg      <=
mem[char[(num*8)+:8]][39:32]; state <= WRITE; state_back <= SCAN; end
5'd 8:   begin      oled_dcn      <=      DATA;      char_reg      <=
mem[char[(num*8)+:8]][31:24]; state <= WRITE; state_back <= SCAN; end
5'd 9:   begin      oled_dcn      <=      DATA;      char_reg      <=
mem[char[(num*8)+:8]][23:16]; state <= WRITE; state_back <= SCAN; end
5'd10:  begin      oled_dcn      <=      DATA;      char_reg      <=
mem[char[(num*8)+:8]][15: 8]; state <= WRITE; state_back <= SCAN; end
5'd11:  begin      oled_dcn      <=      DATA;      char_reg      <=
mem[char[(num*8)+:8]][ 7: 0]; state <= WRITE; state_back <= SCAN; end
5'd12:  begin state <= MAIN; end
        default: state <= IDLE;
    endcase
end
WRITE:begin //
    if(cnt_write >= 5'd16) cnt_write <= 1'b0;
    else cnt_write <= cnt_write + 1'b1;
    case(cnt_write)
        5'd 0:   begin oled_clk <= LOW; oled_dat <= char_reg[7]; end //
        5'd 1:   begin oled_clk <= HIGH; end
        5'd 2:   begin oled_clk <= LOW; oled_dat <= char_reg[6]; end
        5'd 3:   begin oled_clk <= HIGH; end
        5'd 4:   begin oled_clk <= LOW; oled_dat <= char_reg[5]; end
        5'd 5:   begin oled_clk <= HIGH; end
        5'd 6:   begin oled_clk <= LOW; oled_dat <= char_reg[4]; end
        5'd 7:   begin oled_clk <= HIGH; end
        5'd 8:   begin oled_clk <= LOW; oled_dat <= char_reg[3]; end
        5'd 9:   begin oled_clk <= HIGH; end
        5'd10:  begin oled_clk <= LOW; oled_dat <= char_reg[2]; end
        5'd11:  begin oled_clk <= HIGH; end

```

```

                    5'd12:  begin oled_clk <= LOW; oled_dat <= char_reg[1]; end
                    5'd13:  begin oled_clk <= HIGH; end
                    5'd14:  begin oled_clk <= LOW; oled_dat <= char_reg[0]; end    //
                    5'd15:  begin oled_clk <= HIGH; end
                    5'd16:  begin state <= DELAY; end    //
                    default: state <= IDLE;
                endcase
            end
        DELAY:begin //
            if(cnt_delay >= num_delay) begin
                cnt_delay <= 16'd0; state <= state_back;
            end else cnt_delay <= cnt_delay + 1'b1;
        end
        default:state <= IDLE;
    endcase
end
//
always@(negedge rst_n)

begin
    cmd[ 0] = {8'hae};
    cmd[ 1] = {8'h00};
    cmd[ 2] = {8'h10};
    cmd[ 3] = {8'h40};
    cmd[ 4] = {8'h81};
    cmd[ 5] = {8'hcf};
    cmd[ 6] = {8'ha1};
    cmd[ 7] = {8'hc8};
    cmd[ 8] = {8'ha6};
    cmd[ 9] = {8'ha8};
    cmd[10] = {8'h3f};
    cmd[11] = {8'hd3};
    cmd[12] = {8'h00};
    cmd[13] = {8'hd5};
    cmd[14] = {8'h80};

```

```

cmd[15] = {8'hd9};
cmd[16] = {8'hf1};
cmd[17] = {8'hda};
cmd[18] = {8'h12};
cmd[19] = {8'hdb};
cmd[20] = {8'h40};
cmd[21] = {8'h20};
cmd[22] = {8'h02};
    cmd[23] = {8'h8d};
cmd[24] = {8'h14};
    cmd[25] = {8'ha4};
    cmd[26] = {8'ha6};
cmd[27] = {8'haf};
end

always@(negedge rst_n)
//initial
begin
    mem[ 0] = {8'h3E, 8'h51, 8'h49, 8'h45, 8'h3E};    // 48  0
    mem[ 1] = {8'h00, 8'h42, 8'h7F, 8'h40, 8'h00};    // 49  1
    mem[ 2] = {8'h42, 8'h61, 8'h51, 8'h49, 8'h46};    // 50  2
    mem[ 3] = {8'h21, 8'h41, 8'h45, 8'h4B, 8'h31};    // 51  3
    mem[ 4] = {8'h18, 8'h14, 8'h12, 8'h7F, 8'h10};    // 52  4
    mem[ 5] = {8'h27, 8'h45, 8'h45, 8'h45, 8'h39};    // 53  5
    mem[ 6] = {8'h3C, 8'h4A, 8'h49, 8'h49, 8'h30};    // 54  6
    mem[ 7] = {8'h01, 8'h71, 8'h09, 8'h05, 8'h03};    // 55  7
    mem[ 8] = {8'h36, 8'h49, 8'h49, 8'h49, 8'h36};    // 56  8
    mem[ 9] = {8'h06, 8'h49, 8'h49, 8'h29, 8'h1E};    // 57  9
    mem[10] = {8'h7C, 8'h12, 8'h11, 8'h12, 8'h7C};    // 65  A
    mem[11] = {8'h7F, 8'h49, 8'h49, 8'h49, 8'h36};    // 66  B
    mem[12] = {8'h3E, 8'h41, 8'h41, 8'h41, 8'h22};    // 67  C
    mem[13] = {8'h7F, 8'h41, 8'h41, 8'h22, 8'h1C};    // 68  D
    mem[14] = {8'h7F, 8'h49, 8'h49, 8'h49, 8'h41};    // 69  E
    mem[15] = {8'h7F, 8'h09, 8'h09, 8'h09, 8'h01};    // 70  F

    mem[32] = {8'h00, 8'h00, 8'h00, 8'h00, 8'h00};    // 32  sp
    mem[33] = {8'h00, 8'h00, 8'h2f, 8'h00, 8'h00};    // 33  !

```

```

mem[ 34] = {8'h00, 8'h07, 8'h00, 8'h07, 8'h00};    // 34
mem[ 35] = {8'h14, 8'h7f, 8'h14, 8'h7f, 8'h14};    // 35  #
mem[ 36] = {8'h24, 8'h2a, 8'h7f, 8'h2a, 8'h12};    // 36  $
mem[ 37] = {8'h62, 8'h64, 8'h08, 8'h13, 8'h23};    // 37  %
mem[ 38] = {8'h36, 8'h49, 8'h55, 8'h22, 8'h50};    // 38  &
mem[ 39] = {8'h00, 8'h05, 8'h03, 8'h00, 8'h00};    // 39  '
mem[ 40] = {8'h00, 8'h1c, 8'h22, 8'h41, 8'h00};    // 40  (
mem[ 41] = {8'h00, 8'h41, 8'h22, 8'h1c, 8'h00};    // 41  )
mem[ 42] = {8'h14, 8'h08, 8'h3E, 8'h08, 8'h14};    // 42  *
mem[ 43] = {8'h08, 8'h08, 8'h3E, 8'h08, 8'h08};    // 43  +
mem[ 44] = {8'h00, 8'h00, 8'hA0, 8'h60, 8'h00};    // 44  ,
mem[ 45] = {8'h08, 8'h08, 8'h08, 8'h08, 8'h08};    // 45  -
mem[ 46] = {8'h00, 8'h60, 8'h60, 8'h00, 8'h00};    // 46  .
mem[ 47] = {8'h20, 8'h10, 8'h08, 8'h04, 8'h02};    // 47  /
mem[ 48] = {8'h3E, 8'h51, 8'h49, 8'h45, 8'h3E};    // 48  0
mem[ 49] = {8'h00, 8'h42, 8'h7F, 8'h40, 8'h00};    // 49  1
mem[ 50] = {8'h42, 8'h61, 8'h51, 8'h49, 8'h46};    // 50  2
mem[ 51] = {8'h21, 8'h41, 8'h45, 8'h4B, 8'h31};    // 51  3
mem[ 52] = {8'h18, 8'h14, 8'h12, 8'h7F, 8'h10};    // 52  4
mem[ 53] = {8'h27, 8'h45, 8'h45, 8'h45, 8'h39};    // 53  5
mem[ 54] = {8'h3C, 8'h4A, 8'h49, 8'h49, 8'h30};    // 54  6
mem[ 55] = {8'h01, 8'h71, 8'h09, 8'h05, 8'h03};    // 55  7
mem[ 56] = {8'h36, 8'h49, 8'h49, 8'h49, 8'h36};    // 56  8
mem[ 57] = {8'h06, 8'h49, 8'h49, 8'h29, 8'h1E};    // 57  9
mem[ 58] = {8'h00, 8'h36, 8'h36, 8'h00, 8'h00};    // 58  :
mem[ 59] = {8'h00, 8'h56, 8'h36, 8'h00, 8'h00};    // 59  ;
mem[ 60] = {8'h08, 8'h14, 8'h22, 8'h41, 8'h00};    // 60  <
mem[ 61] = {8'h14, 8'h14, 8'h14, 8'h14, 8'h14};    // 61  =
mem[ 62] = {8'h00, 8'h41, 8'h22, 8'h14, 8'h08};    // 62  >
mem[ 63] = {8'h02, 8'h01, 8'h51, 8'h09, 8'h06};    // 63  ?
mem[ 64] = {8'h32, 8'h49, 8'h59, 8'h51, 8'h3E};    // 64  @
mem[ 65] = {8'h7C, 8'h12, 8'h11, 8'h12, 8'h7C};    // 65  A
mem[ 66] = {8'h7F, 8'h49, 8'h49, 8'h49, 8'h36};    // 66  B
mem[ 67] = {8'h3E, 8'h41, 8'h41, 8'h41, 8'h22};    // 67  C
mem[ 68] = {8'h7F, 8'h41, 8'h41, 8'h22, 8'h1C};    // 68  D
mem[ 69] = {8'h7F, 8'h49, 8'h49, 8'h49, 8'h41};    // 69  E
mem[ 70] = {8'h7F, 8'h09, 8'h09, 8'h09, 8'h01};    // 70  F

```

```

mem[ 71] = {8'h3E, 8'h41, 8'h49, 8'h49, 8'h7A};    // 71  G
mem[ 72] = {8'h7F, 8'h08, 8'h08, 8'h08, 8'h7F};    // 72  H
mem[ 73] = {8'h00, 8'h41, 8'h7F, 8'h41, 8'h00};    // 73  I
mem[ 74] = {8'h20, 8'h40, 8'h41, 8'h3F, 8'h01};    // 74  J
mem[ 75] = {8'h7F, 8'h08, 8'h14, 8'h22, 8'h41};    // 75  K
mem[ 76] = {8'h7F, 8'h40, 8'h40, 8'h40, 8'h40};    // 76  L
mem[ 77] = {8'h7F, 8'h02, 8'h0C, 8'h02, 8'h7F};    // 77  M
mem[ 78] = {8'h7F, 8'h04, 8'h08, 8'h10, 8'h7F};    // 78  N
mem[ 79] = {8'h3E, 8'h41, 8'h41, 8'h41, 8'h3E};    // 79  O
mem[ 80] = {8'h7F, 8'h09, 8'h09, 8'h09, 8'h06};    // 80  P
mem[ 81] = {8'h3E, 8'h41, 8'h51, 8'h21, 8'h5E};    // 81  Q
mem[ 82] = {8'h7F, 8'h09, 8'h19, 8'h29, 8'h46};    // 82  R
mem[ 83] = {8'h46, 8'h49, 8'h49, 8'h49, 8'h31};    // 83  S
mem[ 84] = {8'h01, 8'h01, 8'h7F, 8'h01, 8'h01};    // 84  T
mem[ 85] = {8'h3F, 8'h40, 8'h40, 8'h40, 8'h3F};    // 85  U
mem[ 86] = {8'h1F, 8'h20, 8'h40, 8'h20, 8'h1F};    // 86  V
mem[ 87] = {8'h3F, 8'h40, 8'h38, 8'h40, 8'h3F};    // 87  W
mem[ 88] = {8'h63, 8'h14, 8'h08, 8'h14, 8'h63};    // 88  X
mem[ 89] = {8'h07, 8'h08, 8'h70, 8'h08, 8'h07};    // 89  Y
mem[ 90] = {8'h61, 8'h51, 8'h49, 8'h45, 8'h43};    // 90  Z
mem[ 91] = {8'h00, 8'h7F, 8'h41, 8'h41, 8'h00};    // 91  [
mem[ 92] = {8'h55, 8'h2A, 8'h55, 8'h2A, 8'h55};    // 92  .
mem[ 93] = {8'h00, 8'h41, 8'h41, 8'h7F, 8'h00};    // 93  ]
mem[ 94] = {8'h04, 8'h02, 8'h01, 8'h02, 8'h04};    // 94  ^
mem[ 95] = {8'h40, 8'h40, 8'h40, 8'h40, 8'h40};    // 95  _
mem[ 96] = {8'h00, 8'h01, 8'h02, 8'h04, 8'h00};    // 96  '
mem[ 97] = {8'h20, 8'h54, 8'h54, 8'h54, 8'h78};    // 97  a
mem[ 98] = {8'h7F, 8'h48, 8'h44, 8'h44, 8'h38};    // 98  b
mem[ 99] = {8'h38, 8'h44, 8'h44, 8'h44, 8'h20};    // 99  c
mem[100] = {8'h38, 8'h44, 8'h44, 8'h48, 8'h7F};    // 100 d
mem[101] = {8'h38, 8'h54, 8'h54, 8'h54, 8'h18};    // 101 e
mem[102] = {8'h08, 8'h7E, 8'h09, 8'h01, 8'h02};    // 102 f
mem[103] = {8'h18, 8'hA4, 8'hA4, 8'hA4, 8'h7C};    // 103 g
mem[104] = {8'h7F, 8'h08, 8'h04, 8'h04, 8'h78};    // 104 h
mem[105] = {8'h00, 8'h44, 8'h7D, 8'h40, 8'h00};    // 105 i
mem[106] = {8'h40, 8'h80, 8'h84, 8'h7D, 8'h00};    // 106 j
mem[107] = {8'h7F, 8'h10, 8'h28, 8'h44, 8'h00};    // 107 k

```

```

        mem[108] = {8'h00, 8'h41, 8'h7F, 8'h40, 8'h00};    // 108 l
        mem[109] = {8'h7C, 8'h04, 8'h18, 8'h04, 8'h78};    // 109 m
        mem[110] = {8'h7C, 8'h08, 8'h04, 8'h04, 8'h78};    // 110 n
        mem[111] = {8'h38, 8'h44, 8'h44, 8'h44, 8'h38};    // 111 o
        mem[112] = {8'hFC, 8'h24, 8'h24, 8'h24, 8'h18};    // 112 p
        mem[113] = {8'h18, 8'h24, 8'h24, 8'h18, 8'hFC};    // 113 q
        mem[114] = {8'h7C, 8'h08, 8'h04, 8'h04, 8'h08};    // 114 r
        mem[115] = {8'h48, 8'h54, 8'h54, 8'h54, 8'h20};    // 115 s
        mem[116] = {8'h04, 8'h3F, 8'h44, 8'h40, 8'h20};    // 116 t
        mem[117] = {8'h3C, 8'h40, 8'h40, 8'h20, 8'h7C};    // 117 u
        mem[118] = {8'h1C, 8'h20, 8'h40, 8'h20, 8'h1C};    // 118 v
        mem[119] = {8'h3C, 8'h40, 8'h30, 8'h40, 8'h3C};    // 119 w
        mem[120] = {8'h44, 8'h28, 8'h10, 8'h28, 8'h44};    // 120 x
        mem[121] = {8'h1C, 8'hA0, 8'hA0, 8'hA0, 8'h7C};    // 121 y
        mem[122] = {8'h44, 8'h64, 8'h54, 8'h4C, 8'h44};    // 122 z

    end

endmodule

```

(4) 设计文件 Calculate.v

```

// *****
// File name : MMA7660_test
// Module name : Calculate
//
// Author      : STEP
// Date        : 2023.4.19
// Version     : V 1.0
// Description  : Calculate module
//
// Modification history
// //
// $Log$
//
// *****_
// *****
// DEFINE MODULE PORT //
// *****
module Calculate

```



```

    (
// INPUT
input          rst_n,          //复位信号
input          [7:0]  xout,    //写入数据
input          [7:0]  yout,
input          [7:0]  zout,
// OUTPUT
output         [7:0]  x_data_bcd,
output         [7:0]  y_data_bcd,
output         [7:0]  z_data_bcd
);

    // *****
    // INSTANCE MODULE    //
    // *****

bin_to_bcd u1
(
.rst_n          (rst_n          ),    //系统复位，低有效
.bin_code       (xout          ),    //需要进行 BCD 转码的二进制数据
.bcd_code       (x_data_bcd    )    //转码后的 BCD 码型数据输出
);

bin_to_bcd u2
(
.rst_n          (rst_n          ),    //系统复位，低有效
.bin_code       (yout          ),    //需要进行 BCD 转码的二进制数据
.bcd_code       (y_data_bcd    )    //转码后的 BCD 码型数据输出
);

bin_to_bcd u3
(
.rst_n          (rst_n          ),    //系统复位，低有效
.bin_code       (zout          ),    //需要进行 BCD 转码的二进制数据
.bcd_code       (z_data_bcd    )    //转码后的 BCD 码型数据输出
);
endmodule

```

(5) 设计文件 bin_to_bcd.v

```
// *****
```

```

// File name : MMA7660_test
// Module name : bin_to_bcd
//
// Author      : STEP
// Date        : 2023.4.19
// Version     : V 1.0
// Description  : bin_to_bcd module
//
// Modification history
// //
// $Log$
//
// *****_
// *****
//  DEFINE MODULE PORT  //
// *****
module bin_to_bcd
(
// INPUT
input          rst_n,    //系统复位，低有效
input          [7:0]    bin_code, //需要进行 BCD 转码的二进制数据
// OUTPUT
output reg [7:0]    bcd_code //转码后的 BCD 码型数据输出
);

/*
此模块为了将 ADC 采样的数据转换为我们常用的十进制显示而存在，
主要知识涉及数学中不同制式数据的转换，详细原理这里不做介绍，去百度搜索<FPGA 二进制转
BCD 码>可得
*/
// REGS
reg      [15:0]    shift_reg;
always@(bin_code or rst_n)begin
    shift_reg = {8'h0,bin_code};
    if(!rst_n) bcd_code = 0;
    else begin
        repeat(8) begin //循环 8 次

```

```

        //BCD 码各位数据作满 5 加 3 操作,
        if (shift_reg[11:8] >= 5) shift_reg[11:8] = shift_reg[11:8] + 2'b11;
        if (shift_reg[15:12] >= 5) shift_reg[15:12] = shift_reg[15:12] + 2'b11;
        shift_reg = shift_reg << 1;

    end

    bcd_code = shift_reg[15:8];

end

endmodule

//module bin2bcd
//#( parameter          W = 18)  // input width
// ( input      [W-1      :0] bin    ,  // binary
//   output reg [W+(W-4)/3:0] bcd    ); // bcd {...,thousands,hundreds,tens,ones}

// integer i,j;

//  always @(bin) begin
//      for(i = 0; i <= W+(W-4)/3; i = i+1) bcd[i] = 0;          // initialize with zeros
//      bcd[W-1:0] = bin;                                         // initialize with input vector
//      for(i = 0; i <= W-4; i = i+1)                             // iterate on structure depth
//          for(j = 0; j <= i/3; j = j+1)                         // iterate on structure width
//              if (bcd[W-i+4*j -: 4] > 4)                        // if > 4
//                  bcd[W-i+4*j -: 4] = bcd[W-i+4*j -: 4] + 4'd3; // add 3
//      end

//endmodule

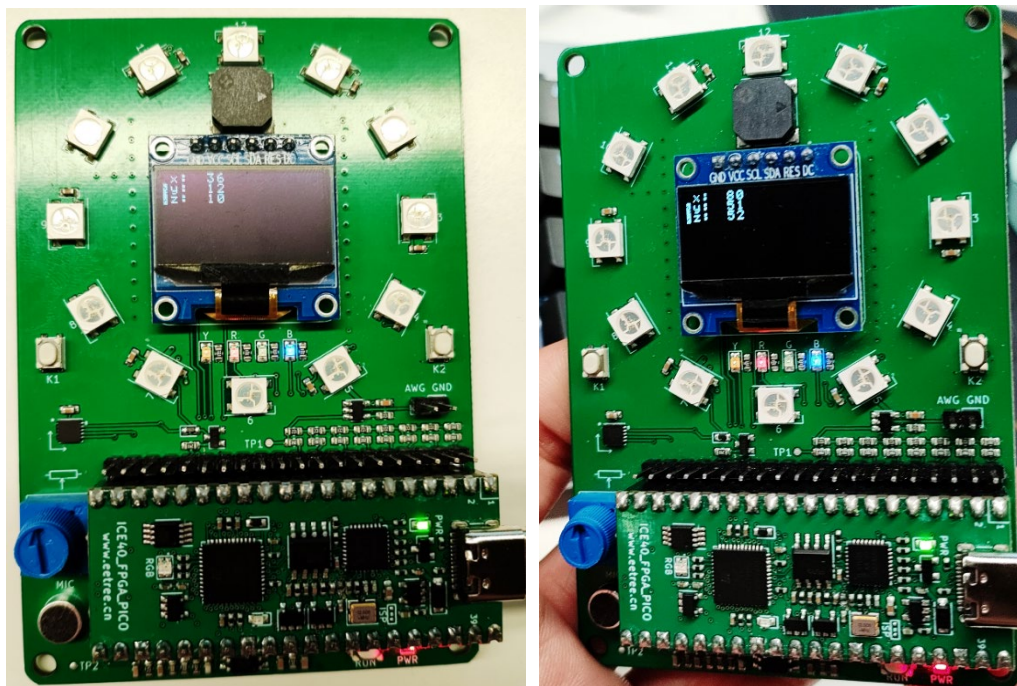
```

5.1.4 实验步骤

- 1.打开 Ridian, 建立工程。
- 2.新建 Verilog HDL 设计文件, 并键入设计代码, 包括添加需要调用的模块文件。
- 3.构建并输出编程文件, 烧写至 FPGA 的 Flash 之中。

5.1.5 实验现象

实验结果如下：



左图为板子平放在桌面上得到的坐标信息，右图为斜放板子得到的坐标信息。

版本信息

版本编号	修改日期	修改
V1.0	2023/4/3	初始版本